

Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales

Carrera Ingeniería en Computación

Proyecto integrador para la obtención del título de grado de Ingeniero en Computación

---

COMUNICACIÓN DESDE ESPACIO DE USUARIO A ESPACIO DE KERNEL MEDIANTE METADATOS EN ARCHIVOS ELF

Alumnos

Cancinos, José María

DNI: 40328011 - Email: jmcancinos@mi.unc.edu.ar - Tel: 3875002201

González, Julián

DNI: 40443147 - Email: julian.agustin.gonzalez@mi.unc.edu.ar - Tel: 2995125906

Director

Ing. Eschoyez, Maximiliano Andrés

Co-Director

Ing. Papp, Nicolás

Córdoba, República Argentina

Octubre / 2022

Dedicatoria

El presente trabajo está dedicado a nuestras familias.

Agradecimientos

Agradecemos a nuestras familias y amigos por su apoyo a lo largo de toda la carrera.

Agradecimiento especial a nuestros directores, Ing. Maximiliano Eschoyez e Ing Nicolás Papp, por el tiempo y acompañamiento brindado en esta etapa final.

Reconocimiento al cuerpo de profesores y el resto del personal de la Facultad de Ciencias Exactas, Físicas y Naturales de la Universidad Nacional de Córdoba, por la oportunidad de realizar esta carrera de grado.

Resumen

Los lenguajes de programación, y los programas desarrollados con ellos, han ido evolucionado para proveer soluciones a problemas de gran complejidad a partir de un mayor nivel de expresividad, mayor abstracción y mayor facilidad de uso al programador que emplea los mismos.

La relación programador con el lenguaje de programación es, sin dudas, bastante fuerte ya que es este el usado para generar el programa que responderá al problema que el desarrollador desea atender.

El programador y el compilador se relacionan al responder a las instrucciones de compilación del código por parte del primero. Sumado a ello, el desarrollador puede elegir modos de mejorar la *performance* del programa generado utilizando posibles opciones de optimización en un compilador.

El compilador y el sistema operativo y su hardware se relacionan debido a que el compilador transforma el código en un programa destinado a un sistema y una arquitectura determinada.

Finalmente, es el sistema operativo quien lleva a cabo el control del programa desarrollado.

De lo anterior, se observa que la relación entre el programador y el sistema operativo está poco explotada, limitándose a una relación indirecta al momento en que el primero ejecuta el programa, y este comienza su carga en el sistema operativo.

En este proyecto, presentamos un mecanismo de comunicación utilizado para enviar datos, en forma de metadatos insertados en archivos ejecutables, desde espacio de usuario a espacio de *kernel* y lograr así una mayor asociatividad entre ambos.

Se presenta y detalla la implementación de un prototipo genérico y abstracto que permita comunicar el programador con la *kernel* del sistema operativo.

El programador que utilice este mecanismo podrá insertar los datos deseados dentro de la estructura del archivo ejecutable resultante del programa que él mismo desarrolla en tiempo de compilación en espacio de usuario.

Posteriormente, como resultado de ejecutar el archivo ejecutable que contiene los datos insertados, se dará lugar a las fases de lectura del archivo, decodificación y almacenaje de los datos insertados como parte del espacio de memoria de la *kernel*.

La implementación consiste en herramientas complementarias a las *suites* de compilación *GNU GCC* y *CLANG/LLVM*, en espacio de usuario: y parches del sistema operativo en espacio de *kernel*.

Áreas Temáticas del Proyecto Integrador:

Las Áreas Temáticas son: Compiladores, Archivos Ejecutables, *Kernel* del Sistema Operativo.

Asignaturas:

Práctica y Construcción de Compiladores, Sistemas Operativos, Sistemas de Computación, Ingeniería de *Software*.

Palabras Claves:

Herramientas de software, sistemas operativos, entorno en tiempo de ejecución.

Abstract

Programming languages, and the programs developed using them, have developed to solve more complex problems due to the adoption of a more expressive power, greater abstraction capabilities and easier and a higher programmer friendly usage.

The relationship between the programmer and the programming language is very straightforward because the last one is used by the developer to obtain the program that is answering the problem.

The programmer and the compiler are directly related when the first mentioned instructs the compiling of the code. Also, the developer is able to choose among optimization options in a compiler to improve the performance of the compiled program.

The compiler and both, the operating system and the hardware, are directly connected because the code is transformed into a program for a specific system and architecture.

Finally, the operating system is in charge of the control of the program.

Therefore, the relationship between the programmer and the operating system is indirect and limited only to the moment when the programmer could execute the program and then, the last one is loaded in the operating system.

We present in this report a communication mechanism designed to send data, as metadata embedded in executable files, from user space to kernel space.

The implementation of a generic prototype version of the previously mentioned mechanism is detailed in this report.

A developer who uses this mechanism could insert data inside the structure of the executable, in compilation time. This file is obtained as the result of the compilation of the code that the developer has created in user space.

Then, due to the execution of the previously mentioned metadata embedded executable file and its loading into the operating system; the executable file parsing phase, the data decoding phase and inserted data storing phase occur in kernel space.

This project consists of software developed in both user and kernel space. *GNU GCC* and *CLANG/LLVM* compilation suites plug-ins have been developed to perform the insertion of the data in user space. Whereas, in kernel space, operating system patches have been generated to decode the inserted

data.

Thematic area:

The thematic areas related to this project are: Compilers, Executable Files, Operating System Kernel.

Subjects:

Compiler Design, Operating Systems, Computer Systems, Software Engineering.

Keywords:

Software tools, *operating systems*, *runtime environment*.

Índice general

1. Introducción **1**

1.1. Antecedentes breves del problema 1

1.2. Relevancia de trabajo 2

1.3. Motivación para la elección del tema 2

1.4. Formulación del problema 2

1.5. Objetivo General y Objetivos Específicos 4

1.6. Alcances 5

1.7. Metodología utilizada para lograr los objetivos propuestos 5

1.8. Requerimientos 6

1.8.1. Requerimientos del usuario 6

1.8.2. Requerimientos del sistema 6

1.8.3. Requerimientos funcionales 6

1.8.4. Requerimientos no funcionales 6

1.9. Orientación al lector de la organización del texto 7

I Marco teórico **9**

2. Compilación **11**

2.1. Conceptos básicos sobre la compilación 11

2.1.1. Preprocesamiento 12

2.1.2. Compilación 13

2.1.3. Ensamblado 15

2.1.4. Enlazamiento 15

2.2. Etapas de la fase de compilación 16

2.2.1. *Frontend* 16

2.2.2. *Middle-end* 18

2.2.3. *Backend* 18

2.3. *Suites* de compilación de *Software* Libre más difundidas . . . 19
2.3.1. GNU Compiler Collection . . . 20
2.3.2. CLANG/LLVM . . . 20
2.3.3. Comparación entre GNU GCC y CLANG/LLVM . . . 20
2.3.4. Optimizador . . . 23
2.3.5. Generador de código ensamblador . . . 23

3. Archivos ejecutables ELF 29
3.1. Introducción a los archivos ejecutables . . . 29
3.2. Archivos ELF . . . 30
3.2.1. Estructura . . . 31

4. Proceso de carga de ejecutables ELF en *kernel* de FreeBSD 41
4.1. Carga del programa en el sistema operativo . . . 41
4.2. Enlazamiento dinámico (intérprete) . . . 45

II Marco metodológico 47

5. Iteraciones 49
5.1. Primera iteración . . . 49
5.1.1. Objetivos a alcanzar . . . 49
5.1.2. Propuesta . . . 49
5.1.3. Implementación . . . 50
5.1.4. Análisis de resultados . . . 50
5.1.5. Próximos pasos . . . 51
5.2. Segunda iteración . . . 51
5.2.1. Objetivos a alcanzar . . . 51
5.2.2. Propuesta . . . 51
5.2.3. Implementación . . . 51
5.2.4. Análisis de resultados . . . 52
5.2.5. Próximos pasos . . . 53
5.3. Tercera iteración . . . 53
5.3.1. Objetivos a alcanzar . . . 53
5.3.2. Propuesta . . . 53
5.3.3. Implementación . . . 53
5.3.4. Análisis de resultados . . . 54

5.3.5. Próximos pasos . . . 54
5.4. Cuarta iteración . . . 55
5.4.1. Objetivos a alcanzar . . . 55
5.4.2. Propuesta . . . 55
5.4.3. Implementación . . . 55
5.4.4. Análisis de resultados . . . 56
5.4.5. Próximos pasos . . . 56
5.5. Quinta iteración . . . 56
5.5.1. Objetivos a alcanzar . . . 56
5.5.2. Propuesta . . . 56
5.5.3. Implementación . . . 56
5.5.4. Análisis de resultados . . . 57
5.5.5. Próximos pasos . . . 58
5.6. Sexta iteración . . . 58
5.6.1. Objetivos a alcanzar . . . 58
5.6.2. Propuesta . . . 58
5.6.3. Implementación . . . 58
5.6.4. Análisis de resultados . . . 60
5.6.5. Próximos pasos . . . 61
5.7. Séptima iteración . . . 61
5.7.1. Objetivos a alcanzar . . . 61
5.7.2. Propuesta . . . 61
5.7.3. Implementación . . . 61
5.7.4. Análisis de resultados . . . 65
5.7.5. Próximos pasos . . . 65
5.8. Octava iteración . . . 65
5.8.1. Objetivos a alcanzar . . . 65
5.8.2. Propuesta . . . 66
5.8.3. Implementación . . . 66
5.8.4. Análisis de resultados . . . 67
5.8.5. Próximos pasos . . . 67
5.9. Novena iteración . . . 68
5.9.1. Objetivos a alcanzar . . . 68
5.9.2. Propuesta . . . 68

5.9.3. Implementación . . . 68
5.9.4. Análisis de resultados . . . 71
5.9.5. Próximos pasos . . . 71
5.10. Décima iteración . . . 71
5.10.1. Objetivos a alcanzar . . . 71
5.10.2. Propuesta . . . 71
5.10.3. Implementación . . . 72
5.10.4. Análisis de resultados . . . 72
5.10.5. Próximos pasos . . . 74
5.11. Onceava iteración . . . 74
5.11.1. Objetivos a alcanzar . . . 74
5.11.2. Propuesta . . . 74
5.11.3. Implementación . . . 74
5.11.4. Análisis de resultados . . . 76
5.11.5. Próximos pasos . . . 76
5.12. Doceava iteración . . . 76
5.12.1. Objetivos a alcanzar . . . 76
5.12.2. Propuesta . . . 76
5.12.3. Implementación . . . 77
5.12.4. Análisis de resultados . . . 78
5.12.5. Próximos pasos . . . 78
5.13. Décimotercera iteración . . . 78
5.13.1. Objetivos a alcanzar . . . 78
5.13.2. Propuesta . . . 78
5.13.3. Implementación . . . 79
5.13.4. Análisis de resultados . . . 82
5.13.5. Próximos pasos . . . 82
5.14. Décimocuarta iteración . . . 82
5.14.1. Objetivos a alcanzar . . . 82
5.14.2. Propuesta . . . 82
5.14.3. Implementación . . . 83
5.14.4. Análisis de resultados . . . 86
5.14.5. Próximos pasos . . . 86
5.15. Décimoquinta iteración . . . 86

5.15.1. Objetivos a alcanzar . . . 86
5.15.2. Propuesta . . . 87
5.15.3. Implementación . . . 87
5.15.4. Análisis de resultados . . . 89
5.15.5. Próximos pasos . . . 90

6. Descripción del modelo experimental **91**
6.1. *Plugin* para inserción de metadatos . . . 91
6.2. Parche de *kernel* . . . 92
6.2.1. *Changelog* . . . 92
6.2.2. Funcionalidad agregada en parches de *kernel* . . . 93

7. Resultados **97**
7.1. Archivos a insertar . . . 97
7.2. Código fuente del programa de usuario . . . 97
7.3. Compilación de *plugin* de compilación . . . 98
7.3.1. *Plugin* para CLANG . . . 99
7.3.2. *Plugin* para GCC . . . 99
7.4. Obtención del ejecutable con metadatos insertados . . . 100
7.4.1. Compilación de código de usuario y uso de *plugin* para CLANG . . . 100
7.4.2. Compilación de código de usuario y uso de *plugin* para GCC . . . 102
7.5. Ejecución del programa . . . 104
7.6. Lectura de *log* del *kernel* . . . 104

III Conclusiones **107**

Conclusiones **109**

Anexo I. Llamada a funciones durante la carga de ejecutables ELF en el sistema **125**
I.1. Llamada a función de sistema `fork()` . . . 125
I.2. Llamada a función de sistema `exec()` . . . 127

Anexo II. Preparación del entorno de desarrollo para FreeBSD **129**
II.1. Compilación e instalación del *kernel* de FreeBSD . . . 129
II.2. Compilación e instalación del parche de *kernel* de FreeBSD . . . 129
II.3. Recompilado rápida e instalación de *kernel* de FreeBSD . . . 130

Índice de figuras

2.1. Fases del proceso de compilación (Fuente: Practical Binary Analysis, [4]) 12
2.2. Partes de la etapa de compilación del proceso de compilación (Fuente: Engineering a Compiler, [5]) 13
2.3. Módulos de la fase de compilación (Fuente: (Modificado) Compilers: Principles, Techniques, and Tools, [6]) 16
2.4. Ejemplo completo de fases de compilación (Fuente: Compilers: Principles, Techniques, and Tools, [6]) 26
2.5. Etapas del proceso de compilación en GNU GCC, junto con sus entradas y salidas. (Fuente: GCC and Make - Compiling, Linking and Building C/C++ Applications, [24]) 27
2.6. Módulos que componen la etapa de compilación en GNU GCC. (Fuente: Compilers: Principles and Practice, [7]) 27
2.7. Esquema de la etapa de compilación en CLANG/LLVM. (Fuente: LLVM IR and Go, [25]) 27
2.8. Partes de la estructura de CLANG. (Fuente: Getting Started with LLVM Core Libraries, [8]) 27
2.9. Partes de la estructura de LLVM. (Fuente:Getting Started with LLVM Core Libraries, [8]) 28

3.1. Vistas de un archivo ELF. (Fuente: Executable and Linking Format Specification, Version 1.2, [9]) 31

4.1. Mapeo entre el archivo ELF y la memoria virtual del proceso asociado. (Fuente: ELF, [32]) 45

5.1. Comparación entre el formato de un archivo ELF y la sección `.metadata` planteada. Fuente: Propia 61

Índice de tablas

2.1. Comparación con respecto a preprocesador. 21
2.2. Comparación con respecto a analizador léxico. 21
2.3. Comparación con respecto a analizador sintáctico. 22
2.4. Comparación con respecto a analizador semántico. 22
2.5. Comparación del generador de código intermedio - *Middle-end*. 23
2.6. Comparación con respecto a optimizador de código intermedio. 24
2.7. Comparación con respecto a generador de código ensamblador. 25

3.1. Principales formatos de códigos ejecutables en Unix. 30

6.1. Nombre y descripción de las funciones definidas en `metadata_elf_reader.c`. . . . 94

Lista de Códigos

2.1. Código fuente `ejemplo.c`. 12
2.2. Salida del preprocesador para `ejemplo.c`. 13
2.3. Salida de la fase de compilación para `ejemplo.c`. 14
2.4. Salida de la fase de ensamble durante la compilación de `ejemplo.c` 15
2.5. Ejecutable final obtenido de la compilación de `ejemplo.c` 16
3.1. Información de la cabecera ELF del ejecutable `ejemplo` usando la herramienta `readelf`. 33
3.2. Tabla de cabeceras de sección del ejecutable `ejemplo`. 36
3.3. Mapa de secciones y segmentos para el ejecutable `ejemplo`. 37
3.4. Tabla de cabeceras de programa para el ejecutable `ejemplo`. 38
4.1. Llamadas realizadas al ejecutar el archivo `ejemplo` usando la herramienta `ktrace`. . 42
4.2. Estructura `image_params` en FreeBSD. Definida en `imgact.h`. 43
4.3. Iteración sobre activadores de imágenes. Función `do_execve` en `kern_exec.c`.. . . 44
5.1. Método inicial de inserción en sección metadata [34]. 50
5.2. Campos agregados a la estructura proc en proc.h [40]. 54
5.3. Inserción de datos en sección `.metadata` establecido en código de usuario de forma directa. 55
5.4. Línea que contiene múltiples payloads de forma codificada. 56
5.5. Header con struct Payload y nueva macro de inserción definidas [45]. 59
5.6. Inserción de metadatos usando estructuras Payload. [46]. 59
5.7. Extracto de log de kernel con resultados de decodificación de payload en forma de estructuras de C. [51]. 60
5.8. Estructuras que conforman la estructura de la sección .metadata 63
5.9. Programa de inserción de metadatos de forma estructurada simil a organización de ELF. [55]. 64
5.10. Salida de logs de kernel mostrando decodificación correcta de Payload A y Payload B. [57]. 65

5.11. Arreglo de funciones de decodificación de payloads. [58]. 66
5.12. Función que decodifica e imprime el contenido de datos de tipo `Payload_B`. [58]. . . 67
5.13. Salida de log de kernel con estructuras `Payload_A` y `Payload_B` decodificadas. [60]. 67
5.14. Archivo de texto que contiene los datos a insertar. [61]. 69
5.15. Instrucciones de compilación del plugin y su uso en compilación con programa de usuario. [62]. 69
5.16. Código de usuario [64] que incluye la variable `__PAYLOAD__` a ser reemplazada por el plugin. 70
5.17. Reemplazo de la variable __PAYLOAD__ por líneas metadata mediante el plugin en durante la compilación de código de usuario. 70
5.18. Makefile con instrucciones para extraer e insertar sección utilizando `objcopy` [69]. . 73
5.19. Resultado de decoficación de la sección .metadata insertada utilizando objcopy [70]. 73
5.20. Función VisitFunctionDecl con líneas para encontrar la función de nombre PAYLOAD en el código usando plugin de CLANG. [72]. 75
5.21. Comando de uso del plugin de CLANG en la compilación del programa de usuario `test.c` [75]. 75
5.22. Programa `test.c` con función PAYLOAD a remplazar. [75]. 75
5.23. Programa `test.c` con función reemplazada y líneas metadata agregadas. 76
5.24. Chequeo de bloqueo por parte del hilo en `metadata_elf_reader.c` [76] [77] . . . 77
5.25. Contenido del archivo `decl_payA.txt` a convertir en binario [93] 79
5.26. Resultados de la conversión de `decl_payA.txt` 79
5.27. Comando hexdump con flag `-C` sobre `declarations.txt_array.bin.txt` 79
5.28. Estructura `Payload_Binary` [95] 80
5.29. Extracto del `Makefile`. Uso de archivos binarios como argumentos para el *plugin* de GCC [96] 80
5.30. Línea con estructuras metadata resultado del uso del *plugin* de GCC [97] 81
5.31. Resultado de la decodificación en *kernel* usando estructuras `Payload_Binary`. . . . 81
5.32. Contenido del archivo `decl_payA.txt_binary.bin` [109] 83
5.33. Contenido del archivo `decl_payB.txt_binary.bin` [110] 83
5.34. Función `VisitVarDecl` [111] para encontrar la variable `__PAYLOAD__` 84
5.35. Programa de usuario con variable __PAYLOAD__ a ser reemplazada [112] 85
5.36. Comando para compilación de programa de usuario utilizando el plugin de CLANG [113] 85
5.37. Programa de usuario con líneas *metadata* colocada por el plugin de CLANG 85

5.38. Logs del kernel con la decodificación de las estructuras *Payload* correctamente insertadas por plugin de CLANG [115] 86

5.39. Campos metadata de la estructura `proc` del *header* `proc.h` [116] 87

5.40. Archivo cabecera `metadata_payloads.h` [117] con soporte de único tipo de *payload*: `Payload_Binary` 88

5.41. Comando para ejecutar el *script* `test.sh` [125] con los archivos `decl_payA.txt` [126] y `decl_payB.txt` [127] para insertarlos utilizando *plugin* de CLANG 89

7.1. Contenido del archivo `decl_payA.txt` a insertar con los *plugins* para CLANG [140] y GCC [141] 97

7.2. Contenido del archivo `decl_payB.txt` a insertar con los *plugins* para CLANG [142] y GCC [143] 98

7.3. Código fuente (`prueba.c` [144] para GCC y `test.c` [145] para CLANG) con líneas que habilitan la inserción de metadatos 98

7.4. Comandos y salida de la instalación del paquete de LLVM 11 99

7.5. Comando y salida reducida de la ejecución del *script* `setup.sh` para la compilación del *plugin* para CLANG: `InsertPayload_Plugin` [146] 99

7.6. Comandos y salida de la instalación del paquete de GCC 11 100

7.7. Comando y salida reducida de la ejecución del *script* `setup.sh` [147] para la compilación del *plugin* para GCC: `InsertPayload_GCCPlugin` 100

7.8. Comando de ejecución y salida del *script* `test.sh` [148] pasando a `decl_payA.txt` [140] y `decl_payB.txt` [142] como argumentos al *plugin* de CLANG. breaklines . 101

7.9. Salida del comando `readelf` con secciones del ejecutable `metadataTestProgram` . 102

7.10. Comando de ejecución y salida del *script* `test.sh` [150] pasando a `decl_payA.txt` [141] y `decl_payB.txt` [143] como argumentos al *plugin* de GCC 103

7.11. Salida del comando `readelf` con secciones del ejecutable `metadataTestProgram` . 103

7.12. Ejecución y salida en terminal del programa `metadataTestProgram` compilado con CLANG/LLVM 104

7.13. Ejecución y salida en terminal del programa `metadataTestProgramGCC` compilado con GCC 104

7.14. *Logs* de sistema mostrando datos insertados en ejecutable `metadataTestProgram` compilado con CLANG/LLVM. 105

7.15. *Logs* de sistema con datos insertados en ejecutable `metadataTestProgramGCC` compilado con GCC. 105

Lista de Símbolos y Convenciones

GCC GNU Compiler Collection

LAC Laboratorio de Arquitectura de Computadoras

FCEFYN Facultad de Ciencias Exactas, Físicas y Naturales

UNC Universidad Nacional de Córdoba

IR Intermediate Representation - Representación intermedia

ELF Executable and Linking Format

Capítulo 1

Introducción

Los lenguajes de programación, los compiladores, los sistemas operativos y el hardware sobre el que corre el sistema operativo se encuentran en constante evolución. Continuamente, aparecen nuevos lenguajes de programación que intentan cubrir las falencias de los ya existentes, mejorar la usabilidad y disminuir los tiempos de programación, adaptarse a nuevos dominios de problema, cubrir una nueva arquitectura, entre otras razones. Los compiladores han acompañado el aumento en la complejidad de los nuevos lenguajes teniendo que aumentar su eficiencia para dar soporte a las nuevas funcionalidades de cada lenguaje. Asimismo, los compiladores han tenido que desarrollar multiples capas de análisis y transformación para poder aprovechar al máximo las nuevas soluciones de arquitectura de las distintas computadoras, obteniendo así una mejora en la *performance* de los programas de usuario. Por último, los sistemas operativos se encargan del manejo de recursos, brindados por el hardware y cada vez más avanzados en cuanto a capacidad y velocidad. Los sistemas operativos buscarán distribuir dichos recursos de la manera más óptima para que los programas se ejecuten de manera tal que el rendimiento de los mismos sea el mejor posible.

No obstante, se advierte que persiste una disociación entre estos jugadores que no siempre nos permite abordar el desarrollo de los sistemas de computación como un todo.

1.1. Antecedentes breves del problema

Los antecedentes que se han encontrado durante el proceso de investigación inicial han sido trabajos relacionados al análisis de archivos ELF, principalmente con el objetivo de inyectar código (potencialmente malicioso) e información adicional dentro de la estructura del archivo de dicho formato [13]. Sumado a eso, se han encontrado proyectos que postulan soluciones utilizando secciones del propio archivo para agregar una capa de seguridad sobre los mismos [14]. Sin embargo, estos proyectos no presentaban cobertura en cuanto al par problema-solución planteado para el presen-

te proyecto integrador con respecto a la profundidad, nivel de abstracción y enfoque de la solución especificada.

El informe "Compiladores y Ejecutables" [15] contiene las bases del trabajo realizado. En dicho documento, se han repasado aspectos generales sobre compiladores, con detalle en las fases de compilación y en la generación de código intermedio durante dicho proceso; características y diferencias entre las *suites* de compilación *GNU GCC* y *CLANG/LLVM*; revisión sobre archivos ejecutables, especialmente formato ELF, sus características y estructura; y diversos métodos de inserción de datos dentro de archivos ELF, lectura, extracción y recuperación de datos insertados en espacio de usuario.

1.2. Relevancia de trabajo

La relevancia del trabajo realizado es la de disponer de un instrumento para que el desarrollador de *software* de espacio de usuario pueda enviar información útil hacia el *kernel* del sistema operativo. Éste mecanismo podrá servir como base para dar soporte a los objetivos del resto y futuras versiones de proyectos que mantiene el Laboratorio de Arquitectura de Computadoras (LAC) [16] de la Facultad de Ciencias Exactas, Físicas y Naturales (FCEFYN) de la Universidad Nacional de Córdoba (UNC). Por ejemplo, poder suministrar información representada como Redes de Petri al planificador dinámico de FreeBSD [17], proyecto integrador también realizado en la facultad.

1.3. Motivación para la elección del tema

La elección del tema nace a partir de la oferta de posibles temas de proyecto integrador que el equipo, formado por el Director y Co-director del presente proyecto, han realizado a los integrantes alumnos. Como ya se ha mencionado anteriormente, éste trabajo corresponde a la continuación de aquel abordado para la asignatura "Práctica Supervisada" de la carrera de Ingeniería en Computación (trabajo reflejado en el informe "Compiladores y Ejecutables" [15], bajo la dirección, codirección y realización por parte los mismos actores ya nombrados. Una vez finalizada dicha asignatura, y habiendo obtenido un resultado favorable en cuanto a la factibilidad de insertar metadatos dentro de archivos ejecutables, se ha decidido continuar el trabajo abordando también el espacio de *kernel*, junto a la optimización y abstracción de la inserción de datos en espacio de usuario.

1.4. Formulación del problema

Existen casos en los cuales el programador, que desarrolla un programa específico, quisiera establecer ciertos parámetros del sistema operativo que permitan obtener un mejor tratamiento del programa.

El desarrollador quisiera que estos parámetros tengan validez solamente durante la ejecución del programa generado, y pudieran variarse los parámetros cada vez que se genere un nuevo programa, es decir, como resultado de un nuevo proceso de compilación.

Estos parámetros deberían ser incorporados como información útil asociada al programa obtenido. La inserción debería ser realizada de manera transparente para el programador e implicar, además, que el proceso no deba ser complejo en cuanto a los pasos a realizar para dicho cometido.

Esto permitiría adjuntar información útil que podría ser recuperada durante o luego de la carga del programa en el sistema operativo. Así, se podría, por ejemplo, modificar ciertos aspectos que alterarían la ejecución de los programas en búsqueda de una mejora en cuanto a ajustes estáticos del programa en sí (tales como *setear* el valor de una constante en tiempo de compilación), incorporar información destinada a un tratamiento controlado por parte del sistema operativo o tener datos disponibles para uso por parte del programa.

Un ejemplo de esto podría ser el proveer información, representada como Redes de Petri, al planificador dinámico del sistema operativo FreeBSD, complementando lo realizado en un proyecto integrador [17] previamente desarrollado en la facultad. Por otro lado, pensado en contexto de trabajos de investigación, se podría suministrar un valor a una nueva variable sumada en una de las estructuras manejadas por el *kernel*. En otro caso, el desarrollador podría definir características de ejecución del programa que indiquen tomar mayor cantidad o liberar recursos de *hardware* para ajustarse al nivel de criticidad del programa. En más detalle, se podría definir, en tiempo de compilación del programa, que los hilos del proceso corran en un *core* determinado, garantizándose así velocidad en la ejecución de los mismos y reducción de la penalidad dada por la invalidación de caché que se sufre al reiniciar la ejecución del hilo en un *core* distinto. En este último caso, al realizar esta definición en tiempo de compilación, se libraría de usar la llamada al sistema correspondiente una vez iniciada la ejecución del proceso.

La justificación del problema abordado durante el proyecto, resulta del trabajo realizado en la materia Práctica Supervisada; plasmado en el informe “Compiladores y Ejecutables” [15].

En dicho informe, se ha mostrado la factibilidad técnica de llevar a cabo inserción de información dentro de la estructura de un archivo ELF. La generación de un programa que permita insertar la información al suministrar un archivo ELF [18], así como la posterior lectura y recuperación de los mismos datos insertados [19], se reflejan en las páginas de dicho informe, así como en el repositorio de trabajo [20]. Diversas técnicas de inserción de datos en distintas partes del archivo ELF fueron presentadas en [15]. La primera de ellas [21] implica el uso de lenguaje ensamblador dentro del código C, para insertar un *string* en la sección ELF especificada. La segunda técnica [22] involucra el uso de la herramienta *Objcopy* para la inserción, realizada post-compilación, obteniendo una un nuevo

archivo ELF que contiene copia del ELF original sumado a los datos insertados. La tercera técnica [23] utiliza la unión de `__attribute__`, palabra reservada de C para habilitar atributos especiales; junto con la función atributo `section`, que permite insertar código en secciones de la imagen del archivo ejecutable. Finalmente, la técnica elegida para insertar los datos ha sido una combinación de la anterior, uso de `__attribute section__`, pero indicando como destino una sección nueva dentro del archivo ELF [18]. Se ha corroborado la correcta inserción mediante un programa [19] en espacio de usuario, que lleva a cabo la recuperación y lectura de los datos insertados mediante la carga y búsqueda de la sección nueva insertada. Luego de encontrar la sección, el programa extrae los *bytes* del contenido y, finalmente, se imprimen por consola.

Estos programas de inserción y posterior lectura de los datos insertados son la justificación que nos ha dado la señal para abordar la solución propuesta frente al problema detectado. Por lo tanto, es factible realizar la recuperación de metadatos insertados previamente. Este proyecto intentará replicar dicha lectura, esta vez en espacio de *kernel*, para obtener así datos colocados por el programador desde espacio de usuario, para un posterior uso por parte del sistema operativo. Asimismo, se explorará un camino alternativo al uso de llamadas al sistema para comunicar el espacio de usuario con el espacio de *kernel*.

1.5. Objetivo General y Objetivos Específicos

El objetivo principal del trabajo es:

Desarrollar un mecanismo, y la/s herramienta/s asociadas, para la comunicación entre espacio de usuario y espacio de *kernel* mediante la inserción de metadatos en archivos ejecutables de formato ELF en tiempo de compilación y la recuperación de dichos datos para un posterior uso por parte del sistema operativo, previamente haber sido cargado el archivo ejecutable ELF en la *kernel* del sistema operativo *FreeBSD*.

Los objetivos específicos o secundarios planteados para el proyecto son los siguientes.

- Obtener el mayor nivel de transparencia para el programador en el uso de la herramienta de inserción de datos.
- Lograr la mayor abstracción posible en el protocolo de comunicación para poder cubrir así distintos casos de uso a futuro.
- Alcanzar el menor nivel posible de restricción en cuanto a la naturaleza de los posibles datos a insertar como metadatos dentro de los archivos ejecutables ELF.
- Organizar los datos de tal manera que se favorezca a la inserción y, posterior, decodificación de los metadatos insertados.
- Insertar los metadatos en archivos ejecutables de formato ELF.
- Desarrollar soluciones compatibles con el *kernel* del sistema operativo *FreeBSD*.

1.6. Alcances

El desarrollo final corresponde a una versión de prototipo del protocolo de comunicación planteado. Ésto implica una versión inicial del mismo.

El enfoque ha sido puesto en cumplir los objetivos principales y secundarios expuestos anteriormente, con énfasis en alcanzar la funcionalidad planteada y simplicidad deseada para la inserción de datos. No se llevarán a cabo mediciones de performance y tiempo de carga de los archivos ELF con metadata de diversos tamaños incluidos.

La versión obtenida se limita a una versión abstracta en cuanto a independencia con respecto al tipo de datos insertados y posibles usos por parte del sistema operativo en el futuro. No se desarrolla el protocolo teniendo en base un uso específico por parte del sistema operativo. En cambio, se plantea un modelo listo para definir casos de uso en forma sencilla para su definición y coordinación entre el programador y el *kernel*.

El protocolo está limitado al sistema operativo FreeBSD, especificamente las versiones 11, 12 y 13. Ésto implica ajustarse a las funciones y estructuras definidas para dichas versiones del sistema operativo, y su consiguiente versión de *kernel*. Por lo tanto, la portabilidad entre diferentes plataformas (otros sistemas operativos) y/o nuevas versiones de FreeBSD se relega para versiones futuras del trabajo. Las suites de compilación usadas son:

- GNU GCC, en su versión 11.
- CLANG/LLVM, en su versión 11.

1.7. Metodología utilizada para lograr los objetivos propuestos

La metodología empleada para poder cumplir con los objetivos detallados anteriormente está basada en el desarrollo incremental [1].

Tal como se explica en [1], el desarrollo incremental está basado en generar una implementación inicial. Luego, el desarrollo continúa con versiones intermedias que suman funcionalidades, a partir de etapas guiadas por la validación del usuario a quien se destina el proyecto. Este proceso continúa hasta que se obtiene una versión final.

Esto ha permitido responder rápidamente ante los cambios que se han ido presentando a medida que se obtenían resultados tanto favorables como desfavorables, para obtener entregas continuas y propiciar un desarrollo sostenido, al ir sumando funcionalidades de inserción, lectura y decodificación en forma progresiva

1.8. Requerimientos

1.8.1. Requerimientos del usuario

- El desarrollador podrá insertar datos dentro del programa generado, que puedan impactar en alguna funcionalidad o comportamiento del sistema operativo con respecto al tratamiento de dicho programa.

1.8.2. Requerimientos del sistema

- La inserción de datos se realizará en espacio de usuario en tiempo de compilación.
- El soporte del protocolo se realizará solo en archivos ejecutables de tipo ELF.
- Los datos serán insertados dentro del programa resultante del código deseado por el desarrollador.
- Los datos serán leídos por el sistema operativo una vez que el programa sea ejecutado.
- Los datos se utilizarán en espacio de *kernel* una vez que el programa sea ejecutado.
- Se generará un parche del sistema operativo para poder leer y utilizar en espacio de *kernel* los datos insertados.

1.8.3. Requerimientos funcionales

- El usuario podrá insertar los datos en cualquier programa que éste desee.
- El usuario podrá insertar tipo de dato que desee dentro del programa.
- El usuario podrá insertar múltiples unidades de datos y formatos de los datos en un mismo programa.
- El usuario podrá insertar los datos al mismo tiempo en que se produce la compilación de su programa principal, es decir, en tiempo de compilación.
- El sistema operativo deberá realizar la búsqueda y procesamiento de los datos insertados durante el proceso de carga del programa en el sistema operativo.
- El sistema operativo deberá tomar la/s acción/es correspondiente/s frente a todos los datos que se insertaron en el programa.

1.8.4. Requerimientos no funcionales

- El sistema operativo deberá mantener su funcionalidad por defecto frente a los programas que no contengan datos insertados.
- El formato de los archivos ejecutables es ELF.
- El uso de la herramienta deberá ser transparente para el usuario.
- El modelo de inserción deberá ser abstracto en cuanto a tipo de datos que permite insertar.

- El tamaño total de los datos insertados deberá no superar un umbral máximo definido, siempre menor al los 10 `KiB`, valor discrecional tomado como límite para la versión prototipo. Se generará un alerta en caso de querer insertar un tamaño total de datos mayor y no permitiendo realizar la inserción.
- El sistema operativo no deberá acusar fallos al momento de cargar el programa, leerlo y extraer los datos insertados.

1.9. Orientación al lector de la organización del texto

En primer lugar, se explaya sobre el marco teórico y conceptual que da soporte al desarrollo de las herramienta generada.

En el Capítulo 2, se abordan conceptos relacionados al proceso de compilación de código, sus fases y las *suites* de compilación más relevantes.

En el Capítulo 3, se tratan temas relacionados a archivos ejecutables. Se pone foco en los archivos ejecutables de formato ELF y se detalla su estructura interna.

En el Capítulo 4, se ocupa de especificar el proceso de carga de archivos ejecutables de tipo ELF en el *kernel* del sistema operativo *FreeBSD*.

Posteriormente, se desarrolla el marco metodológico del proyecto realizado.

En el Capítulo 5, se exponen las distintas iteraciones de trabajo llevadas a cabo; detallando los pasos, resultados intermedios y decisiones tomadas durante la realización del proyecto.

En el Capítulo 6, se describen el modelo prototipo final obtenido y la estructura general del mismo.

En el Capítulo 7, se especifican los resultados obtenidos, dando lugar a una descripción de las partes de la herramienta y detallando los pasos que se deben llevar a cabo para poder realizar la correcta inserción en espacio de usuario, así como la consecuente decodificación y lectura de los datos insertados en espacio de *kernel*

Por último, en la Sección III, se presentan las conclusiones del trabajo realizado, los hitos logrados, las limitaciones encontradas y una breve sección con trabajos futuros a desarrollar a partir de ideas que no se llevaron a cabo en el proyecto actual.

Parte I

Marco teórico

Capítulo 2

Compilación

Introducción

En el presente capítulo se abordarán temáticas relacionadas al proceso de compilación de código. En primer lugar, se expondrán las características y etapas de la compilación. Luego, se analizarán en mayor detalle cada una de las fases, junto con las entradas y salidas a cada una de ellas. Finalmente, se presentarán las suites de compilación más relevantes en los sistemas *Unix* y *Unix-like*, sus características principales y otros detalles relevantes.

2.1. Conceptos básicos sobre la compilación

La compilación de código es el proceso de convertir el código fuente, escrito en un lenguaje de programación determinado (C, C++, Java, etc) en un archivo ejecutable, también referido como archivo binario, formado por código máquina; tal como lo exponen Gough y Stallman en [2].

Una definición más técnica es la que presenta Muchnick en [3]: "Un compilador, estrictamente definido, consiste en una serie de fases que secuencialmente analiza determinados programas y sintetiza nuevos, comenzando con la secuencia de caracteres que constituyen un programa fuente para ser compilado y producir, en la mayoría de los casos, un objeto relocalizable que puede ser enlazado con otros y cargado en la memoria de la máquina para ser ejecutado"

El proceso de compilación involucra 4 etapas fundamentales. Ellas son:

1. Preprocesamiento.
2. Compilación.
3. Ensamblado.
4. Enlazamiento (*Linking*, en inglés)

La Fig. 2.1 presenta una representación visual del proceso de compilación.

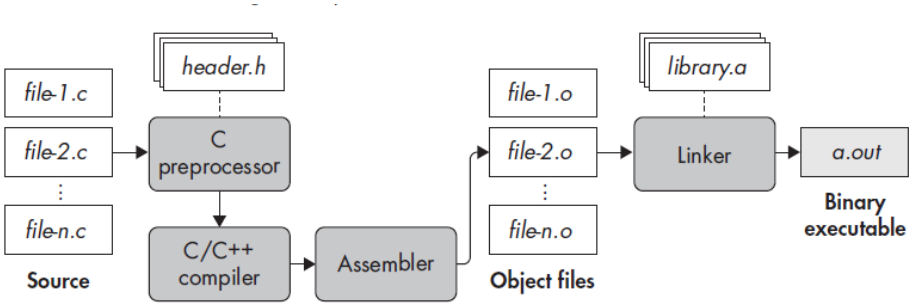


Figura 2.1: Fases del proceso de compilación (Fuente: Practical Binary Analysis, [4])

```c
#include <stdio.h>

#define VAL 3

int main() {
        int num_a = VAL;
        int num_b = 4;

        num_b = num_a;

        return 0;
}
```

Código 2.1: Código fuente `ejemplo.c`.

2.1.1. Preprocesamiento

La etapa de preprocesamiento es la etapa inicial. Todos aquellos archivos fuentes involucrados en la compilación se cargan con el objetivo de realizar las primeras modificaciones. En esta etapa, se remueven los comentarios agregados al código, se agrega el código de los archivos de cabecera (o *headers*) en la línea donde se las ha incluído, se reemplazan las `macros` por los valores definidos para ellas y se modifica el código dependiendo de la directiva de preprocesador que se haya escrito. El objetivo final es obtener un código que solamente tenga código puro del lenguaje de programación.

En el lenguaje C, las directivas de preprocesador se distinguen del resto del código al utilizar el símbolo `#` al inicio de las mismas y no contienen el símbolo `;` al finalizar la línea. Algunas de estas directivas son, por ejemplo: `#define`, `#include`, `#if`, `#else`, `#line`, `#error`, `#pragma`, etc.

También existen macros predefinidas reservadas para el procesador, tales como `__TIME__`, `__FILE__`, `__cplusplus`, entre otros. Sumado a ello, existen operadores especiales que impactan en el preprocesador; algunos son: el operador de continuación de macro `\` o el operador `defined()`.

Un ejemplo de esto se puede observar en los *listings* 2.1 y su correspondiente salida del preprocesador en 2.2. Se observa como se ha insertado en el mismo archivo el contenido del *header* `stdio.h` y se ha reemplazado el valor de `VAL`.

```
typedef signed char __int8_t;
typedef unsigned char __uint8_t;
typedef short __int16_t;
typedef unsigned short __uint16_t;

/***/

void *mmap(void *, size_t, int, int, int, __off_t);
int truncate(const char *, __off_t);
int __srget(FILE *);
int __swbuf(int, FILE *);

/**/

int main() {
        int num_a = 3;
        int num_b = 4;
        num_b = num_a;
        return 0;
}
```

Código 2.2: Salida del preprocesador para `ejemplo.c`.

2.1.2. Compilación

La fase de compilación es la segunda etapa del proceso de compilación. Cabe recalcar, que el nombre de la fase y del proceso coinciden. En esta fase, se toma como entrada al código preprocesado y, a partir de sucesivos pasos, se obtiene finalmente el código ensamblador correspondiente. La fase de compilación está compuesta por diferentes módulos que conforman tres grandes partes de la misma: *a*) *Frontend b*) *Middle-end c*) *Backend*.

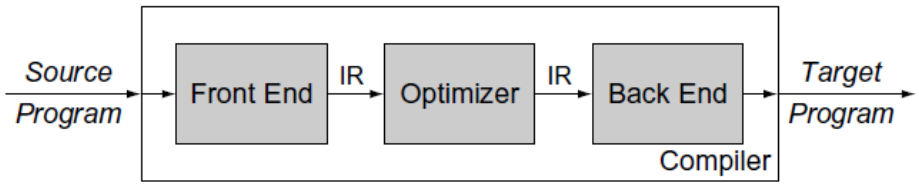


Figura 2.2: Partes de la etapa de compilación del proceso de compilación (Fuente: Engineering a Compiler, [5])

La Fig. 2.2 muestra un esquema que identifica las partes mencionadas.

El *frontend* se encarga de realizar el análisis léxico, sintáctico y semántico del código preprocesado. El objetivo de realizar estos análisis es el de detectar errores en la escritura y, entender y verificar el significado del código.

La salida de la etapa de *frontend* es el llamado código intermedio, libre de errores sintácticos y semánticos. El código intermedio es un lenguaje común al compilador que se utiliza para optimizaciones y mapeo a código ensamblador en las siguientes etapas.

El código intermedio, o también llamado Intermediate Representation - Representación interme-

dia (IR) permite que cualquier *frontend* de cualquier lenguaje soportado por el compilador (C, C++, Java, Python, etc), se traduzca al mismo lenguaje genérico dentro del compilador. A su vez, esto habilita el uso de múltiples *backends* destinados a obtener código máquina para distintas arquitecturas. Ésto define la portabilidad del compilador en cuestión. No hay un único código intermedio, puede variar entre los distintos compiladores y un mismo compilador puede tener varios códigos intermedios. También, debe ser fácilmente optimizable sin implicar una gran complejidad en su procesamiento. En algunos casos, puede también ser usado para ejecución interpretativa (en el caso de Java, Pascal o Forth)

El *middle-end* involucra la optimización del código intermedio obtenido de manera sucesiva; traduciendo, en cada una de las iteraciones, expresiones complejas en formas algebráicas equivalentes. Se obtiene así un programa escrito en código intermedio que se analiza y reescribe mejorando aspectos de, por ejemplo, velocidad o tamaño, cada vez que se reingresa a esta etapa.

```
.file   "ejemplo.c"
.text
.globl  main
.type   main, @function
main:
.LFB1:
.cfi_startproc
pushq   %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq    %rsp, %rbp
.cfi_def_cfa_register 6
movl    $3, -4(%rbp)
movl    $4, -8(%rbp)
movl    -4(%rbp), %eax
movl    %eax, -8(%rbp)
movl    $0, %eax
popq    %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE1:
.size   main, .-main
.ident  "GCC: (FreeBSD Ports Collection) 11.2.0"
.section        .note.GNU-stack,"",@progbits
```

Código 2.3: Salida de la fase de compilación para `ejemplo.c`.

Finalmente, el *backend* está compuesto por etapas que mapean el código intermedio optimizado, obteniendo como salida el código ensamblador de la arquitectura deseada (ver Código 2.3. Las tareas de esta parte de la fase implicna la optimización y traducción hacia instrucciones que cumplan con el set de instrucciones y referencias a recursos (ej.: registros, secciones, etc) de la arquitectura deseada, garantizando el uso correcto y eficiente de estos últimos.

2.1.3. Ensamblado

La fase de ensamble en el proceso de compilación toma el código ensamblador obtenido por la fase anterior y lo transforma en código máquina, también denominado archivo objeto. Los archivos objeto están formados por instrucciones binarias que el procesador puede comprender pero todavía no ejecutar en su totalidad. Las instrucciones del archivo objeto pueden hacer referencia tanto a variables o funciones del propio archivo objeto (conocidos como símbolos) así como a aquellas funciones o variables presentes en otros archivos objetos a los que se hace referencia (llamados símbolos no definidos). Debido a que el ensamblador realiza este proceso de manera independiente a cada archivo, no conoce la dirección de memoria del símbolo no definido, presente en otro archivo objeto distinto al actual. Se deja entonces el símbolo sin resolver. Es por ello que todavía no puede ejecutarse. Dichos símbolos serán resueltos y conocidos una vez que se produzca la siguiente fase de compilación.

```
$objdump -M intel -d  ejemplo.o

ejemplo.o:     file format elf64-x86-64-freebsd

Disassembly of section .text:

0000000000000000 <main>:
    0:   55                      push   rbp
    1:   48 89 e5                mov    rbp,rsp
    4:   c7 45 fc 03 00 00 00    mov    DWORD PTR [rbp-0x4],0x3
    b:   c7 45 f8 04 00 00 00    mov    DWORD PTR [rbp-0x8],0x4
    12:   8b 45 fc                mov    eax,DWORD PTR [rbp-0x4]
    15:   89 45 f8                mov    DWORD PTR [rbp-0x8],eax
    18:   b8 00 00 00 00          mov    eax,0x0
    1d:   5d                      pop    rbp
    1e:   c3                      ret
```

Código 2.4: Salida de la fase de ensamble durante la compilación de `ejemplo.c`

Un ejemplo del archivo objeto obtenido se observa en el Código 2.4.

2.1.4. Enlazamiento

La última etapa del proceso de compilación es la etapa de enlazamiento, *linking* en inglés. Como su nombre lo indica, se encarga de vincular los diferentes archivos objeto en un único archivo binario totalmente ejecutable y completar las referencias (símbolos no definidos de la etapa anterior) a variables o funciones que se encontraban en otro archivo objeto distinto. Se vinculan también referencias a librerías estáticas. En el caso de las librerías dinámicas, pueden resolverse en esta etapa; caso contrario se resuelven al momento de ejecutarse el programa y cargarse en memoria.

El Código 2.5 presenta el tipo de archivo ejecutable obtenido.

```
$ gcc -o ejemplo  ejemplo.c
$ file ejemplo

ejemplo: ELF 64-bit LSB executable, x86-64, version 1 (FreeBSD), dynamically
    ↪ linked,
interpreter /libexec/ld-elf.so.1, for FreeBSD 13.0 (1300139), FreeBSD-style, with
    ↪  debug_info, not stripped
```

Código 2.5: Ejecutable final obtenido de la compilación de `ejemplo.c`

2.2. Etapas de la fase de compilación

La fase de compilación, como se mencionó anteriormente, es la segunda etapa del proceso de compilación.

Dentro de la etapa de compilación, el código fuente atraviesa una serie de cambios a través de diversos módulos hasta obtener el código ensamblador correspondiente a la salida. Dichos módulos constituyen las mencionadas partes de la fase de compilación: *frontend*, *middle-end* y *backend*.

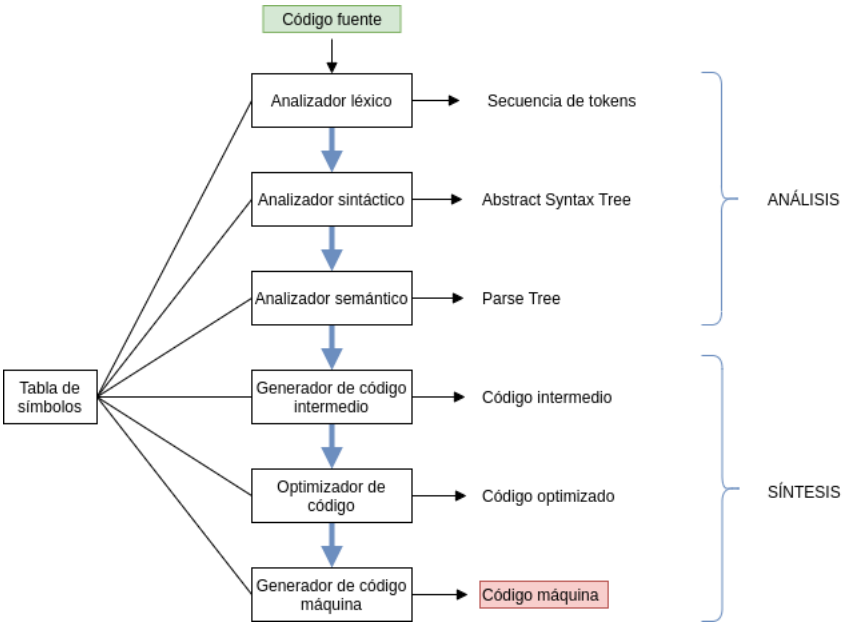


Figura 2.3: Módulos de la fase de compilación (Fuente: (Modificado) Compilers: Principles, Techniques, and Tools, [6])

La Fig. 2.3 presenta un esquema con los componentes de la fase.

2.2.1. *Frontend*

2.2.1.1. Analizador léxico

El analizador léxico, también conocido como *scanner* o *lexer*, es aquel módulo que lee la secuencia de caracteres del código fuente y los agrupa en secuencias llamadas **lexemas**. Por cada lexema, el

scanner genera un *token* de la forma:

$$< \textit{nombre-token}, \textit{valor-atributo} > \tag{2.1}$$

Siendo,

- **nombre-token**: símbolos abstractos usados durante el análisis sintáctico.
- **valor-atributo**: puntero a una entrada en la tablas de símbolos.

El analizador léxico también se encarga de detectar aquellos *tokens* inválidos para el lenguaje, generando un error debido a ellos. Éstos *tokens* inválidos pueden generarse por identificadores, operadores o palabras claves mal escritas o con caracteres ilegales en los mismos. La Fig. 2.4 muestra las entradas y salidas del módulo en la parte superior.

Tabla de símbolos El compilador almancena los nombres de variables o funciones que aparecen en el código fuente en la **tabla de símbolos**. También, en ella se almacenan atributos varios de dicha variable, por ejemplo: tipo, alcance actual (llamado *scope*), argumentos, tipo de pasaje de argumentos, tipo de retorno, etc. La tabla permite realizar un mapeo de *tokens* con sus respectivos nombres en la tabla para conocer sus atributos.

2.2.1.2. Analizador sintáctico

El analizador sintáctico, también llamado *parser*, utiliza el primer componente de cada *token* (*nombre-token* en la Ecuación 2.1) para crear una representación en forma de árbol que exponga la estructura de los *tokens*. El objetivo es verificar que el código fuente ha sido escrito correctamente cumpliendo las reglas estructurales de sentencias del lenguaje de programación del *frontend*.

El mencionado árbol también es conocido como **árbol sintáctico**.

El árbol sintáctico es una estructura jerárquica formada por nodos interiores, que representan operaciones, y los hijos de cada nodo u hojas, representan los operandos (o argumentos) para dichas operaciones. El parseo y lectura de dicho árbol permite generar los errores sintácticos detectados en la estructura.

La Fig. 2.4 refleja un esquema de esta fase en la parte superior.

2.2.1.3. Analizador semántico

El analizador semántico emplea el árbol sintáctico y la tabla de símbolos, anteriormentes obtenidas, para revisar la consistencia semántica del código fuente con respecto a la definición del lenguaje, es decir, si el significado del código es correcto y consistente. Se chequean errores de escritura, uso

correcto de datos con respecto a su definición, operandos incompatibles, llamadas de funciones con argumentos erroneos, variables no declaradas, referencias a etiquetas, etc.

También se realizan conversiones de un tipo de dato a otro (llamadas coerciones) y verificar su compatibilidad.

Ver Fig. 2.4 en la fase correspondiente.

2.2.2. *Middle-end*

2.2.2.1. Generador de código intermedio

Dentro de la parte de *middle-end* de la fase de compilación se encuentra la generación de código intermedio.

El generador de código intermedio tiene como meta principal producir código de tipo intermedio, también llamado IR, para obtener un lenguaje flexible y optimizable para la parte *backend* del compilador. Observar dentro de la Fig. 2.4 para una ilustración de la fase.

El código intermedio puede tener muchas formas distintas (dependiendo de cada compilador) y sirve como lenguaje común de procesamiento para poder alcanzar distintas arquitecturas en el *backend*, proveniendo también de múltiples *frontends* de diversos lenguajes.

Propiedades de todo código intermedio Todo código intermedio debe cumplir las propiedades de: *a*) ser fácil de generar; *b*) ser fácil de traducir a lenguaje máquina

Código de tres direcciones Es una forma de código intermedio que consiste en una secuencia de instrucciones cuasi-*assembler* con tres operandos por cada instrucción (cómo máximo). Cada operando puede actuar como un registro.

Algunas características de este tipo de código son:

- Las instrucciones deben tener como máximo un **único operador** (órden de operaciones)
- El compilador debe generar un **nombre temporal** para guardar el valor retornado por la instrucción de tres direcciones (variables temporales tienen nombre).

2.2.3. *Backend*

2.2.3.1. Optimizador de código intermedio

El optimizador de código intermedio realiza un análisis estático sobre el código obtenido y ejecuta optimizaciones para obtener un código más eficiente a través de una serie de iteraciones sobre el código optimizado resultante, sin cambiar la semántica del código. Las mejoras se reflejan en código de mayor rapidez de ejecución, de menor tamaño o de menor consumo de potencia. En concreto, en

este módulo se evalúan aspectos como propagación de variables, invariantes dentro de *loops*, llamada de funciones, cálculo de direcciones, etc. A grandes rasgos, la optimización del código se puede llevar a cabo en dos niveles:

- Nivel local: dentro de un bloque de código determinado, denominado bloque básico.
- Nivel global: abarcando distintos bloques básicos. Optimización de mayor complejidad al tener que analizar el flujo de los datos a través de los bloques seleccionados.

Se presenta también una ilustración en la parte inferior de la Fig. 2.4.

2.2.3.2. Generador de código final

El generador de código final utiliza el código intermedio optimizado para generar el código de salida deseado. El código de salida puede ser:

- Código máquina absoluto.
- Código máquina relocalizable.
- Código ensamblador.

El código máquina de tipo absoluto siempre es cargado en la misma dirección de memoria. Es por ello, que no se deben realizar etapas adicionales de enlace sobre dichas direcciones. Debido a ésto, el código máquina absoluto puede ser ejecutado una vez obtenido.

El código máquina relocalizable, a diferencia del anterior, necesita de su paso por la etapa de enlace (ver Sección 2.1.4) del proceso de compilación ya que existen direcciones que deben ser rellenadas al utilizar módulos relocalizables en su interior.

El código ensamblador es código formado por instrucciones de lenguaje ensamblador de fácil lectura para el programador, correspondiente a la arquitectura de destino. Este código deberá pasar por las etapas de ensamble (ver Sección 2.1.3) y enlace (ver Sección 2.1.4) del proceso de compilación para poder obtener código ejecutable.

La parte inferior de la Fig. 2.4 muestra una ilustración sobre esta fase.

2.3. *Suites* de compilación de *Software* Libre más difundidas

Dentro de los sistemas *linux* (por ejemplo *Ubuntu*, *Linux Mint* y *Arch*); de los sistemas *UNIX* (tales como los sistemas operativos *MacOS* o *Solaris*) y de los sistemas *Unix-like* (como pueden ser *FreeBSD* y *OpenBSD*), las *suites* de compilación más relevantes y utilizadas para los lenguajes C y C++, son:

- GNU Compiler Collection (GCC).

- CLANG/LLVM.

A continuación se expondrá con mayor detalle cada una de estas *suites* de compilación.

2.3.1. GNU Compiler Collection

GCC es el conjunto de compiladores desarrollados por el proyecto GNU. Los compiladores más relevantes de esta *suite* son: `gcc` para C y `g++` para C++. Asímismo se encuentran `gcj` para Java, `gobjc` para Objective-C, `gfortran` para Fortran, entre otros.

Es la *suite* de compilación por *default* en todos aquellos sistemas operativos *GNU/Linux*.

GCC permite realizar las etapas de preprocesamiento, compilación, ensamble y enlace con la misma *suite*. Ver Fig. 2.5.

Los módulos que componen la etapa de compilación de GNU GCC se observan en la Fig. 2.6.

2.3.2. CLANG/LLVM

La *suite* de compilación CLANG/LLVM está formada por dos componentes fundamentales: la parte *frontend*: **CLANG** y la parte *middle-end* y *backend*: **LLVM**. CLANG es el *frontend* dedicado al lenguaje de programación C y C++. Como otros *frontend* compatibles con la parte *backend*, LLVM, se encuentran: `llgo` para Go, `flang` para Fortran, `rust-lang` para Rust, etc.

Al igual que GCC, CLANG/LLVM permite realizar el proceso de compilación por completo usando la misma *suite*.

Un esquema de la etapa de compilación en CLANG/LLVM se observa en la Fig 2.7.

Haciendo foco en la etapa de compilación, las Fig. 2.8 y Fig. 2.8 reflejan un esbozo de la estructura interna de CLANG y LLVM, en ese orden.

2.3.3. Comparación entre GNU GCC y CLANG/LLVM

Si bien los módulos que componen a ambas suites, GNU GCC y LLVM, son los mismos y la funcionalidad de cada uno es la descripta en la etapa de compilación de un compilador moderno (ver Sección 2.2); se presentarán las semejanzas y diferencias de entre los módulos de las dos *suites* de compilación presentadas anteriormente.

2.3.3.1. Preprocesador

La comparación entre ambas *suites* con respecto a la etapa de preprocesamiento se observa en la Tabla 2.1 y se percata la similitud en el manejo de formatos para las entradas y salidas a dicha etapa.

Tabla 2.1: Comparación con respecto a preprocesador.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Etapa a cargo del preprocesador de GNU GCC, **cpp**	Etapa construida en base a la clase `Preprocessor` [26], componente del *frontend* de CLANG
Funciones	Remover los comentarios, incluir el código de los archivos de cabecera (*headers*) y reemplazar macros por sus valores.	Ídem
Entradas	Código en lenguaje C, `.c` o `.h`.	Ídem
Salidas	Código extendido `.i`.	Ídem

2.3.3.2. Analizador léxico o *scanner* o *lexer*

Las diferencias y semejanzas se muestran en la Tabla 2.2. En ambos casos, la salida procesada por el analizador léxico se emite en forma de *tokens*. Estos no son posibles de guardar en disco. En vez, estos son archivos internos en el proceso de compilación.

Tabla 2.2: Comparación con respecto a analizador léxico.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Módulo con base en la herramienta **Flex**, *scanner*/*lexer* que no forma parte de GNU, pero es usado junto a GNU Bison (*parser*).	Basado en la clase `Lexer` [27], parte del *frontend* de CLANG.
Funciones	1. Crear *tokens* a partir del código extendido. 2. Funcionar a la par de *parser*. El *scanner* es invocado cuando el *parser* necesita un nuevo *token*.	Ídem
Entradas	1. Código extendido `.i`. 2. Archivo `lex.yy.c` por cada secuencia de *tokens*, cuando trabaja con *parser*. NOTA: archivos `lex.yy.c` son internos al proceso de compilación. GCC no lo almacena (***no dump***)	1. Código extendido `.i` como punto de entrada, si no se usa el preprocesador anteriormente. 2. Tokens
Salidas	*Tokens* (***no dump***)	Llamados *Tokens* normales. (***no dump***)

2.3.3.3. Analizador sintáctico o *parser*

La comparativa se expone en la Tabla 2.3. En el caso de *CLANG*, es posible obtener la salida de la acción ejecutada por el par *lexer-parser* en forma de *tokens* anotados. En cambio, la salida en *GCC*

es *no dump*, es decir, archivos internos no posibles de guardar en un archivo.

Tabla 2.3: Comparación con respecto a analizador sintáctico.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Basado en **Bison**, el generador de *parser* de GNU.	Construido a partir de la clase `Parser` [28], dentro del *frontend* de CLANG.
Funciones	1. Validar la secuencia de *tokens* generados por el *lexer* con respecto a la gramática válida del lenguaje. 2. Reportar errores de sintáxis. 3. *Error recovery*, si es posible. El funcionamiento de *lexer-parser* se realiza de forma jerárquica, con el siguiente orden: *1* Funciones; *2* Líneas internas de función; *3* Palabras.	Ídem
Entradas	Archivo `parse.y` (***no dump***) por cada secuencia de *tokens*.	*Tokens* normales
Salidas	*Tokens* de salida de parser (***no dump***)	*Tokens* con anotaciones (*Annotation tokens*, en inglés)

2.3.3.4. Analizador semántico

Las diferencias entre ambas *suites* de compilación con respecto a la fase de análisis semántico se divisan en la Tabla 2.4. Se observa que la salida en ambas *suites* contiene la representación del árbol abstracto sintáctico, AST.

Tabla 2.4: Comparación con respecto a analizador semántico.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Módulo ejecutado por **Bison**, el generador de *parser* de GNU.	Basado en la clase `Sema` [29], presente en el *frontend* de CLANG.
Funciones	Crear la representación en forma de árbol de la estructura sintáctica del código (AST).	Ídem
Entradas	*Tokens* resultantes del accionar de *lexer* y *parser*.	Ídem.
Salidas	Archivo `.ORIGINAL`, representación del AST (*abstract syntax tree*), árbol sintáctico abstracto.	Archivo AST (árbol abstracto sintáctico). CLANG permite observar el AST de dos formas: 1. Como código más cercano al modo en que es *parseado* el código fuente. (*Flag* `-ast-print`) 2. Como código en representación parecida a LISP. (*Flag* `-ast-dump`)

2.3.3.5. Generador de código intermedio

La comparación entre los generadores de código intermedio de ambas suites se presentan en la Tabla 2.5. Para el caso de la *suite* GCC, el código intermedio generado por defecto es de tipo `GIMPLE`. Mientras que para CLANG/LLVM, es llamado `LLVM IR` e interno para el compilador. Sin embargo, este último puede ser transformado en una forma codificada, de menor tamaño, o en una forma legible para humanos.

Tabla 2.5: Comparación del generador de código intermedio - *Middle-end*.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Existen distintas transformaciones de código intermedio en GCC. Por defecto, el *frontend* para compilación en C y C++ convierte desde el árbol AST de *frontend* (*ORIGINAL*) a *GIMPLE*, sin pasar por *GENERIC*.	En LLVM/CLANG el código intermedio usado se denomina *LLVM IR Assembly* o simplemente LLVM IR.
Funciones	Transformar el código generado por el analizador semántico en código intermedio.	Ídem
Entradas	Código en formato *.ORIGINAL*	Código AST.
Salidas	Código intermedio de tipo GIMPLE. Más específicamente, *low level GIMPLE*.	Código intermedio de tipo LLVM IR, con formatos: ▪ Representación en memoria, interno al compilador (por *default*) ▪ Bitcode `.bc`, representación en disco de la anterior. ▪ Readable `.ll`, representación en disco de la primera.

2.3.4. Optimizador

Las diferencias en los módulos de optimización se presentan en la Tabla 2.6. Se observa que en ambos casos, las optimizaciones implican un pasaje de sendos códigos intermedios a un acercamiento a un pseudo código máquina, sin llegar a ser tal.

2.3.5. Generador de código ensamblador

Por último, la comparación del módulo de generación de código ensamblador de ambas *suites*, se presenta en la la Tabla 2.7. Ambas *suites* generan los respectivos códigos en lenguaje *Asssembly*.

Tabla 2.6: Comparación con respecto a optimizador de código intermedio.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Pertenece tanto al *middle-end* como al *backend* de GCC. El optimizador de GCC utiliza un código intermedio de muy bajo nivel (*RTL*). Las instrucciones son llevadas a una forma algebráica que describe la acción a realizar. No es usado como código intermedio ya que no permite extraer información de alto nivel como referencias a arreglos o tipo de datos.	Pertenece a la etapa *middle-end* de la *suite* CLANG/LLVM. El optimizador realiza sucesivas iteraciones que generan un código totalmente optimizado.
Funciones	▪ Generar *RTL* y, posteriormente, optimizarlo para obtener código de un nivel más bajo que *GIMPLE*. ▪ Introducir características de código de bajo nivel tales como registros, modos de direccionamiento de memoria, tamaño de palabras y tipos de palabras, instrucciones de *branch* y de comparación, convenciones de llamadas, operaciones de bit, etc.	Ejecutar operaciones realizadas sobre LLVM IR que suponen un cambio en el código o cambio sobre información acerca del código, llamadas *passes*. Existen dos tipos de *passes*: ▪ **De análisis**: operaciones sobre la información relacionada a una unidad de código (bloque básico) LLVM IR sin cambiar dicho código. Generalmente, implican grafos. ▪ **De transformación**: transforman una unidad de código IR. Pueden depender de resultados de análisis e invalidan resultados de análisis cuando se realiza la transformación.
Entradas	Código `.GIMPLE`.	Código LLVM IR.
Salidas	Archivo de tipo *RTL* con notación `.final`.	LLVM IR de un nivel más bajo (todavía válido como código intermedio) y con mayor optimización.

Reseña del capítulo

En el presente capítulo se ha repasado el concepto de compilación. Luego, se han expuesto las fases que conforman al proceso de compilación en su totalidad. Seguido a ello, se ha detallado los módulos que conforman a la fase de compilación del proceso del mismo nombre. Por último, se han comparado las *suites* de compilación GCC y CLANG/LLVM con respecto los distintos módulos atraviesa el código fuente hasta convertirse en código ensamblador; concluyendo que se obtiene el mismo código en lenguaje *Assembly* en esencia, independientemente del compilador usado. Las diferencias encontradas al analizar en detalle los archivos ensambladores (`.s`) residen en cuestiones de organización interna del código por cada compilador, tales como el nombre de variables y otros *tags*, así como las direcciones de cada elemento (por no ocupar la misma posición en la organización del código)

El capítulo siguiente tiene a cargo presentar y detallar aspectos de los archivos ejecutables de tipo

Tabla 2.7: Comparación con respecto a generador de código ensamblador.

Compilador	GNU GCC	CLANG/LLVM
Descripción	Módulo final. Ejecuta la generación de código ensamblador.	Integra el *backend* de CLANG/LLVM, es responsable de la generación de código. Convierte LLVM IR a código *assembler* específico de arquitectura o a código máquina.
Funciones	Obtención de la salida (código máquina o ensamblador) orientado a una determinada arquitectura.	Ídem. Implica el paso por etapas bien definidas dentro de LLVM, estas son: 1. Asignación de registros. 2. Transformaciones de *loops*. 3. Optimizadores de *peepholes*. 4. Optimizaciones específicas de arquitectura. Con ello, el paso por disintos códigos internos, siendo instrucciones de tipo instancias *MCInst* anterior a obtención de código ensamblador o máquina.
Entradas	Código de tipo RTL, en particular, `.final`.	LLVM IR optimizado
Salidas	Código en lenguaje ensamblador `.s`.	Ídem

ELF. Estos archivos se obtienen a partir de aplicar las etapas de ensamblado (ver Sección 2.1.3) y enlazado (ver Sección 2.1.4) sobre el código ensamblador obtenido como resultado de la etapa de compilación.

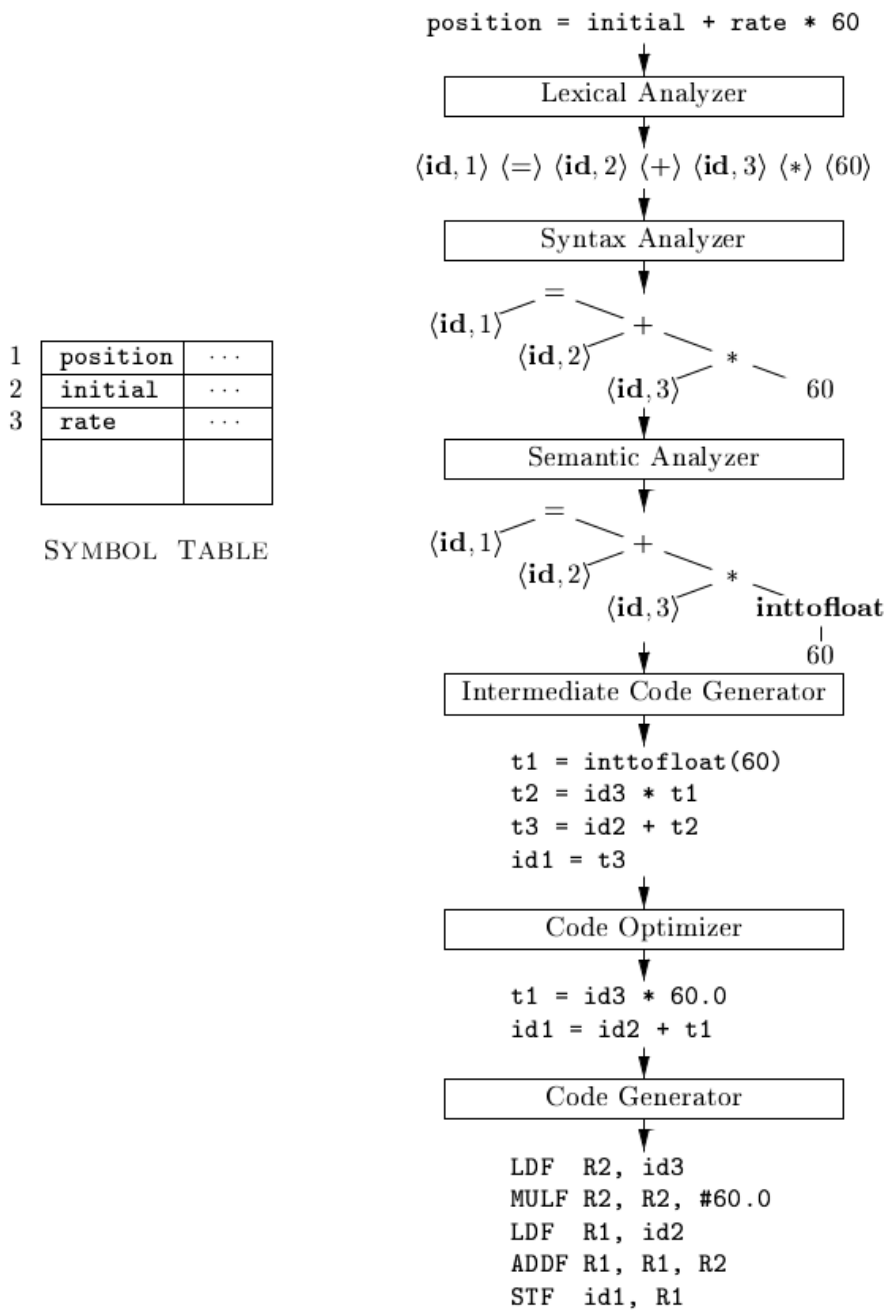


Figura 2.4: Ejemplo completo de fases de compilación (Fuente: Compilers: Principles, Techniques, and Tools, [6])

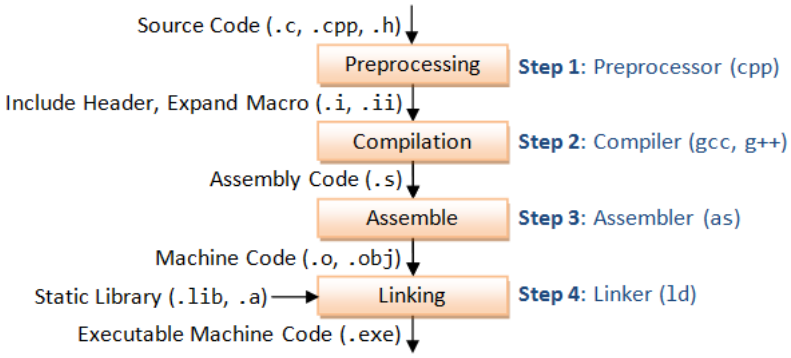


Figura 2.5: Etapas del proceso de compilación en GNU GCC, junto con sus entradas y salidas. (Fuente: GCC and Make - Compiling, Linking and Building C/C++ Applications, [24])

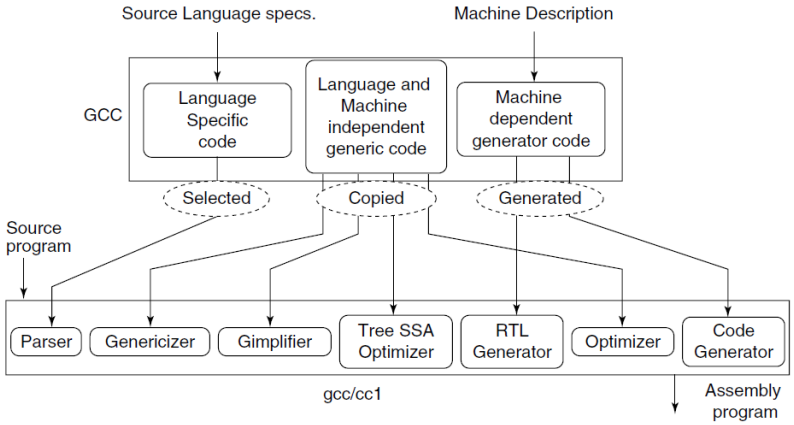


Figura 2.6: Módulos que componen la etapa de compilación en GNU GCC. (Fuente: Compilers: Principles and Practice, [7])

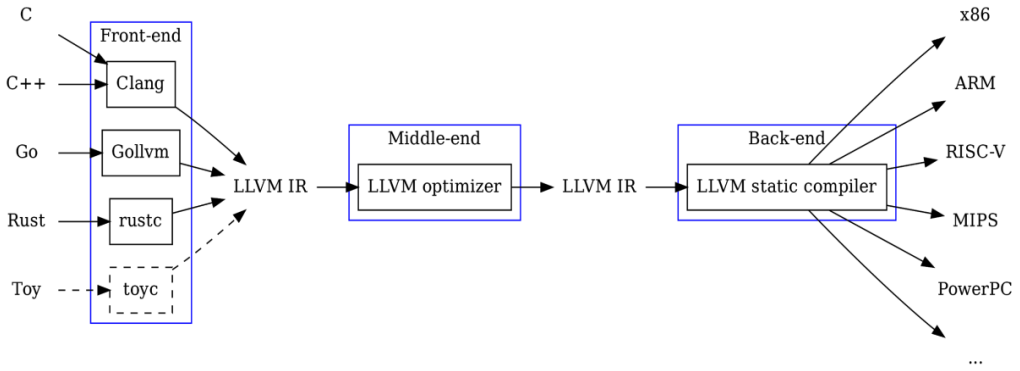


Figura 2.7: Esquema de la etapa de compilación en CLANG/LLVM. (Fuente: LLVM IR and Go, [25])

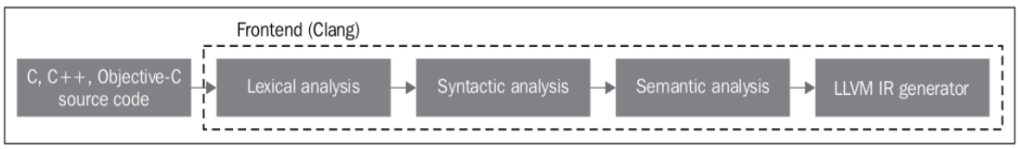


Figura 2.8: Partes de la estructura de CLANG. (Fuente: Getting Started with LLVM Core Libraries, [8])

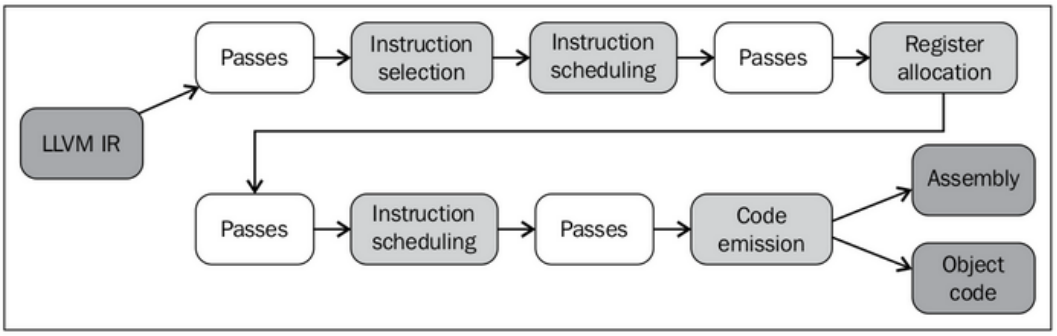


Figura 2.9: Partes de la estructura de LLVM. (Fuente:Getting Started with LLVM Core Libraries, [8])

Capítulo 3

Archivos ejecutables ELF

En el presente capítulo se analizarán los archivos ejecutables de formato ELF. En primer lugar, se expondrá una breve revisión sobre los archivos ejecutables y los principales formatos dentro de los sistemas *Linux*, *UNIX* y *Unix-like*. Posteriormente, se detallarán las características más importantes de los archivos binarios ELF, el formato ejecutable de mayor adopción. Finalmente, se procederá a detallar la estructura de un archivo ELF, analizando sus partes más relevantes.

3.1. Introducción a los archivos ejecutables

Antes de exponer acerca de los archivos ejecutables, presentaremos la fase de ensamble del proceso. se sitúa un paso atrás dentro de las etapas del proceso de compilación: la fase de ensamble.

Existen tres tipos de archivos objeto, así como se define en [9]:

- Archivos relocalizables: archivos que necesitan de la etapa de enlace junto con otros archivos objeto para poder completar las direcciones restantes para formar un ejecutable u objeto compartido.
- Archivos ejecutables: contiene un programa listo para ser ejecutado por el procesador.
- Archivos objeto compartidos: el código y los datos que contienen deben ser enlazados en dos etapas:
    1. Enlace con otros archivos objetos o archivos relocalizables para obtener un nuevo archivo objeto.
    2. Enlace dinámico para unir archivos ejecutables con archivos objeto compartidos, y así crear un nuevo ejecutable final.

El enfoque estará puesto en los **archivos ejecutables**.

Los archivos ejecutables son el resultado final de todo el proceso de compilación. Específicamente, son los obtenidos a la salida del proceso de enlace. Éstos pueden ser ejecutados sin necesitar una etapa extra de procesamiento.

Tabla 3.1: Principales formatos de códigos ejecutables en Unix.

OUT	COFF	ELF
Assembler Output Format (OUT) Es el formato más antiguo y clásico. Usa un *header* corto Contiene tres segmentos: ▪ *.text* ▪ *.data* ▪ *.bss*, con una tabla de simbolos y una tabla de *strings*	Common Object File Format (COFF) Implica un objeto de formato SVR3. El *header* está formado por una tabla de sección. Entonces, se puede tener más secciones aparte de *.text*, *.data* y *.bss*.	Executable and Linking Format (ELF) Es un formato sucesor a COFF. Tiene múltiples secciones. Versiones 32 o 64 bits. Es el formato estándar en Unix. Más detalle en Sección 3.2.

La ejecución comienza con una llamada a sistema que inicia la carga del ejecutable dentro del sistema operativo para crear la imágen del proceso correspondiente para, luego, retomar el control en espacio de usuario y ejecutar las instrucciones del programa cargado.

Dentro de los sistemas *Linux*, *UNIX* y *Unix-like*, los formatos más utilizados de ejecutables son los formatos OUT, COFF y ELF. Éstos se presentan en la Tabla 3.1;

3.2. Archivos ELF

El formato Executable and Linking Format (ELF) es un formato de archivos binario, tales como archivos objeto, librerías compartidas, archivos ejecutables y *core-dump* (archivos que contienen la imagen de la memoria cuando se produce un *crash* o fallo en un proceso o sistema).

El enfoque en este análisis está puesto sobre archivos ejecutables de formato ELF.

Los archivos ejecutables ELF han sido definidos como **formato estándar** para archivos binarios ejecutables en sistemas *Linux*, *UNIX* y *Unix-like* en procesadores x86 (32 bits) y x64 (64 bits).

Debido a cómo se encuentra organizada la estructura interna de los archivos ELF, es fácil su *parseo* hasta encontrar los datos necesarios; siendo esto aprovechado por todas aquellas herramientas que deben realizarlo de manera rápida y sin demasiadas complicaciones.

El formato ELF tiene las características de ser: **flexible**, **extensible** y **multiplataforma**. Es compatible con diferentes *endianness* y tamaños de direcciones (32 y 64 bits), pudiendo abarcar diferentes arquitecturas y sus correspondientes conjuntos de instrucciones, sistemas operativos y demás plataformas. Además, no tienen extensión definida. [30]

El ejemplo, Código 2.5, obtenido como resultado del proceso de compilación, muestra un archivo ELF ejecutable. Se verifica el tipo de archivo, arquitectura, el intérprete a usar, entre otra información.

3.2.1. Estructura

La estructura de los archivos ELF se encuentra organizada alrededor de tres componentes básicos:

1. Cabeceras (*headers*, en inglés)
2. Secciones.
3. Segmentos

La explicación de cada uno de estos componentes será presentada en los siguientes títulos. Las distintas estructuras que constituyen cada componente están definida dentro del *header* `elf.h`, encontrado en la dirección (*path*) `/usr/include/elf.h`.

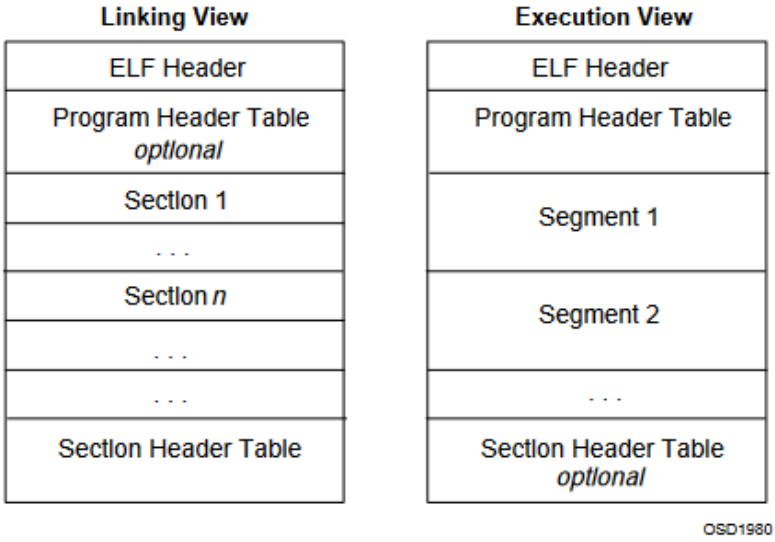


Figura 3.1: Vistas de un archivo ELF. (Fuente: Executable and Linking Format Specification, Version 1.2, [9])

Un archivo ELF puede verse de dos formas distintas: la vista de **enlace**, aquella que observa el *linker* al realizar el enlace entre los distintos archivos objeto; y la vista de **ejecución**, la que observa el sistema operativo al cargar el ejecutable para poder correr el programa. El esquema que plantea la Fig. 3.1 muestra las vistas nombradas.

3.2.1.1. Cabecera ELF (*ELF header*)

La cabecera ELF es un conjunto de bytes que se encuentran al principio de todo archivo ELF. Tiene un largo de 32 o 64 *bytes*, de acuerdo a la arquitectura correspondiente. El objetivo de la cabecera es identificar al archivo como un archivo formato ELF, así como proveer información general sobre el archivo: tipo, arquitectura, secciones, segmentos, etc; así como los *offset* para encontrar las estructuras relevantes.

La cabecera ELF define los requerimientos iniciales de memoria virtual que el proceso del ejecutable necesitará al ser cargado en el sistema.

La cabecera contiene los siguientes campos:

- `e_ident`: este campo contiene el "número mágico"que identifica a un archivo ELF (4 *bytes* - `0x7f` en hexadecimal), seguido por los valores también en hexadecimal del ASCII de las letras E (`0x45`), L (`0x4c`) y F (`0x46`). Luego, se encuentran bytes que representan distintos flags:
  - `EI_PAD`: *padding*.
  - `EI_CLASS`: hace referencia a la arquitectura 1, si es de 32 *bits*; o 2, si es de 64 *bits*.
  - `EI_DATA`: hace referencia al *endianess*: 1, si *little-endian*; o 2 si *big-endian*.
  - `EI_VERSION`: indica la versión de ELF (actualmente 1).
  - `EI_OSABI`: indica que el archivo contiene o no secciones no estándar. 0, si concuerda con estándar ABI de *UNIX System V*; otro valor, para versiones específicas de ABI.
- `e_type`: indica el tipo de archivo binario ELF al que pertenece: `ET_REL`, si se refiere a un archivo objeto relocalizable; `ET_EXEC`, ejecutable; `ET_DYN`, archivo objeto compartido (librería dinámica, o también llamadas librería compartida).
- `e_machine`: precisa la arquitectura para la cual fue compilado el archivo.
  Ejemplos: `EM_X86_64`, `EM_MIPS`, `EM_ARM`, etc.
- `e_version`: indica la versión ELF del archivo. Actualmente, existe un único valor válido: 1 (`EV_CURRENT`).
- `e_entry`: detalla el punto de entrada (*entry point*) al proceso, la dirección virtual a la cuál intérprete del binario transfiere el control después de cargar el binario en memoria virtual. Es en esa dirección en la cuál comienza la ejecución del programa.
- `e_phoff`: indica el *offset* a la tabla de cabeceras de programa (tabla que contiene los segmentos del archivo ELF). Desplazándose la cantidad de *bytes* indicados por este campo desde el inicio del archivo, se llega al inicio de la mencionada tabla.
- `e_shoff`: indica el *offset* a la tabla de secciones. Desplazándose la cantidad de *bytes* indicados por este campo desde el inicio del archivo, se llega al inicio de la tabla de cabeceras de secciones.
- `e_flags`: hace referencia a *flags* específicos de la arquitectura destino del archivo.
- `e_ehsize`: indica el tamaño total de la cabecera ELF en *bytes*.
- `e_shentrsize`: señala el tamaño de una cabecera de sección en *bytes*. Una cabecera de sección (*section header*) es una entrada en la tabla de secciones del archivo ELF. Todas las cabeceras de sección tienen el mismo tamaño.
- `e_shnum`: denota la cantidad de entradas de cabeceras de sección en la tabla de cabeceras de secciones.
- `e_phentrsize`: indica el tamaño de una cabecera de programa en *bytes* de la tabla de cabeceras de programas. Una cabecera de programa (*program header*) es una entrada en la tabla de cabeceras de programas. Todas las cabeceras de programas tienen igual tamaño.
- `e_phnum`: denota la cantidad de entradas de cabeceras de programas en la tabla de cabeceras de programas.

```
$ readelf -h ejemplo

ELF Header:
  Magic:   7f 45 4c 46 02 01 01 09 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            FreeBSD
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices x86-64
  Version:                           0x1
  Entry point address:               0x4004d0
  Start of program headers:          64 (bytes into file)
  Start of section headers:          11432 (bytes into file)
  Flags:                             0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         8
  Size of section headers:           64 (bytes)
  Number of section headers:         36
  Section header string table index: 35
```

Código 3.1: Información de la cabecera ELF del ejecutable `ejemplo` usando la herramienta `readelf`.

- `e_shstrndx`: indica el índice dentro de la tabla de cabecera de secciones (*section header table*) que contiene la cabecera de la sección `.shstrtab`. Dicha sección contiene la tabla de *strings*. La tabla de *strings* almacena los nombres de todas las secciones presentes en el archivo ELF.

Siguiendo con el ejecutable de ejemplo, se muestra en el Código 3.1 la salida al analizar la cabecera ELF del mismo.

3.2.1.2. Secciones

Las secciones son un grupo de bytes dentro del archivo ELF que representan información común agrupada de manera ordenada. Las secciones responden a una organización o separación entre el código y los datos del archivo binario.

Cada sección ocupa una secuencia continua de bytes dentro del archivo ELF. Sin embargo, pueden existir *bytes* que no pertenezcan a ninguna sección. Estos últimos *bytes* se consideran como datos no especificados. Las secciones no se suporponen, es decir, ningún *byte* del archivo pertenece a más de una sección.

Las secciones responden a la vista que se tiene en la etapa de enlace para reorganizar los datos estáticos del archivo objeto en un archivo final ejecutable tipo ELF. No todas las secciones serán cargadas finalmente en memoria. Algunas de ellas solamente sirven para dar información extra al *linker* y no son necesarias para la ejecución.

Las secciones especiales de un archivo ELF, contienen información importante para el sistema. Ellas son las siguientes:

- `.init`: contiene código ejecutable utilizado para llevar a cabo tareas y acciones de inicialización que deben ejecutarse antes del inicio del programa. El sistema operativo ejecuta las instrucciones de esta sección para luego, transferir el control al punto de entrada del binario ejecutable.
- `.fini`: al finalizar sin errores el programa, el sistema ejecuta el código de esta sección para realizar acciones de terminación de código.
- `.init_array`: determina que la sección contiene un arreglo de punteros a funciones, llamadas todas durante el proceso de inicialización del programa.
- `.fini_array`: la sección almacena un arreglo de punteros a funciones que se ejecutarán después de finalizar correctamente el programa.
- `.text`: contiene el código principal del programa. Sección de tipo `SHT_PROGBITS` indicando que contiene información definida por el usuario.
- `.bss`: reserva espacio para variables no inicializadas. La sección es de tipo `SHT_NOBITS` ya que no ocupa espacio en el archivo ELF (tamaño cero *bytes* en el archivo) pero indica que al cargar el proceso en memoria, se deben reservar X *bytes* para contener variables no inicializadas.
- `.data`: guarda los valores asignados en la declaración de variables.
  Tipo de sección: `SHT_PROGBITS`.
- `.rodata`: contiene los valores de las variables definidas como constantes.
  Tipo: `SHT_PROGBITS`.
- `.rel*`: secciones que contienen información usada por el *linker* para relocalización. El nombre está dado por la sección a la cuál se aplicará la relocalización. Por ejemplo: la sección de relocalización para la sección `.text` será `.rel.text`.
- `.rela*`: idem anterior pero la relocalización recae sobre sección que serán parte de un segmento cargable en el proceso. Por lo tanto, se pone en alto el *flag*: `SHF_ALLOC`, indicando que la sección ocupará memoria durante la ejecución.
- `.dynamic`: señala que la sección contiene información de enlace dinámico.
  Atributos: `SHF_ALLOC` y/o `SHF_WRITE`.
- `.shstrtab`: sección que contiene una lista de *strings* con los nombres de todas las secciones del archivo.
- `.symtab`: indica que la sección contiene una tabla de símbolos, que vinculan un nombre simbólico de, por ejemplo: una función, con información correspondiente dentro del archivo.
- `.strtab`: explicita que la sección almacena los *strings* de los nombres simbólicos mencionados anteriormente.
- `.dynsym`: la sección incluye una tabla de símbolos usados al realizar enlace dinámico.
- `.dynstr`: contiene *strings* que representan a los nombres simbólicos dinámicos anteriores.

3.2.1.2.1. Cabeceras de secciones (*Section headers*) Toda sección está descripta por su correspondiente cabecera de sección.

Todas las cabeceras de sección se encuentran agrupadas en la tabla de cabeceras de sección (*section header table* en inglés).

Los archivos ELF que no necesitan enlazamiento pueden prescindir de la tabla de cabeceras de sección.

La estructura que describe una cabecera de sección se encuentra en la dirección (*path*) `/usr/include/elf.h` bajo el nombre de `Elf32_Shdr` o `Elf64_Shdr`.

Los campos de la estructura de la cabecera de sección ELF son:

- `sh_name`: Con valor 1, si la sección a la que refiere contiene una entrada en la tabla de strings (sección `.shstrtab`). Con valor 0, si la sección no tiene nombre.
- `sh_type`: indica el tipo de sección:
  - `SHT_PROGBITS`: sección contiene datos de programa suministrados por el usuario (código, variables).
  - `SHT_SYMTAB`: almacena tablas de símbolos estáticas.
  - `SHT_DYNSYM`: contiene tablas de símbolos usadas durante el enlace dinámico.
  - `SHT_STRTAB`: almacena tablas de *strings*.
  - `SHT_REL`: contiene información usada en la relocalización.
  - `SHT_RELA`: contiene información usada en la relocalización, que ocupará lugar en memoria del proceso.
  - `SHT_DYNAMIC`: guarda información usada por el enlazador (*linker*) dinámico.
- `sh_flags`: señala los atributos de la sección a la que apunta. Algunos *flags* importantes son:
  - `SHF_WRITE`: sección contiene datos que pueden sobreescribirse durante la ejecución del proceso.
  - `SHF_ALLOC`: la sección contiene datos que se cargarán en memoria al ejecutarse el proceso.
  - `SHF_EXECINSTR`: sección contiene código ejecutable.
  - `SHF_STRINGS`: la sección almacena *strings*.
- `sh_addr`: indica la dirección virtual que tomará la sección al cargarse en memoria. Si la sección no se cargará en memoria, entonces este campo es cero.
- `sh_offset`: señala el *offset* en *bytes* desde el inicio del archivo hasta el inicio de la sección correspondiente.
- `sh_size`: presenta el tamaño de la sección en *bytes*.
- `sh_link`: define la relación de la sección con el índice de la tabla de cabeceras de sección. Dicha relación tiene un significado diferente dependiendo del tipo de la sección (`sh_type`).
- `sh_info`: define información extra de la sección cuyo significado está dado por el tipo de sección.
- `sh_addralign`: indica el valor de alineamiento a memoria que debe tener la sección al ser cargado en memoria.
- `sh_entsize`: si la sección correspondiente contiene una tabla con entradas de tamaño fijo, este campo señala el tamaño de las entradas de dicha tabla.

La tabla del ejecutable `ejemplo`, utilizado como ilustración en este capítulo, se muestra en el Código 3.2.

```
$ readelf -S ejemplo

There are 36 section headers, starting at offset 0x2ca8:

Section Headers:
[Nr] Name              Type            Address          Offset    Size   EntSize  Flags  Link  Info  Align
[ 0] NULL                              000000           00000000  0000   0000              0     0     0
[ 1] .interp           PROGBITS        400200           00000200  0015   0000      A       0     0     1
[ 2] .note.tag         NOTE            400218           00000218  0048   0000      A       0     0     4
[ 3] .hash             HASH            400260           00000260  0034   0004      A       4     0     8
[ 4] .dynsym           DYNSYM          400298           00000298  00c0   0018      A       5     1     8
[ 5] .dynstr           STRTAB          400358           00000358  006e   0000      A       0     0     1
                       ...
[ 8] .rela.dyn         RELA            4003f8           000003f8  0030   0018      A       4     0     8
[ 9] .rela.plt         RELA            400428           00000428  0048   0018     AI       4    21     8
[10] .init             PROGBITS        400470           00000470  0013   0000     AX       0     0     4
[11] .plt              PROGBITS        400490           00000490  0040   0010     AX       0     0     16
[12] .text             PROGBITS        4004d0           000004d0  03f4   0000     AX       0     0     16
[13] .fini             PROGBITS        4008c4           000008c4  000e   0000     AX       0     0     4
[14] .rodata           PROGBITS        4008d2           000008d2  0001   0001    AMS       0     0     1
                       ...
[19] .dynamic          DYNAMIC         600978           00000978  0190   0010     WA       5     0     8
                       ...
[22] .data             PROGBITS        600b48           00000b48  0010   0000     WA       0     0     8
[23] .bss              NOBITS          600b58           00000b58  0018   0000     WA       0     0     8
[24] .comment          PROGBITS        000000           00000b58  0094   0001     MS       0     0     1
                       ...
[33] .symtab           SYMTAB          000000           000024a8  0468   0018            34    30     8
[34] .strtab           STRTAB          000000           00002910  024d   0000             0     0     1
[35] .shstrtab         STRTAB          000000           00002b5d  0144   0000             0     0     1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings)
  I (info), L (link order), G (group), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)
```

Código 3.2: Tabla de cabeceras de sección del ejecutable `ejemplo`.

```
$ readelf -l ejemplo

Section to Segment mapping:
Segment Sections...
    00
    01     .interp
    02     .interp .note.tag .hash .dynsym .dynstr .gnu.version .gnu.version_r .
    ↪ rela.dyn .rela.plt .init .plt .text .fini .rodata .eh_frame_hdr .eh_frame
    03     .ctors .dtors .dynamic .got .got.plt .data .bss
    04     .dynamic
    05     .note.tag
    06     .eh_frame_hdr
    07
```

Código 3.3: Mapa de secciones y segmentos para el ejecutable `ejemplo`.

3.2.1.3. Segmentos

Un segmento es una unidad de organización que contiene datos que el sistema operativo y el enlazador dinámico (*dynamic linker*) necesitan para crear la imagen del proceso y cargar dichos datos en memoria previa a la ejecución del archivo.

Los segmentos forman parte de la vista de ejecución de un archivo ELF. Es por ello que los segmentos solo son relevantes para archivos ejecutables o archivos objetos compartidos (librerías compartidas)

Los segmentos contienen cero o más secciones agrupadas y combinadas.

3.2.1.3.1. Mapeo de secciones a segmentos La herramienta `readelf` permite obtener de manera rápida y fácil de entender la correspondencia entre los segmentos y las secciones que los componen.

Continuando con el ejecutable ELF usado como ejemplo, se observa dicho mapeo en el Código 3.3

3.2.1.3.2. Cabeceras de programas (*Program headers*) Todo segmento está descripto por una cabecera de programa.

Al igual que las cabeceras de secciones, las cabeceras de programa forman una **tabla de cabeceras de programa**.

La estructura de una cabecera de programa se divisa en el *header*: `/usr/include/elf.h`, dentro de las estructuras `Elf32_PHdr` y `Elf64_PHdr`.

Los campos de las cabeceras de programa son los siguientes:

- `p_type`: indica el tipo de segmento. Un segmento puede ser de tipo:
  - `PT_NULL`: Segmento no usado.
  - `PT_LOAD`: Los datos del segmento serán cargados a la memoria del proceso.
  - `PT_DYNAMIC`: segmento contiene información usada en el enlace dinámico.
  - `PT_INTERP`: contiene el *path* del interpretador a usar.

```
$ readelf -l ejemplo

Elf file type is EXEC (Executable file)
Entry point 0x4004d0
There are 8 program headers, starting at offset 64

Program Headers:
    Type           Offset   VirtAddr PhysAddr FileSiz  MemSiz    Flg    Align
    PHDR           0x000040 0x400040 0x400040 0x0001c0 0x0001c0  R      0x8
    INTERP         0x000200 0x400200 0x400200 0x000015 0x000015  R      0x1
        [Requesting program interpreter: /libexec/ld-elf.so.1]
    LOAD           0x000000 0x400000 0x400000 0x000954 0x000954  R E    0x200000
    LOAD           0x000958 0x600958 0x600958 0x000200 0x000218  RW     0x200000
    DYNAMIC        0x000978 0x600978 0x600978 0x000190 0x000190  RW     0x8
    NOTE           0x000218 0x400218 0x400218 0x000048 0x000048  R      0x4
    GNU_EH_FRAME   0x0008d4 0x4008d4 0x4008d4 0x00001c 0x00001c  R      0x4
    GNU_STACK      0x000000 0x000000 0x000000 0x000000 0x000000  RW     0x10
```

Código 3.4: Tabla de cabeceras de programa para el ejecutable `ejemplo`.

  - `PT_NOTE`: segmento contiene información adicional.
  - `PT_SHLIB`: segmento reservado.
  - `PT_PHDR`: el segmento contiene la localización y el tamaño de la tabla de cabeceras de programa con respecto al archivo y en memoria del programa.
  - `PT_LOPROC` y `PT_HIPROC`: información del procesador.
- `p_flags`: indica atributos del segmento. Pueden ser: `PF_X`, información ejecutable (código); `PF_W`, información que puede sobreescribirse; `PF_R`, información de solo lectura.
- `p_offset`: detalla el *offset*, los *bytes* que hay que leer desde el inicio del archivo hasta el comienzo del segmento.
- `p_vaddr`: señala la dirección virtual del primer *byte* del segmento al cargarse en memoria.
- `p_paddr`: indica la dirección física del segmento en memoria. En la mayoría de los sistemas operativos modernos, este campo no es relevante ya que los programas se ejecutan en memoria virtual.
- `p_filesz`: detalla el tamaño del segmento en el archivo ELF, medido en *bytes*.
- `p_memz`: informa el tamaño del segmento en *bytes* en la imagen del proceso al ser cargado en memoria.
- `p_align`: definen el alineamiento a memoria que debe tener el segmento, en *bytes*.

En el Código 3.4, se observa la tabla de cabeceras de programa del archivo de ejemplo.

Reseña del capítulo

En el presente capítulo se ha introducido el concepto de archivos ejecutables. Específicamente, se ha detallado los archivos ejecutables ELF analizando su estructura en sus dos vistas, la vista de enlace y la vista de ejecución. Seguido a ello, se ha descrito cada una de las partes que conforman

un archivo ejecutable ELF indicando la información que contienen sus campos, haciendo uso del ejecutable `ejemplo`.

El capítulo siguiente se encarga de explicar el proceso de carga de los archivos ejecutables de tipo ELF en *kernel* de FreeBSD. Es decir, el camino que realiza en espacio de *kernel* hasta que se encuentra listo para su ejecución en espacio de usuario. “‘“”’

Capítulo 4

Proceso de carga de ejecutables ELF en *kernel* de FreeBSD

Introducción

En el presente capítulo se expondrán detalles acerca del proceso que inicia con la carga del ejecutable en espacio de *kernel* del sistema operativo, hasta el momento en que, finalmente, se ejecuta el programa en espacio de usuario. En primer lugar, se detallarán las llamadas a sistema y demás funciones realizadas para iniciar la carga del ejecutable. Se presentarán aspectos sobre cómo, ya en espacio de *kernel*, se extrae la información desde el ejecutable para crear el proceso en el sistema. Posteriormente, se brindarán los pormenores de las acciones llevadas a cabo durante el enlace dinámico ejecutado por el intérprete.

4.1. Carga del programa en el sistema operativo

Un proceso necesita de diferentes recursos, entre ellos se encuentran el *stack* dentro del *kernel* y espacio de memoria virtual para el mismo. El espacio de memoria virtual incluye espacio necesario para alojar los segmentos de código (`.text`), los segmentos que representan datos (variables) inicializados y no inicializados, y el espacio para el *stack* de tiempo de ejecución (*runtime stack*), espacio de memoria usado para el pasaje y retorno de valores hacia y desde funciones.

La carga del archivo ejecutable en el sistema se inicia con la acción de ejecución en espacio de usuario.

Esta acción generá la invocación a la llamada de sistema: `execve` [31] (Ver Código 4.1). El prototipo de la función `execve` es:

```
int execve(const char *path, char *const argv[], char *const envp[])
```

```
$ ktrace ./ejemplo
$ ktrace -C
$ kdump

5582 ktrace   RET   ktrace 0
5582 ktrace   CALL  execve(0x7ffffffedbf,0x7ffffffeaf0,0x7ffffffeb00)
5582 ktrace   NAMI  "./ejemplo"
5582 ktrace   NAMI  "/libexec/ld-elf.so.1"
5582 ejemplo  RET   execve JUSTRETURN
5582 ejemplo  CALL  mmap(0,0x21000,0x3<PROT_READ|PROT_WRITE>,0x1002<MAP_PRIVATE|
    ↪ MAP_ANON>,0xffffffff,0)
5582 ejemplo  RET   mmap 34366160896/0x800620000
5582 ejemplo  CALL  mprotect(0x80061d000,0x1000,0x1<PROT_READ>)
5582 ejemplo  RET   mprotect 0
5582 ejemplo  CALL  issetugid
5582 ejemplo  RET   issetugid 0

...
```

Código 4.1: Llamadas realizadas al ejecutar el archivo `ejemplo` usando la herramienta `ktrace`.

`Execve` tiene como objetivo crear un nuevo proceso a partir del archivo definido en el argumento `path`.

La llamada a sistema `execve`, se traduce en las llamadas a las funciones de *kernel*: `sys_execve`, `kern_execve` y, finalmente, `do_execve`.

La función `do_execve` se encarga, entre otras cosas, de: crear y *setear* la estructura necesaria para crear la imagen del proceso (estructura **`proc`**) y la estructura que contiene la información requerida para realizar la carga, activación e interpretación del ejecutable (estructura **`image_params`**); modificar las credenciales de usuario, dar a conocer el *path* al archivo ejecutable en distintas estructuras e iterar sobre los distintos activadores de imágenes e invocar al intérprete, si la imagen del ejecutable lo requiere.

La estructura `image_params`, definida en `sys/sys/imgact.h`, representa la imagen del archivo binario a ejecutar. Dicha estructura almacena información útil para completar el proceso de carga. El Código 4.2 muestra el contenido de dicha estructura.

Alguno de los campos más relevantes son:

- `proc`: puntero a la estructura `proc` relacionada al proceso del ejecutable.
- `*vp`: puntero al vnodo (*vnode*), representación interna al *kernel* de todo archivo o directorio del sistema. En este caso, representación del ejecutable ELF en *kernel*.
- `*image_header`: inicio del archivo a ejecutar.
- `entry_addr`: dirección en memoria virtual del punto de entrada del programa, dirección en la que comenzará el programa al finalizar la carga del mismo.
- `reloc_base`: dirección virtual base en la cual el archivo será cargado en memoria durante la ejecución para la relocalización de objetos.
- `*interpreter_name`: nombre del intérprete del archivo.

```
struct image_params {
    struct proc *proc;  /* our process struct */
    struct label *execlabel;  /* optional exec label */
    struct vnode *vp; /* pointer to vnode of file to exec */
    struct vm_object *object; /* The vm object for this vp */
    struct vattr *attr; /* attributes of file */
    const char *image_header; /* head of file to exec */
    unsigned long entry_addr; /* entry address of target executable */
    unsigned long reloc_base; /* load address of image */
    char vmspace_destroyed; /* flag - we've blown away original vm space */
#define IMGACT_SHELL  0x1
#define IMGACT_BINMISC  0x2
    unsigned char interpreted;  /* mask of interpreters that have run */
    char opened;    /* flag - we have opened executable vnode */
    char *interpreter_name; /* name of the interpreter */
    void *auxargs;    /* ELF Auxinfo structure pointer */
    struct sf_buf *firstpage; /* first page that we mapped */
    unsigned long ps_strings; /* PS_STRINGS for BSD/OS binaries */
    size_t auxarg_size;
    struct image_args *args;  /* system call arguments */
    struct sysentvec *sysent; /* system entry vector */
    char *execpath;
    unsigned long execpathp;
    char *freepath;
    unsigned long canary;
    int canarylen;
    unsigned long pagesizes;
    int pagesizeslen;
    vm_prot_t stack_prot;
    u_long stack_sz;
    struct ucred *newcred;    /* new credentials if changing */
    bool credential_setid;    /* true if becoming setid */
};
```

Código 4.2: Estructura `image_params` en FreeBSD. Definida en `imgact.h`.

```
for (i = 0; error == -1 && execsw[i]; ++i) {
    if (execsw[i]->ex_imgact == NULL ||
        execsw[i]->ex_imgact == img_first) {
        continue;
    }
    error = (*execsw[i]->ex_imgact)(imgp);
}
```

Código 4.3: Iteración sobre activadores de imágenes. Función `do_execve` en `kern_exec.c`..

- `auxargs`: puntero a la estructura `Elf_Auxargs` (`__ElfN(Auxargs)`), definida en `imgact_elf.h` y usada para enviar información del ELF desde el activador de imagen a la rutina de *stack fixup* (`__elfN(freebsd_fixup)`).
  Fix up: reemplazo de un símbolo externo por su dirección efectiva por parte del *linker*.
- `image_args *args`: puntero a estructura que contiene los argumentos del programa sumados al lanzar la ejecución en espacio de usuario.
- `*newcred`: puntero a la estructua `ucred` que representa modificaciones a las credenciales de usuario actuales dado a que la ejecución del programa requiere cambios sobre las mismas.

Posteriormente, el proceso en el *kernel* de FreeBSD continúa con la iteración sobre los distintos activadores de imágenes.

Al tratarse de un archivo ELF, la iteración anterior termina al realizar el *match* al activador correspondiente. Éste activador deriva en la ejecución de la función dentro de `imgact_elf.c`

```
static int __CONCAT(exec_, __elfN(imgact))(struct image_params *imgp)
```

Las tareas ejecutadas por esta función son:

- *Parsear* sobre la tabla de cabeceras de programa del archivo ELF, encontrar los segmentos de tipo `PT_LOAD`, `PT_INTERP`, `PT_GNU_STACK` y extraer información de ellos.
- Cargar los segmentos de tipo `PT_LOAD` a memoria. (funciones `__elfN(load_file)` y `__elfN(load_section)`), así como *setear* el punto de entrada al archivo cargado en memoria y la dirección donde efectivamente debe cargarse el archivo dentro de la memoria de *kernel*.
- Detectar qué tipo de archivo binario es el que se está *parseando*.
  Función `__elfN(get_brandinfo)`
- Remover el espacio de memoria virtual anterior, crear un nuevo espacio de memoria virtual para el nuevo proceso y reservar memoria para el *stack* del nuevo proceso. Función `exec_new_vmspace` en `kern_exec.c`.
- Setear *flags* de la imagen del ejecutable de acuerdo a lo encontrado en los segmentos del ejecutable.
- Encontrar el segmento ejecutable más largo dentro del ELF y definirlo como el segmento `.text` principal, haciendo al resto parte del segmento `.data`.
- Chequear que los segmentos `.text`, `.data` y todos los segmentos en conjunto cumplan con los límites del proceso o de recursos del sistema.

- Encontrar el intérprete del archivo ELF y anotarlo.
- Rellenar los valores de la estructura de tipo `Elf_Auxargs` para poder realizar la rutina de *fix-up*.

La función `__elfN(load_section)` termina llamando a la función `__elfN(map_insert)`. A su vez, esta última finaliza en la función `vm_map_insert`, que toma los datos de las secciones del archivo ejecutable cargado en memoria virtual y realiza el mapeo correspondiente al espacio de memoria virtual final reservado para el programa.

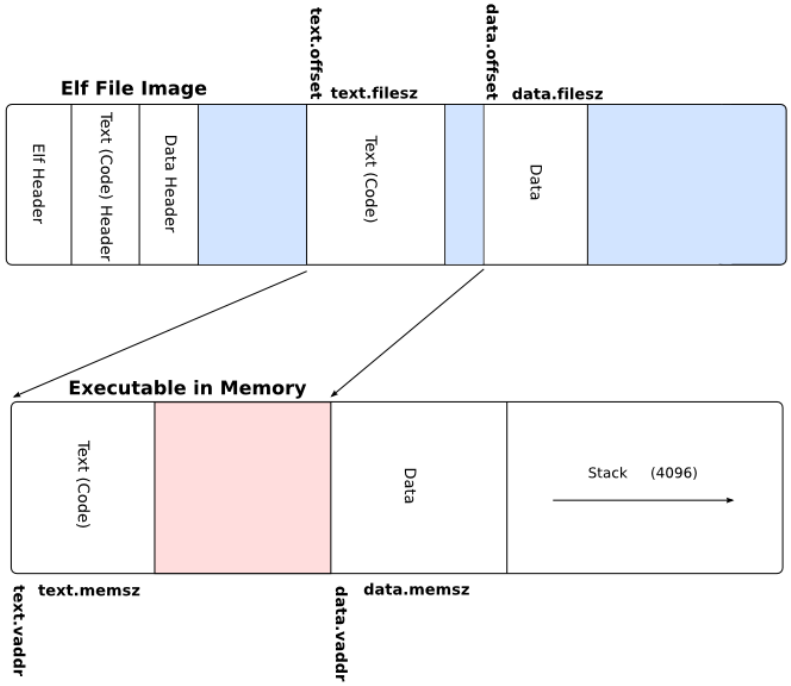


Figura 4.1: Mapeo entre el archivo ELF y la memoria virtual del proceso asociado. (Fuente: ELF, [32])

El mapeo mencionado se observa en la Fig 4.1.

Luego, si el programa utiliza librerías compartidas, el intérprete es ejecutado. Más información en Sección 4.2.

El proceso de carga termina al finalizar la llamada de la función `do_exec` y volver desde la llamada de sistema `execve` a un espacio de usuario distinto al desde el cual se hizo la llamada, dado el *remapeo*. El nuevo espacio de usuario corresponde al espacio anterior pero remapeado para alojar al nuevo proceso, con un punto de entrada y registros distintos, listos para iniciar la ejecución efectiva del nuevo programa.

4.2. Enlazamiento dinámico (intérprete)

El intérprete es un archivo compartido que se ejecuta para ser usado como el enlazador dinámico del ejecutable que se carga en el sistema operativo. Este se encarga de crear la imagen del proceso del

programa que provee el ejecutable.

El intérprete es ejecutado solamente para archivos ELF ejecutables que utilizan librerías compartidas.

El intérprete es cargado implicitamente junto al archivo ejecutable ELF que se busca ejecutar. En el Código 4.1, se observa que se pasa a la llamada de sistema `execve` el *path* de ambos elementos.

Relacionado al primero, el enlazador dinámico (también llamado *dynamic linker*) tiene como objetivo encontrar y realizar el mapeo y enlace de librerías compartidas (*shared libraries*) usadas en el ejecutable a cargar.

El *kernel* no lleva a cabo el mapeo de librerías compartidas durante la carga del ejecutable antes de iniciar el programa. Ésto es menester del enlazador dinámico.

Ésta etapa sucede después de finalizar la carga del ejecutable en el *kernel*, resultante del parseo anterior del ejecutable.

El mapeo se realiza usando la función `mmap()`. El enlace involucra resolver las referencias externas a librerías compartidas presentes en el programa principal.

La cabecera de programa `PT_INTERP` almacena el nombre del programa intérprete dentro del ejecutable. Si el *flag* "interpretado" está en alto, es decir, si necesita de la ejecución del intérprete, se toma el nombre del intérprete desde la estructura `image_params`, mostrada en el Código 4.2. El sistema operativo, entonces, invoca al enlazador dinámico como el programa intérprete. El resultado del mismo es la creación de la imagen inicial del proceso a partir de los segmentos presentes en el archivo compartido que conforma al intérprete.

Tal como se observa en el Código 2.5, el programa que contiene el enlazador dinámico por defecto de FreeBSD está definido en: `/libexec/ld-elf.so.1` [33].

Una vez terminada la funcionalidad del enlazador dinámico como intérprete, se transfiere el control desde el código del enlazador dinámico al punto de entrada (*entry point*) del programa que se carga en el *kernel* para iniciar, efectivamente, la ejecución del mismo.

Reseña del capítulo

En el presente capítulo se ha mostrado como un archivo ejecutable es cargado en memoria en espacio de *kernel*. Específicamente, se han detallado las funciones en su orden de llamada, como así también las estructuras que completan.

El capítulo siguiente se encarga de explicar la evolución iterativa de trabajo, marcando los objetivos propuestos de cada iteración, la implementación llevada a cabo y los resultados obtenidos.

Parte II

Marco metodológico

Capítulo 5

Iteraciones

El presente capítulo estará destinado a detallar los diferentes ciclos de trabajo, o iteraciones, que se llevaron a cabo.

En cada iteración se presentarán:

- El planteo de los objetivos a alcanzar al finalizar la iteración.
- La propuesta encarada para cubrir los objetivos mencionados anteriormente.
- La implementación del modelo propuesto.
- La evaluación final del resultado obtenido en base a: los objetivos planteados y los resultados obtenidos.
- Los pasos a seguir en la próxima iteración.

5.1. Primera iteración

5.1.1. Objetivos a alcanzar

En esta iteración se ha tenido como objetivos encontrar e imprimir la sección *.metadata* en espacio de *kernel* de FreeBSD, versión 11.

5.1.2. Propuesta

La propuesta en esta iteración fue la de implementar un módulo de *kernel* que encuentre e imprima la sección *metadata* en espacio de *kernel*. Éste módulo fue pensado para ser ejecutado en el momento que el sistema operativo cargue el archivo ELF, previo a su ejecución.

El mensaje insertado ha sido una simple cadena de caracteres utilizando el método que combina la palabra reservada `__attribute__`, la función atributo `section` en una línea de definición de un *char array* global. Ver Código 5.1 [18]

```
#ifdef COMENTARIO
    const char str[] __attribute__((section(".metadata"))) = COMENTARIO;
#else
#endif
```

Código 5.1: Método inicial de inserción en sección metadata [34].

5.1.3. Implementación

Para implementar la propuesta en el código fuente, se procedió a crear el archivo `modkernel_LectorELF.c` [35] a partir de un ejemplo de un *driver* con la funcionalidad de realizar un *cat* sobre un archivo. Entonces, los pasos que realiza el módulo de kernel obtenido son:

- Abrir el archivo ELF y leer la cabecera ELF que contiene toda la información sobre el archivo ELF.
- Leer la cabecera de la tabla de cabecera de secciones que contiene toda la información sobre las secciones del archivo.
- Iterar sobre las entradas de la tabla y leer el nombre de la sección a la que apuntan hasta encontrar la sección *metadata*.
- Por último, imprimir el contenido de la sección *metadata*.

Éste código ha sido fundamental para poder conocer las limitaciones en cuanto a reserva de memoria en espacio de kernel y las funciones disponibles, así como su reemplazo, dentro del espacio privilegiado.

La implementación de esta iteración puede verse en los *tags* del s1 y s2 [36].

5.1.4. Análisis de resultados

A continuación se presentarán las ventajas y desventajas de esta iteración.

Ventajas:

- El *payload* de la sección se recupera sin modificaciones.
- Conocimiento de instrucción para reserva de memoria y manejo de archivos en espacio de *kernel*.
- Conocimiento de funciones disponibles y no disponibles en espacio de *kernel*.
- Implementación que puede activarse y desactivarse sin ejecutarse siempre en *kernel*.

Desventajas:

- El *payload* de la sección *metadata* solo se imprime, sin aportar una funcionalidad útil. Se debería plantear como vincularlo con otras funciones de *kernel*.
- Requiere de la existencia de un disparador o *trigger* para poder ejecutarse cuando esté en funcionamiento efectivo.

5.1.5. Próximos pasos

Al visualizarse la necesidad de un evento que dispare la carga y descarga del módulo (carga de un archivo ejecutable en *kernel*) y que, inicialmente, se decidió que los metadatos serán usados por otras partes de la *kernel* del sistema operativo, se decidió que se explorará la implementación lograda dentro del código de *kernel*. Caso contrario, se había decidido volver a pensar el módulo e indagar sobre como setear el *trigger* ante carga de ELF.

5.2. Segunda iteración

5.2.1. Objetivos a alcanzar

El objetivo de esta iteración ha sido crear un parche de *kernel* que permita encontrar y decodificar la metadata insertada en los ejecutables ELF que se cargan en el sistema operativo.

5.2.2. Propuesta

En primer lugar, se ha planteado encontrar el lugar dentro del código de *kernel* en el cual se procesa el archivo ELF ejecutable que se carga al iniciar la ejecución de un programa. Se ha planteado que se elegirá aquel lugar en donde el archivo ELF ya está abierto y haya sido más directo su manipulación.

Luego, una vez encontrado el lugar, escribir el código que permita de forma modularizada y siguiendo los lineamientos de desarrollo de sistemas operativos, implementar la decodificación de los metadatos. Se planteó también modificar lo menos posible el código ya existente de *kernel*.

Finalmente, se ha decidido que, dado que la decodificación implica una iteración sobre todas las secciones del ELF, se debería indicar el resultado de dicha búsqueda de la sección `.metadata`. Así, no se debería realizar la búsqueda de sección nuevamente al extraer la información. Sumado a esto, la extracción debería ejecutarse solamente si el resultado de la búsqueda es exitoso. Con esto también se busca no sumar procesamiento excesivo al *kernel*. Recordar que este enfoque (parche en lugar de módulo) implica que la detección de la sección `.metadata` se realizará para todo ELF ejecutable lanzado por el sistema.

5.2.3. Implementación

Utilizando el mapa de funciones que se ejecutan al lanzar un programa ELF (ver Anexo I) y también el proceso explicitado en el Cap 4, junto con la impresión de mensajes por consola al usar el comando `log(LOG_INFO, «mensaje>")`; se ha logrado identificar la mejor posición en la cuál

colocar el código de detección, lectura y extracción desde la sección `.metadata` del ejecutable. Dicha posición se encuentra dentro del código de *kernel*: `imgact_elf.c` en el directorio `sys/kern`, específicamente dentro de la función:

`static int __CONCAT(exec_, __elfN(imgact))(struct image_params *imgp)`.

En dicha posición del proceso de carga, el archivo ELF se encuentra abierto y con *locks* activos para evitar condiciones de carrera sobre el mismo.

Posteriormente, se ha portado el código del módulo de *kernel* obtenido en la iteración anterior hacia el código dentro del *kernel*.

Se ha agregado los siguientes archivos:

- `metadata_elf_reader.c` [37]: código fuente dentro del directorio `sys/kern`.
  Contiene la función:
  `getMetadataSectionPayload(const Elf_Ehdr *hdr, struct image_params *imgp)`
  Dicha función es usada para iterar sobre el ELF, buscar la sección `.metadata` y extraer, guardar e imprimir por consola los metadatos insertados.
- `metadata_elf_reader.h` [38]: archivo de cabecera alojado en el directorio `sys/sys`. Incluye macro con el nombre de la sección a buscar y la definición de la función `getMetadataSectionPayload()`.

Finalmente, se han realizado cambios dentro de la función __CONCAT(exec_, __elfN(imgact)) de `imgact_elf.c` [39] para llamar a la función `getMetadataSectionPayload()` recientemente agregada, así como para imprimir el *returned_flag*, resultado de encontrar o no la sección `.metadata` en la función anterior.

La implementación de esta iteración se encuentra bajo las etiquetas s3, s4, s5, s6, s7 y s8. Ver [36].

5.2.4. Análisis de resultados

Ventajas:

- Solución como parte de un parche de *kernel*, en lugar de módulo de *kernel* que necesita de un disparador para cargarse.
- Solución integrada al resto de funciones de *kernel*.
- Aprovechamiento de los recursos abiertos (ejecutable abierto) durante el proceso de carga de ejecutable.
- Modularización dentro del código del *kernel* al usar archivo fuente separado.

Desventajas:

- La búsqueda de la sección `.metadata` se realiza para todo ejecutable ELF a cargar en el sistema.

- Suma de procesamiento al implicar iteración sobre secciones del ELF al buscar la sección `.metadata`.
- Cambios en los archivos incorporados (`metadata_elf_reader.c` y `*.h`), así como en el código fuente (`imgact_elf.c`) que llama al código incorporado; implica construir (*build*) e instalar una nueva versión de *kernel*. Mayor riesgo a generar *kernel panics*.

5.2.5. Próximos pasos

El próximo paso está en definir como vincular los metadatos extraídos del ejecutable ELF con el resto de la *kernel* para futuros casos de uso en cualquier funcionalidad de la *kernel* que utilice el protocolo de comunicación espacio de usuario - espacio de *kernel* implementado en este trabajo.

5.3. Tercera iteración

5.3.1. Objetivos a alcanzar

El objeto de la tercera iteración ha sido encontrar una solución para mantener los datos decodificados y extraídos de la sección `.metadata` del ejecutable ELF en espacio de *kernel* para su posterior uso.

5.3.2. Propuesta

Tal como se ha expuesto en el objetivo principal del presente trabajo, los metadatos insertados en el ejecutable serán usados por el *kernel* para diferentes aplicaciones dentro del mismo. Para ello, es necesario que, luego de realizar la búsqueda, decodificación y extracción de los metadatos de la sección `.metadata`, estos estén disponibles para el uso por parte del *kernel* en cualquier momento deseado (diseño pensado de esta forma hasta esta iteración). Por lo tanto, el resultado de la decodificación deberá ser guardado en espacio de *kernel*, asociado al programa del ejecutable.

5.3.3. Implementación

La implementación de la propuesta ha sido modificar la estructura `proc`, declarada en el archivo cabecera `proc.h` en `sys/sys`. Dicha estructura representa la unidad de un proceso del sistema. El código modificado puede verse en [40]. Se han agregado dos campos a la estructura:

- `char p_metadata[MAXHOSTNAMELEN + 1]`: *array* de caracteres de tamaño 257 (256 + 1) *bytes*, con el objetivo de guardar allí los metadatos decodificados y extraídos.
- `int p_metadata_section_flag`: bandera puesta en alto en caso de que el proceso contenga metadatos disponibles, resultante de buscar la sección y encontrarla exitosamente.;

```
char p_metadata[MAXHOSTNAMELEN + 1];  /* Metadata inserted for payload retrieve
    ↪ */
int p_metadata_section_flag;    /* Flag of metadata section presence in
    ↪ executable */
```

Código 5.2: Campos agregados a la estructura proc en proc.h [40].

La bandera `p_metadata_section_flag` es seteada en `imgact_elf.c` con el resultado de la bandera modificada por la función `getMetadataSectionPayload()`.

El arreglo de caracteres dentro de la estructura `proc` no ha sido modificado. Se ha dejado para prueba futura ya que, durante la implementación, no se ha definido si este dato debería estar disponible durante toda la vida del proceso o setear las variables correspondientes en las distintas partes del *kernel* una única vez, al llevar a cabo la decodificación.

En el Código 5.2 se pueden ver los cambios.

La implementación de esta iteración puede verse en los *tags* del s10 al s12. Listado en [36].

5.3.4. Análisis de resultados

Ventajas:

- El flag `p_metadata_section_flag` permite conocer con solo acceder a la estructura del proceso si el mismo contiene o no metadata asociada. Evitando así realizar la búsqueda de sección y decodificación nuevamente.
- Guardar la metadata decodificada en `p_metadata`, permite que la metadata extraída esté disponible durante toda la duración del proceso, dentro de la estructura del mismo proceso.

Desventajas:

- El *payload* no puede superar el tamaño provisto por **MAXHOSTNAMELEN + 1** (257 *bytes*), lo que limita su tamaño y funcionalidad.
- La reserva de 257 *bytes* dentro de la estructura `proc` implica incrementar el tamaño de todos los procesos del sistema, aún cuando no contengan metadatos.

5.3.5. Próximos pasos

Los próximos pasos que han sido tomados al finalizar la tercera iteración fueron los de habilitar la inserción de múltiples *payloads* dentro de la sección en cuestión. Hasta esta iteración, se había manejado solamente un único dato insertado en forma de *string*.

```
1 char str[MAXHOSTNAMELEN] __attribute__((section(".metadata"))) = "Init";
```

Código 5.3: Inserción de datos en sección `.metadata` establecido en código de usuario de forma directa.

5.4. Cuarta iteración

5.4.1. Objetivos a alcanzar

El objetivo de la cuarta iteración ha sido facilitar la inserción del *string* enviado como metadato por parte del programador.

5.4.2. Propuesta

La inserción de los datos por parte del programador en espacio de usuario debe estar siempre guiado por ser de fácil uso. La implementación hasta la cuarta iteración ha sido una macro con valor seteado por terminal en tiempo de compilación (ver Código 5.1) y un único valor de metadato.

Para esta iteración se ha planteado modificar la forma en la cual se inserta la metadata para habilitar la inserción de múltiples metadatos.

5.4.3. Implementación

El cambio en la forma de inserción ha sido abandonar el modelo de macro con valor pasado por terminal hacia definición propia dentro del código de usuario. El programador, ahora, debería agregar la línea de inserción dentro del código de su programa y establecer allí el valor. Ver Código 5.3

Una segunda prueba ha sido intentar insertar un *string* utilizando la función:

`insertPayload(char* payload)` [41].

Dicha función recibe un puntero a *string* seteado en el `main` [42] del programa. La función intenta modificar la variable `str` (ver Código 5.3), definida de forma global anteriormente.

Esta prueba ha resultado fallida. El programa definido en `main` y la llamada a la función nombrada se realiza en tiempo de ejecución. Mientras que la definición de `str` como una variable global y su inserción en el ejecutable se realiza en tiempo de compilación, compilación que finaliza con la generación del ejecutable ELF con los datos insertados. Por lo tanto, al leer el ejecutable, `str` tiene el valor definido en tiempo de compilación. El valor modificado sólo se visualiza al ejecutar el programa y se mantiene durante la duración del tiempo de vida del programa, sin reflejarse en los datos estáticos dentro de la sección `.metadata` del ELF.

La implementación de esta iteración se encuentra en el *tag* s15. Listado en [36].

```
const char procStructPayload[MAXHOSTNAMELEN] = "[3:HIPRIO][4:A]";
```

Código 5.4: Línea que contiene múltiples payloads de forma codificada.

5.4.4. Análisis de resultados

Ventajas:

- Adopción de la inserción de datos con valores definidos dentro del código de usuario, en lugar de valor enviado desde terminal.

Desventajas:

- La inserción de datos debe realizarse con datos estáticos, es decir, ya definidos en tiempo de compilación.

5.4.5. Próximos pasos

Una vez definido que los datos a insertar deben estar explicitados en tiempo de compilación, la próxima iteración debía resolver cómo insertar múltiples *payloads* dentro de la sección `.metadata`.

5.5. Quinta iteración

5.5.1. Objetivos a alcanzar

El objetivo de la quinta iteración es la de enviar distintos *payloads* que representen distintas unidades de información en la sección `.metadata`.

5.5.2. Propuesta

La propuesta es permitir la inserción de múltiples *payloads* dentro de la sección de interés del ejecutable ELF. Hasta la iteración anterior, se ha insertado solamente un único dato en forma de *string*. Ésto ha sido claramente una limitación fáctica de cualquier implementación.

Al habilitar múltiples *payloads*, se debe tener en cuenta que distintos datos pueden estar destinados a distintas aplicaciones dentro del *kernel*. Por lo tanto, se debe estructurar el *payload* o la sección `.metadata` de tal manera que sea fácil la inserción, decodificación y vinculación entre dato - funcionalidad.

5.5.3. Implementación

La implementación del objetivo está basada en que los metadatos puedan insertarse múltiples datos en una mísma línea; o múltiples líneas con uno o múltiples metadatos.

Una línea de ejemplo es la que muestra el Código 5.4.

Los distintos *payloads* se enviarán de forma codificada y ordenada usando un envoltorio o *wrapper* alrededor los datos. Los elementos de la línea son:

- Corchetes ([]): son los delimitadores de cada unidad de metadatos.
- Dos puntos (:): separan el número de función de *kernel* y el dato efectivo.
- Número de función: indica la función de *kernel* a la que está destinado el dato enviado en el ELF. En el caso mencionado, Código 5.4, los números de función son `3` y `4`.
- Dato: dato que será enviado a la función de *kernel* correspondiente. En el ejemplo Código 5.4, los datos son `HIPRIO` y `A`.

El número de función está relacionado a una funcionalidad específica de *kernel*. Este número de función está definido en el *data type*:

`enum metadata_function_num` .

Dicho tipo de datos está especificado en el archivo cabecera `metadata_elf.h` [43]. En el caso del Código 5.4, la función `3` está asociada al *scheduler* (`SCH` en [43])

Al emplear esta forma de enviar los datos, para poder decodificarlo, se debe realizar un parseo extra para obtener la cantidad de *payloads* distintos enviados, extraer y vincular los datos con su función en *kernel*. Ante ello, se ha desarrollado un programa en espacio de usuario que permita realizar esta manipulación del *string* enviado en la sección `.metadata`.

La búsqueda inicia en el *main* [44], pasando el número de función (`function_num`) a la función `char* getPayloadPerFunction(int num_function, int* return_flag)`, declarada en [43]. La operación de la última función nombrada implica buscar el número de función solicitado dentro de la línea que contiene múltiples *payloads* y traer el dato asociado, en caso de encontrarlo. Caso contrario, se setea la bandera `return_flag` con su valor correspondiente de error.

Una vez probado que es posible obtener el dato correspondiente al número de función solicitado, se ha decidido emprender la implementación de la decodificación en *kernel*.

Sin embargo, al llevar a cabo la misma, el manejo de *strings* se ha complicado en espacio privilegiado. La funciones disponibles para manipular *strings* es muy limitada y la complejidad y precisión que se debe tener al reservar memoria en dicho espacio, ha generado que se desista de esta forma de organización de los datos.

La iteración actual responde a los *tags* s16, s17 y s18 de la lista de etiquetas [36].

5.5.4. Análisis de resultados

Ventajas:

- Puntapié inicial para múltiples datos dentro de *payloads*.

Desventajas:

- Codificación de *payloads* tan estricta implica *parseo* sobre toda la cadena de caracteres. Detectar los corchetes y dos puntos, separar el *string* y extraer los datos para todas las unidades de datos.
- En espacio de *kernel*, dado el set reducido de funciones y las buenas prácticas de reserva de memoria en dicho espacio, se hace difícil implementar dicho modelo de codificación.

5.5.5. Próximos pasos

La decisión al finalizar la quinta iteración fue la de encontrar una nueva forma de codificación u organización de las unidades de datos dentro de la sección `.metadata`, que permita de una forma simple, ordenada y fácil decodificación; insertar múltiples *payloads* en el ejecutable.

5.6. Sexta iteración

5.6.1. Objetivos a alcanzar

El objetivo de esta iteración ha sido encontrar una forma de organizar e insertar múltiples *payloads* en la sección `.metadata`, que sea de mayor facilidad al insertar y decodificar los mismos.

5.6.2. Propuesta

Dado el resultado de la exploración en la iteración anterior, se ha decidido evitar una codificación muy estricta, tal como la anterior que incluía un patrón específico de caracteres delimitadores. Se buscó en esta iteración utilizar los tipos de datos definidos por usuario que permite generar el lenguaje C.

Sumado a esto, se planteó una mejora en el método de inserción, buscando lograr una mayor transparencia en el uso de la herramienta para el programador.

5.6.3. Implementación

La implementación de una mejor organización ha resultado en usar `structs`, estructuras del lenguaje C. Éstas permiten definir una unidad que responda a las necesidades del programador, en nuestro caso, una unidad de *payload*.

La estructura `Payload`, que representará una unidad de metadatos, ha sido definida en el archivo cabecera `section_insertion.h` [45].

La definición de la estructura se observa en el Código 5.5.

A su vez, cumpliendo con la meta de mejorar la transparencia en uso para el programador, se ha definido la `macro INSERT_METADATA`, que reemplaza la unión de las palabras `attribute` y `section`

```c
#ifndef ELF_METADATA_INSERTION_H
#define ELF_METADATA_INSERTION_H

#include <stdio.h>

typedef struct {
    int num_a;
    int num_b;
    char caracter;
} Payload;

// Macro to insert static variable in .metadata section
#define INSERT_METADATA __attribute__((__used__, __section__(".metadata")))

#endif
```

Código 5.5: Header con struct Payload y nueva macro de inserción definidas [45].

```c
#include "section_insertion.h"

////////////////// STRUCT INIT AND INSERTION USING MACRO FROM LIBRARY
    ↪ section_insertion.h ///////////////

static Payload foo[] INSERT_METADATA = {
    4, 8, 'c'
};
```

Código 5.6: Inserción de metadatos usando estructuras Payload. [46].

por dicho nombre; favoreciendo así la lectura y escritura del código para el usuario de la herramienta. Dicha macro también se observa en el Código 5.5.

De tal forma, la inserción de variables se realiza al declarar una estructura *Payload* y usando la macro `INSERT_METADATA`, antes de definir los valores que tomarán los campos de la estructura declarada.

Cabe recordar que toda variable a insertar en el ejecutable ELF debe ser una variable global, y por lo tanto, estar completamente definida (sus valores deben ser asignados) en tiempo de compilación.

Un ejemplo de inserción se observa en el programa de espacio de usuario [46] expuesto en el Código 5.6

La inserción y lectura de los campos han sido probados correctamente en el programa [46].

El paso siguiente ha sido comprobar en espacio de *kernel*, la decodificación efectiva del nuevo modelo de organización de los metadatos.

En esta iteración, se ha decidido para una mejor organización, construcción e instalación del espacio de kernel; realizar un *fork* al repositorio completo del *kernel* de FreeBSD 11; e implementar los cambios dentro del mismo, como un parche listo para distribuir.

Se obtiene así el repositorio `metadataFreeBSD` [47].

Como se ha mencionado anteriormente, los archivos modificados en el *kernel*, son:

```
1) ** __CONCAT(exec_, __elfN(imgact)) ** has been called
1) // getMetadataSectionPayload() // LectorELF // -- Seccion .metadata encontrada
   ↪  en offset 0xb80 y de tamanio 24 bytes--
4) // decodeMetadataSection() // LectorELF // decod_payload //// num a: 4 - num b
   ↪ : 8 - caracter: c - int size: 4
```

Código 5.7: Extracto de log de kernel con resultados de decodificación de payload en forma de estructuras de C. [51].

- `imgact_elf.c` en el directorio `sys/kern/`. Archivo modificado para colocar punto de entrada a funciones de búsqueda, decodificación y extracción de metadata en ejecutables ELF.
- `metadata_elf_reader.c`, agregado en `sys/kern/`.
- `metadata_elf_reader.h`, archivo cabecera agregado en `sys/sys/`.

La decodificación de los datos insertados se realiza con la función:

```
void decodeMetadataSection(struct thread *td)
```

.

Esta función está declarada en `metadata_elf_reader.h` [48] y está definida en `metadata_elf_reader.c` [49]. Dicha función es llamada en `imgact_elf.c` [50], unicamente si la búsqueda de la sección `.metadata` ha sido exitosa.

La iteración ha terminado exitosamente con los resultados impresos en el *log* de *kernel* [51].

Las líneas que muestran dicho resultado se observan en el Código 5.7

El trabajo realizado en esta iteración ha sido expuesto bajo las etiquetas s19, s20, s21, s23 y s24 en la lista [36] de *tags* del repositorio principal [52] y bajo las etiquetas s22 y s25 de la lista [53] de etiquetas del repositorio de trabajo de FreeBSD 11 [47].

5.6.4. Análisis de resultados

Ventajas:

- *Padding* de datos realizado automáticamente por el compilador y uniformidad de la unidad de *payload* al usar estructuras de C.
- Evitar el procesamiento complejo ante manejo de strings al ordenar metadatos en forma de *structs*.
- Acceso de los campos de la estructura se realiza directamente con sólo saber el nombre del campo.

Desventajas:

- No es posible variar el orden o tipo de datos a enviar como metadatos ya que la estructura es rígida.
- El usuario debe ingresar a mano el valor de los datos en el código de su programa. La estructura debe estar completa al momento de compilar el programa para poder ser insertada en el ejecutable.

5.6.5. Próximos pasos

Con los resultados positivos de esta iteración, se ha decidido intentar en la próxima iteración el envío de múltiples unidades de metadatos utilizando la estructura *Payload* desarrollada en esta iteración.

5.7. Séptima iteración

5.7.1. Objetivos a alcanzar

El objetivo de la séptima iteración ha sido la de insertar múltiples unidades de *payload* en forma de estructuras de C, dentro de la sección `.metadata` de archivos ELF ejecutables. Sumado a ello, proveer las herramientas de *encoder* y *decoder* de los metadatos.

5.7.2. Propuesta

Los resultados de la iteración anterior han permitido definir que la organización de los metadatos dentro del ELF se realice insertando estructuras de C, al estilo de la estructura `Payload`, previamente mostrada en el Código 5.5.

La propuesta en esta iteración es organizar la sección *.metadata* de una forma análoga a la de los archivos de formato ELF.

5.7.3. Implementación

La implementación de la nueva organización de la sección `.metadata` está inspirada en la organización interna de un archivo ELF.

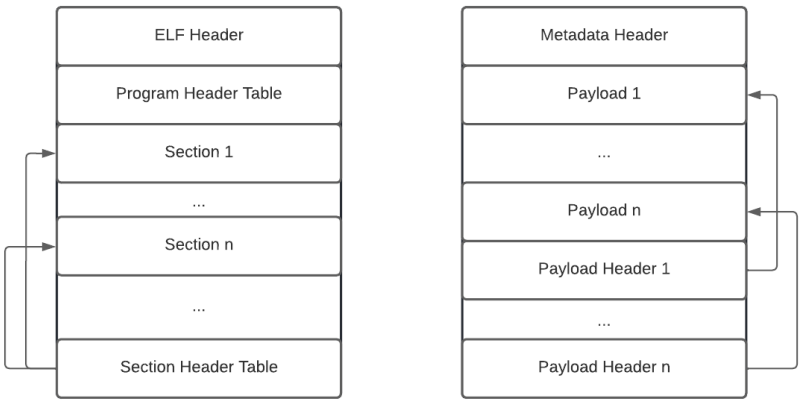


Figura 5.1: Comparación entre el formato de un archivo ELF y la sección `.metadata` planteada. Fuente: Propia

En la Fig. 5.1 puede verse la comparación entre ambas.

Para implementar la propuesta, se procedió a crear el archivo `metadata_payloads.h` [54] dentro del código del *kernel*, en el directorio `sys/sys`. Dicho archivo cabecera define tres estructuras de C que son la base de la estructura de la sección `.metadata`. Las estructuras son:

- `Metadata_Hdr`: Estructura ubicada al inicio de la sección `.metadata` con dos campos:
  - `m_number_payloads`: indica el número de unidades de *payload* distintas. Debe ser igual al número de `Payload_Hdr` definidas en la sección.
  - `ph_size`: Tamaño de cada entrada en `Payload_Hdr`. Todas las entradas de `Payload_Hdr` tienen el mismo tamaño.
- `Payload_Hdr`: Estructuras ubicadas al final de la sección `.metadata`. Aparecen en el orden en que han sido declaradas en el código. Contienen información general sobre la unidad de *payload* que tienen vinculada. Existe uno por cada unidad de datos distinta. Contiene dos campos:
  - `p_function_number`: número de función relacionada a una funcionalidad dentro del *kernel*, asociado al `Payload` vinculado al dato que contiene el *payload* específico.
  - `p_size`: Tamaño del *payload*.
- `Payload`: estructura que contiene efectivamente el dato que se quiere enviar al espacio de *kernel*. Pueden haber múltiples instancias de estructura `Payload` distintas con campos distintos por cada tipo de *payload* diferente. Las estructuras `Payload` se ubicadan al final de `Metadata_Hdr` y antes que el primer `Payload_Hdr`, de acuerdo al orden en el que se han declarado en el código. Cada `Payload` debe tener su `Payload_Hdr` asociada.

.,

El Código 5.8 [54] muestra la definición de dichas estructuras.

Cabe aclarar que también se ha probado incorporar otras estructuras tales como `PayloadHdr_List`, estructura pensada para funcionar como base para una lista de estructuras `Payload_Hdr`. No hizo falta emplear una lista. Solamente definir las estructuras `Payload` con su respectiva `Payload_Hdr` correctamente llena y en el mismo orden de aparición. Dentro de las diferentes estructuras, se ha probado colocar campos que contengan el offset a la estructura `Payload` y la dirección de la misma dentro de su respectiva estructura `Payload_Hdr`, con el objetivo de hacer más fácil la decodificación y localización de cada `Payload`. Sin embargo, estos campos no eran certeros ya que respondían a posiciones en espacio de usuario que no coincidían con las mismas posiciones teniendo en cuenta localización de las estructuras dentro del archivo ELF, qué es lo que buscamos. Finalmente, ambas ideas nombradas anteriormente han sido descartadas.

El *encoder* [55], programa de espacio de usuario, realiza la declaración e inserción de las distintas estructuras y datos de la manera que se expone en el Código 5.9 [55].

En este caso, se han definido dos tipos de estructuras `Payload` distintas: `Payload_A`, asociada a la función número 1, y `Payload_B`, con función número 2. Cada una de ellas con sus diferentes campos, representando distintos tipos de información.

Finalmente, la decodificación se realiza en espacio de *kernel* mediante la función `decodeMetadataSection` de `metadata_elf_reader.c` [56]. Dicha función utiliza los campos de

```c
#ifndef METADATA_PAYLOADS_H_
#define METADATA_PAYLOADS_H_

typedef struct {
    int num_a;
    int num_b;
    char caracter;
    size_t int_size;
} Payload_A;

typedef struct {
    int num_a;
    char char_B1;
    char char_B2;
    size_t char_size;
} Payload_B;

typedef struct {
    int p_function_number;          // Function number associated to the payload
    size_t p_size;                  // Size of the payload
} Payload_Hdr;


typedef struct {
    int m_number_payloads;          // Total number of different payloads in the
    ↪ section (num of entries in the payload header table)
    size_t ph_size;                 // Size of an entry in the payload header
    ↪ table
} Metadata_Hdr;

#endif /* METADATA_PAYLOADS_H_ */
```

Código 5.8: Estructuras que conforman la estructura de la sección .metadata

```
#define NUM_PAYLOADS_A  2               // Total amount of payloads to add to
    ↪ metadata section
#define NUM_PAYLOADS_B  1
#define NUM_PAYLOADS_TOTAL 3
                                                    // First, declare the
    ↪ metadata section header
static Metadata_Hdr metadata_header[] INSERT_METADATA = {
    {NUM_PAYLOADS_TOTAL, sizeof(Payload_Hdr)}
};
                                                  // Then, declare the array of
    ↪  the structs to add to metadata section
static Payload_A payloadsA_st[NUM_PAYLOADS_A] INSERT_METADATA = {
    {4, 8, 'c', sizeof(int)},
    {16, 24, 'd', sizeof(int)}
};

static Payload_B payloadsB_st INSERT_METADATA = { 4, 'a', 'c', sizeof(char) };

////////// PAYLOAD HEADER OF EACH PAYLOAD     // Finally, end the metadata
    ↪ section with the payload headers table

static Payload_Hdr payload_headers[] INSERT_METADATA = {
    {1, sizeof(payloadsA_st[0])},
    {1, sizeof(payloadsA_st[1])},
    {2, sizeof(payloadsB_st)}
};
```

Código 5.9: Programa de inserción de metadatos de forma estructurada simil a organización de ELF. [55].

dirección de inicio (`p_metadata_addr`) y tamaño (`p_metadata_size` de la sección `.metadata`, campos presentes en la estructura `proc` del proceso, para realizar cálculos de *offset* y junto a la cantidad de *payloads* (`m_number_payloads`) definida en Metadata_Hdr, extraer cada campo de cada *payload* insertado en la sección.

El algoritmo de búsqueda de cada estructura es análogo al de un archivo ELF. Se obtiene realizando los siguientes pasos:

- Al principio de la sección `.metadata` se encuentra `Metadata_Hdr`, con lo cual se obtiene la cantidad de `Payload_Hdr` con sus tamaños, que se encuentran al final de la sección.
- Conociendo donde termina la sección y la cantidad de `Payload_Hdr`, se obtiene el tamaño de cada `Payload` y su función asociada.
- Por último, con esta información se busca cada una de las unidades de `Payload`. Estas están ubicados al final de la `Metadata_Hdr`.

La inserción y decodificación se realiza de forma exitosa. El resultado se observa mediante los registros de *kernel* [57] en el Código 5.10.

Las etiquetas s26, s27, s28, s29 y s35 en la lista [36] de *tags* del repositorio principal [52] y las etiquetas s30, s31, s32, s33 y s34 de la lista [53] de etiquetas del repositorio de trabajo de FreeBSD 11 [47]; representan el trabajo realizado en esta iteración.

```
4) ** decodeMetadataSection() ** has been called
4) // decodeMetadataSection() // LectorELF // Metadata_Hdr =
    ↪ metadata_header_decod //// m_number_payloads: 3 - ph_size: 16
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [0] //// p_function_number: 1 - ph_size: 24
4) // decodeMetadataSection() // Payload_A = payloadA_decod //// num a: 4 - num b
    ↪ : 8 - caracter: c - int_size: 4
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [1] //// p_function_number: 1 - ph_size: 24
4) // decodeMetadataSection() // Payload_A = payloadA_decod //// num a: 16 - num
    ↪ b: 24 - caracter: d - int_size: 4
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [2] //// p_function_number: 2 - ph_size: 16
4) // decodeMetadataSection() // Payload_B = payloadB_decod //// num a: 4 -
    ↪ char_B1: a - char_B2: c - char_size: 1
4) ** decodeMetadataSection() ** EXIT
```

Código 5.10: Salida de logs de kernel mostrando decodificación correcta de Payload A y Payload B. [57].

5.7.4. Análisis de resultados

Ventajas:

- Inserción de múltiples unidades de *payload*, así como de diferentes tipos de *payload*.
- Localización de cada estructura definida se realiza de manera más fácil y directa utilizando dirección inicial de la sección *.metadata*, cantidad de *payloads* y tamaño de cada elemento.

Desventajas:

- La inserción de cada instancia de `Payload_Hdr` debe responder al mismo orden de inserción de su `Payload` correspondiente.
- La inserción debe realizarse en orden estricto que permita seguir la organización definida.
- La estructuración de la sección *metadata* aporta al aumento de tamaño del archivo ejecutable ELF.
- Se incrementa la complejidad y el código que debe escribir el programador en su programa para utilizar la herramienta.

5.7.5. Próximos pasos

Con la sección `.metdata` estructurada, el próximo paso decidido ha sido el automatizar el proceso de inserción y asignación de valores con la ayuda de un *plugin*.

5.8. Octava iteración

5.8.1. Objetivos a alcanzar

La octava iteración ha tenido como objetivo el mejorar el manejo de la decodificación ante los distintos tipos de estructuras `Payload` y números de funciones.

```
typedef void (*payload_func)();
payload_func payload_functions[PAYLOADS_TOTAL+1] = {0, &payload_A_func, &
    ↪ payload_B_func};
```

Código 5.11: Arreglo de funciones de decodificación de payloads. [58].

5.8.2. Propuesta

Hasta la iteración 7, se han manejado dos tipos distintos de metadatos: `Payload_A` y `Payload_B`. Es necesario que la herramienta que se desarrolla esté abierta al soporte de múltiples tipo de metadatos, que se reflejan en múltiples números de funciones. Cada funcionalidad a la que puede estar destinado un metadato, puede necesitar una forma específica de decodificación, extracción y conexión con el resto del *kernel* para lograr su cometido. Por lo tanto, es razonable pensar que por cada nuevo tipo de *payload*, se agregará una nueva función de decodificación al código desarrollado.

Ésto implica que las funciones específicas de decodificación deberán ser modularizadas. A su vez, el procesamiento y lanzamiento de dichas funciones debería ser simple y evitar mayores iteraciones sobre una lista de funciones permitidas; recordando siempre que las acciones se ejecutan en espacio de *kernel*, donde la penalidad de tiempo es relevante.

5.8.3. Implementación

Para facilitar la incorporación de futuras funciones de decodificación sin implicar un procesamiento exhaustivo por parte del sistema en espacio de *kernel*, se ha implementado un *arreglo* de funciones dentro de `metadata_elf_reader.c` [58], que contenga todas las funciones de decodificación necesarias para los distintos tipos de *payload*.

El arreglo de funciones `payload_functions` se observa en el Código 5.11.

Dichas funciones específicas que atienden cada tipo de *payload* están declaradas en `metadata_elf_reader.h` [59] y definidas en `metadata_elf_reader.c` [58].

Aquel desarrollador que quiera sumar tipos de estructura *Payload* y, por ende, sumar funciones de decodificación, debería modificar el valor de `PAYLOADS_TOTAL` (definido en [59]), definir su correspondiente *handler* y agregarlo al arreglo de funciones; además de armar la estructura que represente al nuevo tipo de *payload*.

Entonces, el procedimiento de llamada a una función específica de decodificación, por ejemplo la función `payload_B_func()`, comienza al llamar a la función `decodeMetadataSection()` [58]. Allí se lee toda la sección `.metadata` extraída, y por cada estructura `Payload` detectada, solamente con su número de función (campo `p_function_number` dentro del correspondiente `Payload_Hdr`), se lanza la función específica que atiende dicho tipo de *payload*.

La función que atiende metadatos de tipo `Payload_B`, `payload_B_func()`, se observa en el

```
void payload_B_func(Payload_B* payloadB_decod, void* payload_addr){

    memcpy(payloadB_decod, payload_addr, sizeof(Payload_B));

    ////////////
    log(LOG_INFO, "\t\t\t 4) // decodeMetadataSection() // payload_B_func() //
    ↪ Payload_B = payloadB_decod //// num a: %d - char_B1: %c - char_B2: %c -
    ↪ char_size: %lu\n",
        payloadB_decod->num_a, payloadB_decod->char_B1, payloadB_decod->char_B2,
    ↪ payloadB_decod->char_size);
    ////////////
}
```

Código 5.12: Función que decodifica e imprime el contenido de datos de tipo `Payload_B`. [58].

```
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [0] //// p_function_number: 1 - ph_size: 24
4) // decodeMetadataSection() // payload_A_func() // Payload_A = payloadA_decod
    ↪ //// num a: 4 - num b: 8 - caracter: c - int_size: 4
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [1] //// p_function_number: 1 - ph_size: 24
4) // decodeMetadataSection() // payload_A_func() // Payload_A = payloadA_decod
    ↪ //// num a: 16 - num b: 24 - caracter: d - int_size: 4
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [2] //// p_function_number: 2 - ph_size: 16
4) // decodeMetadataSection() // payload_B_func() // Payload_B = payloadB_decod
    ↪ //// num a: 4 - char_B1: a - char_B2: c - char_size: 1
```

Código 5.13: Salida de log de kernel con estructuras `Payload_A` y `Payload_B` decodificadas. [60].

Código 5.12, y su resultado se observa en el Código 5.13.

Las etiquetas s36 y s37 en la lista de *tags* [53] del repositorio de trabajo de FreeBSD 11 [47] reflejan el trabajo realizado en la iteración octava.

5.8.4. Análisis de resultados

Ventajas:

- Se estandariza el lanzamiento de funciones de decodificación específica de cada tipo de *payload*.

Desventajas:

- Al agregar nuevas funciones de decodificación, es necesario agregarlas al arreglo de funciones.

5.8.5. Próximos pasos

La iteración octava ha permitido cerrar, hasta este momento, la parte de decodificación en *kernel*. El objetivo siguiente será facilitar la inserción de datos desde el punto de vista del usuario de la herramienta.

5.9. Novena iteración

5.9.1. Objetivos a alcanzar

El objetivo de la novena iteración es la de facilitar la carga de los datos que se desean insertar en el ejecutable.

5.9.2. Propuesta

Hasta la iteración anterior, el programador que utiliza el protocolo desarrollado tenía que declarar las variables a insertar y completar los datos a insertar, por su cuenta en el código del programa a compilar. Desde el punto de vista del usuario, esto implica insertar varias líneas de código que representan las distintas estructuras que conforman la sección `.metadata`. La mayoría de estas estructuras (estructuras `Metadata_Hdr` y `Payload_Hdr`) están destinadas a soporte de la estructura de la sección, mientras que los datos definidos por el usuario son solamente las correspondientes a estructuras `Payload`.

El usuario no conoce y tampoco debe por qué conocer como llenar las estructuras de soporte estructural.

Es por ello que se ha decidido en esta iteración buscar un mecanismo para hacer más transparente la carga de datos a insertar, sin tener que agregar cantidad de líneas en el programa de usuario.

5.9.3. Implementación

La propuesta será llevada a cabo mediante el desarrollo de un *plugin* de la *suite* de compilación *GCC*. El *plugin* provee una herramienta para modificar el accionar del compilador durante el proceso de compilación. Recordamos que, para que las variables a insertar puedan ser reflejadas en el archivo ejecutable ELF, resultante de la compilación del código fuente, estas deben ser definidas en tiempo de compilación. Es decir, su valor debe estar asignado antes de iniciar la compilación. Con el *plugin*, podemos tomar una de las fases de compilación y hacer los cambios necesarios sobre el código fuente del programa de usuario para lograr nuestro cometido.

Se ha tomado como base un desarrollo de *plugin* de GCC que permitía insertar un valor determinado a una directiva `#define`.

El desarrollo de la versión inicial funcional del *plugin* se encuentra en el código `pluginsegundo.c`.

En lugar de definir el tipo de estructura `Payload` y sus valores en el código del programa, el usuario debería suministrar un archivo que contenga los datos anteriores. Cada línea del archivo indica una unidad de metadatos distinta. El primer elemento debe ser el tipo de Payload mientras que el resto

```
Payload_A/2/4/c/intsize
```

Código 5.14: Archivo de texto que contiene los datos a insertar. [61].

```
HOST_GCC=g++11
TARGET_GCC=gcc11
PLUGIN_SOURCE_FILES= pluginsegundo.c
GCCPLUGINS_DIR:= $(shell $(TARGET_GCC) -print-file-name=plugin)
CXXFLAGS+= -I/usr/local/lib/gcc11/gcc/x86_64-portbld-freebsd13.0/11.2.0/plugin/
    ↪ include -fPIC -fno-rtti -O2

plugin.so: $(PLUGIN_SOURCE_FILES)
    $(HOST_GCC) -shared $(CXXFLAGS) $(PLUGIN_SOURCE_FILES) -o $@

clean:
    rm -f prueba plugin.so

test:
    gcc -o prueba prueba.c -fplugin=./plugin.so -fplugin-arg-plugin-payloadsDec=
    ↪ declarations.txt
```

Código 5.15: Instrucciones de compilación del plugin y su uso en compilación con programa de usuario. [62].

deben ser los valores del campo de la estructura de dicho *payload*. Todos los elementos separados por barras `/`.

Ver Código 5.14.

El *plugin* es compilado en forma de librería compartida (`.so`). Luego, se lo incluye en la compilación del programa del usuario. En el comando de compilación, el *plugin* recibe el nombre del archivo anterior como argumento.

Los comandos de compilación se encuentran en el archivo `Makefile` [62] y se observan en el Código 5.15.

El *plugin parsea* dicho archivo y crea automáticamente las estructuras `Payload` correspondientes, así como las otras estructuras que conforman la organización de la sección `.metadata`.

El usuario solamente debe incluir en su código de programa:

- El archivo cabecera, *header*, que contiene la definición de las tres estructuras que se utilizan para la inserción. En este caso, llamado `section_insertion.h` [63].
- La variable global, sin tipo, de nombre `__PAYLOAD__`. Se utilizan doble guión bajo antes y después del nombre de la variable, en mayúsculas, para coincidir con las recomendaciones de nombrado de palabras reservadas en C.

Un ejemplo de código de programa de usuario con estos elementos se observa en el Código 5.16.

El *plugin* de GCC desarrollado toma acción al producirse el evento `PLUGIN_START_UNIT` [65], antes del procesamiento de toda *translation unit*. Una *translation unit* es definida en [66] como todo archivo fuente, junto con los archivos cabecera y el resto de archivos incluidos usando la directiva

```c
#include <stdio.h>
#include <string.h>
#include <stddef.h>
#include "section_insertion.h"

__PAYLOAD__;

int main(int argc, char *argv[]) {
    printf("\n******** codeinsertion finished ********\n");

    return 0;
}
```

Código 5.16: Código de usuario [64] que incluye la variable `__PAYLOAD__` a ser reemplazada por el plugin.

```c
#include <stdio.h>
#include <string.h>
#include <stddef.h>
#include "section_insertion.h"

static Metadata_Hdr metadata_header __attribute__((__used__, __section__(".
    ↪ metadata"))) = {1, sizeof(Payload_Hdr)};
static Payload_A payloads_st __attribute__((__used__, __section__(".metadata")))
    ↪ = {2,4,'c',4};
static Payload_Hdr payload_headers __attribute__((__used__, __section__(".
    ↪ metadata"))) = {1, sizeof(Payload_A)};

int main(int argc, char *argv[]) {
    printf("\n******** codeinsertion finished ********\n");

    return 0;
}
```

Código 5.17: Reemplazo de la variable __PAYLOAD__ por líneas metadata mediante el plugin en durante la compilación de código de usuario.

`#include`, que han pasado por el preprocesador. En pocas palabras, el *plugin* acciona sobre la salida del preprocesador de GCC.

Lo que hará el *plugin* será detectar la definición de la variable global `__PAYLOAD__`, y reemplazarla por una línea que contenga todas las estructuras necesarias para insertar los datos suministrados en el archivo auxiliar. Dicha línea es la que se obtuvo en las iteraciones anteriores, pero ahora, se inserta de forma automática al compilar el programa de usuario junto al *plugin*.

El Código 5.17 muestra el resultado del reemplazo de código llevado a cabo por el *plugin*.

La implementación actual solamente permite un único tipo de *payload*: `Payload_A` y admite solamente una única unidad de *payload*.

El trabajo realizado en esta iteración se encuentra bajo las etiquetas s38, s39, s40, s43, s45, s46 y s47 de la lista [36] de *tags* del repositorio principal [52].

5.9.4. Análisis de resultados

Ventajas:

- La inserción de los datos se realiza de forma transparente para el programador usuario de la herramienta.
- El usuario define los datos a insertar en un archivo externo al código de su programa.
- El usuario solamente debe insertar la variable `__PAYLOAD__` y el *header* de definición de estructuras metadata.
- El *plugin* se compila una única vez y puede usarse multiples veces al necesitarse junto a la compilación de nuevos programas.

Desventajas:

- Las líneas del archivo externo donde se definen los datos a insertar tiene un formato estricto para que, luego, el *plugin* pueda procesarlo y armar correctamente las líneas.

5.9.5. Próximos pasos

Debido a que la *suite* de compilación por defecto de FreeBSD es `CLANG/LLVM`, se buscará desarrollar la misma solución de *plugin* para dicha *suite*.

5.10. Décima iteración

Nota: La iteración actual ha sido resultado de encontrar caminos alternativos al desarrollo de un *plugin* de GCC, ya que, durante algunos momentos del desarrollo del *plugin* expuesto en la iteración anterior, Sección 5.9, no se había encontrado el mecanismo para poder hacer el reemplazo por las líneas de la sección `.metadata`. Finalmente, dado el resultado negativo de la solución aportada por los mecanismos evaluados en esta iteración, se ha decidido totalmente encarar la solución por el lado del *plugin*.

5.10.1. Objetivos a alcanzar

El objetivo de la décima iteración ha sido explorar y evaluar otros métodos (distintos al *plugin* de GCC), para hacer la inserción de datos en ejecutables ELF de una forma más amigable y transparente para el usuario.

5.10.2. Propuesta

Se ha propuesto en esta iteración encontrar métodos que permitan insertar los datos sin tener que modificar el accionar del compilador. Se ha pensado en algunas herramientas que permitan modificar el ejecutable después de ser éste compilado.

5.10.3. Implementación

La primera prueba ha sido el uso de la herramienta `objcopy` [67], que permite manipular archivos objetos.

La idea consistía en compilar un programa que contenga solamente las definiciones y valores de los *payloads* y demás estructuras de la sección `.metadata` a insertar, tal como se venía realizando hasta la iteración octava: dentro de un código fuente y llenado manual. Dicho programa fue llamado `macrosCommentCLANG` [68]. Por otro lado, obtener solamente el archivo objeto (extensión `.o`), en lugar del ejecutable final, del programa de usuario (`helloworld`). Este programa no hace referencia alguna a *headers* o estructuras metadata.

Una vez obtenidos ambos ejecutables, se utiliza `objcopy` para extraer la sección `metadata` desde `macrosCommentCLANG` y obtener el *dump* de la misma en binario.

Luego, se utiliza el *flag* `-add-section` de `objcopy` para tomar el *dump* anterior e insertarlo dentro del archivo objeto del programa de usuario, `helloworld.o`, como una sección del mismo.

Posteriormente, se *setean* los *flags* (ver *flags* en Título 3.2.1.2.1): `contents` (indica sección no vacía), `alloc` (indica que los datos forman parte de la memoria del proceso), `load` (expone que la sección formará parte de un segmento), `data` (no contiene código) de la sección insertada en `helloworld.o`.

Finalmente, con la sección insertada en el archivo objeto del programa del usuario, se termina la compilación y se obtiene el ejecutable ELF final, denominado `sectionadded`.

Este procedimiento se observa en las instrucciones de compilación del Código 5.18

El resultado obtenido en los *logs* del *kernel* con respecto a la decodificación muestra que la inserción se realizó correctamente. Ver Código 5.19

El trabajo de esta iteración está bajo las etiquetas s41, s42 y s44 de la lista [36] de *tags* del repositorio principal [52].

5.10.4. Análisis de resultados

Ventajas:

- El código de programa de usuario está separado del código en el que se definen las estructuras a insertar.

Desventajas:

- Se mantiene el llenado manual de las estructuras a insertar, que debe completar el usuario de la herramienta. Lo cual no favorece a la facilidad de uso la misma.
- Es un procedimiento que implica varios pasos de compilación, agregando mayor complejidad al proceso.

```
    TARGETCOMPILER= cc
    OBJ_DIR= obj
    BIN_DIR= bin

    USERPROG= helloworld
    USERPROGMAIN= $(USERPROG).c
    INSERT_BIN_DIR=OriginalMetadataInsert/bin
    INSERTMETPROG= macrosCommentCLANG
    INSERTSECTIONNAME= metadata
    DUMPINSERTSECTIONFILE=$(INSERTSECTIONNAME).data
    USERPROGSECTIONNAME= metadata
    SECTIONADDEDFILENAME=sectionadded

    .OBJDIR: ./     # Override the OBJDIR variable for recursive make to work in
    ↪ FreeBSD Make

    all:
        mkdir -p obj
        mkdir -p bin
        $(MAKE) -C OriginalMetadataInsert
        $(TARGETCOMPILER) -c $(USERPROGMAIN) -o $(OBJ_DIR)/$(USERPROG).o
        objcopy -O binary -j .$(INSERTSECTIONNAME) $(INSERT_BIN_DIR)/$(
    ↪ INSERTMETPROG) $(DUMPINSERTSECTIONFILE)
        objcopy --add-section .$(USERPROGSECTIONNAME)=$(DUMPINSERTSECTIONFILE) $(
    ↪ OBJ_DIR)/$(USERPROG).o
        objcopy --add-section .$(USERPROGSECTIONNAME)=$(DUMPINSERTSECTIONFILE) $(
    ↪ OBJ_DIR)/$(USERPROG).o
        objcopy --set-section-flags .$(USERPROGSECTIONNAME)=contents,alloc,load,
    ↪ data $(OBJ_DIR)/$(USERPROG).o
        $(TARGETCOMPILER) $(OBJ_DIR)/$(USERPROG).o -o $(BIN_DIR)/$(
    ↪ SECTIONADDEDFILENAME)
}
```

Código 5.18: Makefile con instrucciones para extraer e insertar sección utilizando `objcopy` [69].

```
1) ** sys_execve() ** has been called for: ./sectionadded
1) // getMetadataSectionPayload() // LectorELF // -- *** .metadata section data:
    ↪ ***
1) // __CONCAT(exec_, __elfN(imgact)) // LectorELF // -- *** METADATA: 0
    ↪ xfffff80013cc5900 -- PAYLOAD SIZE: 128 -- RETURNED FLAG: 1 ***
4) // decodeMetadataSection() // payload_A_func() // Payload_A = payloadA_decod
    ↪ //// num a: 4 - num b: 8 - caracter: c - int_size: 4
4) // decodeMetadataSection() // payload_A_func() // Payload_A = payloadA_decod
    ↪ //// num a: 16 - num b: 24 - caracter: d - int_size: 4
4) // decodeMetadataSection() // payload_B_func() // Payload_B = payloadB_decod
    ↪ //// num a: 4 - char_B1: a - char_B2: c - char_size: 1
```

Código 5.19: Resultado de decoficación de la sección .metadata insertada utilizando objcopy [70].

5.10.5. Próximos pasos

Dado que la implementación obtenida dentro de esta iteración no ha significado mejoras en la usabilidad del protocolo, se prefiere continuar con el desarrollo y uso del *plugin* en las distintas *suites* de compilación.

5.11. Onceava iteración

5.11.1. Objetivos a alcanzar

El objetivo de esta iteración ha sido la de explorar y desarrollar un *plugin* en la *suite* de compilación `CLANG/LLVM` para permitir la inserción "automática"de datos dentro de la sección `.metadata` de un archivo ejecutable ELF.

5.11.2. Propuesta

La propuesta de la onceava iteración ha sido la de replicar la implementación del *plugin* realizada en la iteración novena (ver Sección 5.9) pero para utilizar en el proceso de compilación llevado a cabo con la *suite* de compilación `CLANG/LLVM`. Ya que la nombrada *suite* de compilación es la usada por defecto en el sistema operativo FreeBSD, sistema que se utiliza como base para nuestro trabajo. De tal forma, se podría obtener una solución lista para usar sin necesidad de cambiar la *suite* de compilación.

5.11.3. Implementación

La implementación del *plugin* en `CLANG/LLVM` requiere que su desarrollo sea realizado en el lenguaje `C++`, en lugar de `C` como se venía trabajando hasta ahora. Esto sucede ya que el desarrollo interno de dicha *suite* de compilación está escrita en dicho lenguaje.

Por ello, se ha tenido que comprender cómo funciona el *plugin* para poder modificar las acciones del compilador para lograr nuestro cometido [71].

Después de una extensa revisión de documentación y diversos ejemplos llevados a cabo por la comunidad, se ha decidido que la salida del analizador semántico (ver Título 2.2.1.3), momento en el que se ha obtenido el AST (árbol de sintaxis abstracta, o *abstract syntax tree* en inglés) del código procesado, es el punto temporal en el que actuará el *plugin* a desarrollar. Ésto permite aprovechar la visualización de toda la estructura del código. Sumado a ello, los distintos ejemplos revisados presentan acciones en dicho lugar, antes de terminar con el *frontend*.

El desarrollo inicial se encuentra en el archivo `RenameMacro.cpp` [72]. La clase que realiza lo planteado es `RecursiveASTVisitor` [73] y tiene como función pasar por cada nodo del AST del

```
virtual bool VisitFunctionDecl(FunctionDecl *func) {
    string funcName = func->getNameInfo().getName().getAsString();
    if (funcName == "PAYLOAD") {
        SourceLocation location = func->getLocation();
        rewriter.ReplaceText(location.getLocWithOffset(-5) , funcName.length()+7,
    ↪  "static Metadata_Hdr metadata_header __attribute__((__used__, __section__
    ↪ (\".metadata\"))) = {1, sizeof(Payload_Hdr)}");
        printf("encontro func\n");
    }
```

Código 5.20: Función VisitFunctionDecl con líneas para encontrar la función de nombre PAYLOAD en el código usando plugin de CLANG. [72].

```
$ clang11 -Xclang -load -Xclang ./libRenameMacro.so -Xclang -plugin -Xclang
    ↪ PPSplugin -c ../src/test.c -I../src
```

Código 5.21: Comando de uso del plugin de CLANG en la compilación del programa de usuario `test.c` [75].

código compilado, recorriendo y encontrando los nodos del tipo solicitado, finalmente llamando a la función que ejecutará una acción específica.

En nuestro caso, la función `VisitFunctionDecl` es la función en la que nos permite encontrar los nodos que representan a instancias de clases `FunctionDecl`. Dichas instancias representan declaraciones o definiciones de funciones dentro del código.

El Código 5.20 muestra las líneas agregadas para encontrar el nodo que contiene la declaración de la función `PAYLOAD`, obtener la posición de dicha línea en el código y reemplazarla por una línea de prueba que simule insertar una estructura `Metadata_Hdr`.

La compilación del *plugin* se realiza utilizando un archivo `CMake` [74] Luego, al igual que en el caso del *plugin* de GCC, el uso del mismo se realiza al momento de compilar el programa de usuario.

El Código 5.21 expone el comando de compilación del programa de usuario `test.c` [75].

Finalmente, se observa en el Código 5.22 el programa de usuario original. Posterior al uso del *plugin*, se obtiene el código modificado tal como lo presenta el Código 5.23.

El trabajo realizado en esta iteración se refleja en las *tags* s48, s54, s55 y s56 de la lista [36] de *tags* del repositorio principal [52].

```
#include "section_insertion.h"
void PAYLOAD();

int main(int argc, char *argv[]) {
    return 0;
}
```

Código 5.22: Programa `test.c` con función PAYLOAD a remplazar. [75].

```
#include "section_insertion.h"
static Metadata_Hdr metadata_header __attribute__((__used__, __section__(".
    ↪ metadata"))) = {1, sizeof(Payload_Hdr)};
int main(int argc, char *argv[]) {
        return 0;
}
```

Código 5.23: Programa `test.c` con función reemplazada y líneas metadata agregadas.

5.11.4. Análisis de resultados

Ventajas:

- Herramienta para hacer más transparente la inserción de datos en archivos ejecutables ELF.
- La compilación del *plugin* se realiza una única vez y puede usarse en la compilación cualquier programa.

Desventajas:

- Es una versión inicial del *plugin*.
- El desarrollo de *plugins* tiene poco camino de exploración, por lo tanto, se ha complicado el camino hasta obtener la implementación final de esta iteración.
- Se busca la función `void PAYLOAD()` en lugar de la variable `__PAYLOAD__`, como se hace en la implementación del *plugin* de GCC (recordar revisando el Código 5.16).

5.11.5. Próximos pasos

Habiendo comprobado la factibilidad de obtener el desarrollo de un *plugin* para `CLANG/LLVM`, el próximo paso es ampliar el alcance del parche a versiones más nuevas de FreeBSD.

5.12. Doceava iteración

5.12.1. Objetivos a alcanzar

El objetivo de la presente iteración ha sido desarrollar el parche de *kernel* que permite buscar y decodificar la sección `.metadata` para versiones recientes de FreeBSD.

5.12.2. Propuesta

Hasta la iteración anterior se ha estado trabajando en la versión 11 de FreeBSD. Dicha versión ha sido lanzada en el año 2016 y su soporte fue pactado hasta 2017. Es por ello que se ha decidido implementar el trabajo realizado en este proyecto hacia las versiones más nuevas del sistema operativo.

```
// Check if VP is locked
int prev_locked_status = 0;
prev_locked_status = VOP_ISLOCKED(imgp->vp);  // 0 -> UNLOCKED, else: previously
    ↪ locked.
// If not locked, then lock to serialize the access to the file system
if(prev_locked_status == 0){ // If 0, then UNLOCKED
        vn_lock(imgp->vp, LK_SHARED | LK_RETRY);
}
```

Código 5.24: Chequeo de bloqueo por parte del hilo en `metadata_elf_reader.c` [76] [77]

Se ha planteado trabajar sobre las versiones 12.3 y 13 de FreeBSD. Ante ello, se debe tener en cuenta los distintos cambios que se han llevado a cabo, en nuestro caso, en el código del *kernel* para poder amoldar la implementación ya obtenida para versión 11 hacia las nuevas versiones planteadas. Algunas funciones pueden estar ahora deprecadas o distintas estructuras o propiedades haber sido modificadas.

5.12.3. Implementación

Al código de *kernel* de FreeBSD 12.3 y FreeBSD se han agregado los archivos:

- `metadata_elf_reader.c` [76] [77]
- `metadata_elf_reader.h` [78] [79]
- `metadata_payloads.h` [80] [81]

Y se han modificado los archivos:

- `proc.h` [82] [83]
- `imgact_elf.c` [84] [85]

En un principio, el agregado de dichos archivos ha generado *kernel panic* tanto en la versión 12.3 como 13. Después de llevar a cabo el *debug* de dichos eventos, se llegó a la conclusión de que, en la línea en la que se llama a las funciones *metadata* dentro de `imgact_elf.c`, ahora existe un *lock* ya tomado. En el parche para la versión 11, dentro de la función `getMetadataSectionPayload()` se aplicaba un lock (`vn_lock()`) incondicional sobre el `vnodo` del archivo que representa al archivo ejecutable. Ésto generaba que, al ya estar tomado el `vnodo`, el hilo quedaba esperando a tomar el *lock* que ya había tomado, entrando en espera infinita ya que estaba seteada la bandera `LK_RETRY` [86].

Es por ello, que se incorpora una línea de código para preguntar si el *lock* ya está tomado. Si lo está, se continúa normalmente. Si no, se ejecuta la instrucción para tomarlo usando `vn_lock()`. Ésto se observa en el Código 5.24.

Ante ello, y para asegurar las buenas prácticas de programación y evitar futuros *kernel panics*, se ha agregado dicha verificación en la versión del parche para FreeBSD 11 [87]

El trabajo realizado se encuentra bajo las etiquetas s50 Y s51 de la lista [88] del repositorio del parche para FreeBSD 12.3 [89], las etiquetas s49 y s52 de la lista [90] de *tags* del repositorio del parche versión FreeBSD 13 [91] y la etiqueta s53 de la lista [53] de etiquetas del repositorio de trabajo de FreeBSD 11 [47].

5.12.4. Análisis de resultados

Ventajas:

- Funcionalidad para leer y decodificar ejecutables con sección `.metadata` disponible en nuevas versiones de FreeBSD (versiones 12.3 y 13).
- Agregado de seguridad al chequear estado de *locks* para evitar *kernel panic*.

5.12.5. Próximos pasos

El siguiente paso del trabajo ha sido abstraer aún más la herramienta al generalizar el tipo de dato a insertar en la sección `.metadata`.

5.13. Décimotercera iteración

5.13.1. Objetivos a alcanzar

El objetivo de esta iteración ha sido lograr una mayor abstracción en el tipo de datos soportado para insertar en la sección `.metadata`.

5.13.2. Propuesta

La propuesta es que permita que los datos a enviar sean lo más abstractos posible. Evitando así datos que respondan a una estructura de tipo específica como contenedor de los datos a enviar. Los datos a insertar, hasta la iteración anterior, debían responder a una de las estructuras `Payload_X` usadas para definir una unidad de metadatos. Dichas estructuras tenían definidos campos que debían ser respetados para poder generar información válida. Además, para dar soporte a nuevas funcionalidades en *kernel*, se debía agregar una de dichas estructuras, así como definir su función de decodificación. Ésto implica un trabajo extra para expandir el soporte de nuevos tipos de datos.

En esta iteración se ha pensado y definido una nueva estructura que actúe como contenedor de cualquier tipo de datos, solamente para coherencia de organización de la sección `.metadata` y no como limitante del tipo de dato en sí.

```
Payload_A/2/4/c/int_size
```

Código 5.25: Contenido del archivo `decl_payA.txt` a convertir en binario [93]

```
File_size: 23

Datos leidos en ASCII - sizeof(uint8_t):
Payload_A/2/4/c/intsize

Canonical hex (two hex digits per byte) (same as hexdump -C)
50 61 79 6C 6F 61 64 5F 41 2F 32 2F 34 2F 63 2F 69 6E 74 73 69 7A 65
```

Código 5.26: Resultados de la conversión de `decl_payA.txt`

5.13.3. Implementación

La implementación ha comenzado con la idea de convertir cualquier archivo o dato que se desee enviar en su forma de archivo binario. Para ello, se ha implementado un programa que realice de conversor. El programa `chararray_binRuntimeEncDec.c` [92] abre el archivo deseado en forma binaria (*flag* `rb`), lee y extrae su contenido en dicho formato para, luego, guardarlo en formato binario.

Se ha definido un archivo de prueba, `decl_payA.txt`, cuyo contenido se presenta en el Código 5.25, es el *input* del programa convertidor. Su salida en formato hexademical canónico, es decir, dos dígitos hexadecimales por *byte*, se muestra en el Código 5.26. Dicha conversión es la misma que el resultado que arroja la salida del comando `hexdump` sobre archivo convertido [94]. Ver Código 5.27.

Una vez obtenido el archivo binario, se ha creado y definido la estructura `Payload_Binary` en el *header* `section_insertion_rawbin.h`. [95] Dicha estructura, que se puede observar en el Código 5.28, contiene los siguientes campos:

- `bin_data_size`: tamaño del archivo binario. Responde a un `sizeof()`.
- `charArray_bin_size`: largo efectivo del contenido del arreglo de caracteres `binary_data`. Responde a un `length()`.
- `binary_data[BINPAYLOAD_MAXSIZE]`: arreglo de caracteres, de tamaño 128 *bytes*, almacena el valor binario del archivo convertido, en forma de *string* respondiendo a valores canónicos hexadecimal de dos dígitos de los datos de dicho archivo binario.

La estructura `Payload_Binary` reemplaza las estructuras específicas de cada dato, tales como `Payload_A` y `Payload_B`, así como el número de funcionalidad de *kernel* ahora toma el valor **3**, de-

```
$ hexdump -C declarations.txt_array.bin

00000000  50 61 79 6c 6f 61 64 5f  41 2f 32 2f 34 2f 63 2f  |Payload_A/2/4/c/|
00000010  69 6e 74 73 69 7a 65                              |intsize|
00000017
```

Código 5.27: Comando hexdump con flag `-C` sobre `declarations.txt_array.bin.txt`

```c
typedef struct {
    long bin_data_size;
    size_t charArray_bin_size;
    char binary_data[BINPAYLOAD_MAXSIZE];         // Binary data as a string
} Payload_Binary;
```

Código 5.28: Estructura `Payload_Binary` [95]

```
INPUTFILE_1= decl_payA.txt_binary.bin
INPUTFILE_2= decl_payB.txt_binary.bin
FLAG_PLUGIN_ARG_1_KEY_VALUE= -fplugin-arg-plugin-3=$(INPUTFILE_1)
FLAG_PLUGIN_ARG_2_KEY_VALUE= -fplugin-arg-plugin-3=$(INPUTFILE_2)

plugin.so: $(PLUGIN_SOURCE_FILES)
  $(HOST_GCC) -shared $(CXXFLAGS) $(PLUGIN_SOURCE_FILES) -o $@

test:
  $(TARGET_GCC) -o $(EXE) $(EXE_SRC) -fplugin=./plugin.so $(
    ↪ FLAG_PLUGIN_ARG_1_KEY_VALUE) $(FLAG_PLUGIN_ARG_2_KEY_VALUE) -I$(
    ↪ INCLUDE_SYS_DIR)
```

Código 5.29: Extracto del `Makefile`. Uso de archivos binarios como argumentos para el *plugin* de GCC [96]

jándo libre uso para que en espacio de *kernel* se use el *payload* enviado para futuras implementaciones y usos. De tal forma, la estructura de la sección `.metadata` queda definida por las estructuras:

- `Metadata_Hdr`.
- `Payload_Hdr`.
- `Payload_Binary`.

El siguiente paso ha sido el de realizar la formulación e inserción de las líneas *metadata* en el código del programa de usuario, utilizando los desarrollos de *plugins* obtenidos en las iteraciones anteriores. En la presente iteración, se ha implementado la nueva función en el *plugin* de GCC.

Se ha dado soporte a múltiples archivos binarios posibles de insertar en la sección `.metadata` al pasar el nombre de dichos archivos como argumentos al momento de usar el *plugin* en la compilación del programa del usuario.

El Código 5.29 muestra que se desea insertar los archivos: `decl_payA.txt_binary.bin` y `decl_payB.txt_binary.bin`.

El *plugin* [97] ahora admite múltiples argumentos a procesar. Leyendo los archivos, extrayendo los datos en su equivalente *string* hexadecimal de dos dígitos por *byte*, asignándolo al campo `binary_data` de la estructura `Payload_Binary`, armando cada línea de `Payload_Binary` y su correspondiente `Payload_Hdr` por cada archivo binario pasado como *input* al *plugin*. Luego, completa la línea de la estructura `Metadata_Hdr` con información global de todas estructuras `Payload_Binary` agregadas. Finalmente, el *plugin* busca dentro del código de usuario [98] la variable `__PAYLOAD__` y la reemplaza por la línea *metadata* armada anteriormente.

```
__PAYLOAD__=static Metadata_Hdr metadata_header __attribute__((__used__,
    ↪ __section__(".metadata"))) = {2, sizeof(Payload_Hdr)};static Payload_Binary
    ↪  payloads[] __attribute__((__used__, __section__(".metadata"), __aligned__
    ↪ (8))) = {{24,48,"5061796C6F61645F412F322F342F632F696E745F73697A65"},{26,52,
    ↪ "5061796C6F61645F422F31362F7A2F772F636861725F73697A65"}};static Payload_Hdr
    ↪  payload_headers[] __attribute__((__used__, __section__(".metadata"))) =
    ↪ {{3, sizeof(payloads[0])},{3, sizeof(payloads[1])}};
```

Código 5.30: Línea con estructuras metadata resultado del uso del *plugin* de GCC [97]

```
1) // __CONCAT(exec_, __elfN(imgact)) // LectorELF // -- *** METADATA: 0
    ↪ xfffff80003e84a80 -- PAYLOAD SIZE: 352 -- RETURNED FLAG: 1 ***
4) // decodeMetadataSection() // LectorELF // Metadata_Hdr =
    ↪ metadata_header_decod //// m_number_payloads: 2 - ph_size: 16
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [0] //// p_function_number: 3 - ph_size: 144
4) // decodeMetadataSection() // payload_Binary_func() // Payload_Binary =
    ↪ payloadBinary_decod //// bin_data_size: 24 - charArray_bin_size: 48
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - Canonical
    ↪ hex: 50 61 79 6C 6F 61 64 5F 41 2F 32 2F 34 2F 63 2F 69 6E 74 5F 73 69 7A
    ↪ 65
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - ASCII read:
    ↪ Payload_A/2/4/c/int_size
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [1] //// p_function_number: 3 - ph_size: 144
4) // decodeMetadataSection() // payload_Binary_func() // Payload_Binary =
    ↪ payloadBinary_decod //// bin_data_size: 26 - charArray_bin_size: 52
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - Canonical
    ↪ hex: 50 61 79 6C 6F 61 64 5F 42 2F 31 36 2F 7A 2F 77 2F 63 68 61 72 5F 73
    ↪ 69 7A 65
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - ASCII read:
    ↪ Payload_B/16/z/w/char_size
```

Código 5.31: Resultado de la decodificación en *kernel* usando estructuras `Payload_Binary`.

La línea final armada por el *plugin* de GCC se presenta en el Código 5.30.

Finalmente, se ha implementado la decodificación de la estructura `Payload_Binary` dentro del código de *kernel* como parte de los parches para FreeBSD 11, 12.3 y 13. Se han modificado los siguientes archivos:

- `metadata_elf_reader.c`: [99] [100] [101] implementación de la función `payload_Binary_func()`.
- `metadata_elf_reader.h`: [102] [103] [104] declaración de la función `payload_Binary_func()`.
- `metadata_payloads.h`: [105] [106] [107] definición de la estructura `Payload_Binary`.

La función de decodificación `payload_Binary_func()` guarda el valor leído desde el arreglo campo `binary_data` de la estructura `Payload_Binary` e imprime el resultado por pantalla.

La salida observada en los *logs* del *kernel* se presenta en el Código 5.31.

El resultado de la presente iteración se observa bajo las etiquetas:

- s57, s58, s59, s60, s61, s62, s63, s65, s68, s69, s70 y s73 [36] del repositorio principal [52].

- s64 [90] del repositorio del parche versión FreeBSD 13 [91]
- s66 [88] del repositorio del parche para FreeBSD 12.3 [89].
- s67 [53] del repositorio de trabajo de FreeBSD 11 [47].

5.13.4. Análisis de resultados

Ventajas:

- Mayor abstracción en los datos soportados.
- Admisión de múltiples archivos a insertar dentro de la sección `.metadata`.

5.13.5. Próximos pasos

El siguiente paso planteado en esta iteración ha sido la de implementar los cambios respecto a inserción y decodificación usando estructuras `Payload_Binary` en el desarrollo del *plugin* de `CLANG/LLVM`, así como lograr el funcionamiento total del *plugin* para dicha *suite* de compilación.

5.14. Décimocuarta iteración

5.14.1. Objetivos a alcanzar

El objetivo de la iteración número 14 ha sido finalizar la implementación del *plugin* para `CLANG`, que permita insertar automáticamente las líneas *metadata* en ejecutables ELF, utilizando estructuras `Payload_Binary` como unidades de datos.

5.14.2. Propuesta

La propuesta de esta iteración ha sido la de completar el desarrollo del *plugin* de `CLANG`, iniciado en la iteración 11 (ver Sección 5.11), considerando ahora la generalización del tipo de datos al utilizar estructuras `Payload_Binary` con el valor binario hexadecimal canónico de dos dígitos del archivo a insertar en el ejecutable ELF. Tal como se había mencionado anteriormente, el uso del *plugin* permite realizar el procesamiento del archivo a insertar en el ELF de forma automática y transparente para el usuario de la herramienta.

La iteración 11 (ver Sección 5.11) había terminado al mostrar el reemplazo de la función `void PAYLOAD()` (ver Código 5.22) en el código de usuario por una línea *metadata* reducida con sólo declaración de la estructura `Metadata_Hdr` (ver Código 5.23).

Se buscará compatibilizar el uso de variable `__PAYLOAD__`, variable que es buscada por el *plugin* de GCC (ver Código 5.16), en lugar de la función del mismo nombre. Permitiendo así que el programador utilice un único identificador para declarar la línea en la cual desea insertar las líneas

metadata, independientemente si la compilación se realiza utilizando la *suite* de compilación GCC o CLANG/LLVM.

5.14.3. Implementación

La implementación ha empezado teniendo en cuenta el desarrollo obtenido para el plugin de GCC de la iteración anterior [97]. Dicho desarrollo permite la inserción de múltiples archivos binarios en forma de `Payload_Binary` dentro de la sección `.metadata` de los ejecutables ELF. Sin embargo, el mismo está escrito en lenguaje C. Todo desarrollo de *plugin* de CLANG debe realizarse en lenguaje C++. Es por ello, que se ha formulado un programa de prueba que permita obtener las líneas *metadata*. El código `prueba.cpp` [108] representa dicho desarrollo. Toma como entrada los mismos dos archivos binarios (`decl_payA.txt_binary.bin` [109] y `decl_payB.txt_binary.bin` [110]) que el *plugin* de CLANG.

Una vez generada y verificada la línea con las estructuras *metadata*, el siguiente paso ha sido implementar dicho desarrollo como parte del código de un *plugin* de CLANG.

El archivo `RenameMacro_Payload.cpp` [111] contiene la implementación del *plugin* de CLANG con el desarrollo obtenido en el programa de prueba anteriormente mencionado.

```
1 Payload_A/2/4/c/int_size
```

Código 5.32: Contenido del archivo `decl_payA.txt_binary.bin` [109]

```
1 Payload_B/16/z/w/char_size
```

Código 5.33: Contenido del archivo `decl_payB.txt_binary.bin` [110]

Los archivos binarios `decl_payA.txt_binary.bin` [109] (Código 5.32) y `decl_payB.txt_binary.bin` [110] (Código 5.33) son sumistrados como entradas al *plugin*.

Éstos serán cargados y guardados en el vector de *strings* `args`, una vez dentro del *plugin*. En la función `ParseArgs`, dentro de la clase *PluginRenameAction*, dicho vector será copiado al vector de *strings* global `global_args`. Allí se copian los nombres de los archivos pasados como argumentos en el uso del *plugin*. Luego de ello, se llama a la función `CreateASTConsumer`, para crear una instancia de `ASTConsumer`, interfaz usada para recorrer el árbol AST que representa al código analizado.

El objetivo actual ha consistido en visitar nodos del AST que representen a variables, instancias de clases `VarDecl`, y no a funciones, instancias `FunctionDecl` como anteriormente se había definido. Ésto es posible empleando la función `VisitVarDecl` dentro de la clase `RecursiveASTVisitor` [73]. Tal como se expone en el Código 5.34, solamente si se encuentra la variable `__PAYLOAD__` dentro del código del programa del usuario y si se ha suministrado al menos un archivo como argumento del *plugin*, se llama a la función `getMetadataLines()` para iniciar el procesamiento de archivos y armado de las líneas *metadata*.

La función `getMetadataLines()` [111], a la cual se le pasa como argumento el vector global

```
virtual bool VisitVarDecl(VarDecl *vardecl) {
    string VarDeclName = vardecl->getName().str();
    if (VarDeclName == "__PAYLOAD__") {
        // 0) Check if no arguments were passed
        if(global_args.empty() == true){
            cout << "No arguments were passed. __PAYLOAD__ is not replaced";
            return false;   // Return false to abort the tree traversing
        }
        // 1) Obtain the whole line to replace
        string complete_metadata_replace = getMetadataLines(global_args);
        // 2) Get the location in source code of __PAYLOAD__
        SourceLocation location = vardecl->getLocation();
        // 3) Replace __PAYLOAD__ with the full processed metadata lines
        rewriter.ReplaceText(location.getLocWithOffset(0) , VarDeclName.length(),
    ↪  complete_metadata_replace);
        return false;       // Return false to abort the entire traversal of the
    ↪ tree
    }
    return true;
}
```

Código 5.34: Función `VisitVarDecl` [111] para encontrar la variable `__PAYLOAD__`

`global_args`, por cada uno de los nombres de archivo (elementos del vector) lleva a cabo la lectura del archivo binario, extracción de su contenido, transformación a *string* hexadecimal canónico de dos dígitos por *byte*, llenado de sus estructuras `Payload_Binary` y `Payload_Hdr`, llenado de la estructura `Metadata_Hdr` con información global del contenido de toda la sección `.metadata` y armado de las líneas de código necesarias para insertar los datos en el programa a compilar. Esta función retorna el `string macro_complete` con la línea de código entera con la declaración y definición de cada una de las estructuras resultantes del procesamiento.

De vuelta en la función `VisitVarDecl`, el *string* con la línea completa se denomina `complete_metadata_replace`.

Se busca la posición y línea de la variable `__PAYLOAD__` en el código, y allí, se reemplaza la declaración de la variable por el *string* `complete_metadata_replace`, con las líneas que indican la inserción de las estructuras que componen a la sección `.metadata` con los valores de los archivos pasados como argumento al *plugin*.

El *plugin* de `CLANG` se compila utilizando el archivo `CMake` [114]. El uso del *plugin* se realiza durante la compilación del programa de usuario `test.c` (ver Código 5.35) [112] utilizando el script `test.sh` [113]. Observar el Código 5.36 para ver el comando de compilación y uso del *plugin*.

El Código 5.37 muestra el código del programa de usuario con las líneas *metadata* reemplazadas correctamente.

Compilado el programa de usuario con el código que contiene las estructuras definidas y completadas, se obtiene el archivo ejecutable ELF correspondiente y se ejecuta el mismo.

El resultado por *logs* del *kernel*, expuesto en el Código 5.38 [115], muestra la correcta inserción

```c
#include <stdio.h>
#include <string.h>
#include <stddef.h>
#include <inttypes.h>
#include "section_insertion_rawbin.h"

__PAYLOAD__;

int main(int argc, char *argv[]) {

    printf("\n******** codeinsertion finished ********\n");

    return 0;
}
```

Código 5.35: Programa de usuario con variable __PAYLOAD__ a ser reemplazada [112]

```
clang11 -Xclang -load  -Xclang ./build/libRenameMacro_Payload.so -Xclang -plugin
    ↪ -Xclang RenameMacro_PayloadPlugin -Xclang -plugin-arg-
    ↪ RenameMacro_PayloadPlugin -Xclang decl_payA.txt_binary.bin -Xclang -plugin-
    ↪ arg-RenameMacro_PayloadPlugin -Xclang decl_payB.txt_binary.bin  -c src/test
    ↪ .c -Isrc
```

Código 5.36: Comando para compilación de programa de usuario utilizando el plugin de CLANG [113]

```c
#include <stdio.h>
#include <string.h>
#include <stddef.h>
#include <inttypes.h>
#include <sys/metadata_payloads.h>

static Metadata_Hdr metadata_header __attribute__((__used__, __section__(".
    ↪ metadata"))) = {2, sizeof(Payload_Hdr)};static Payload_Binary payloads[]
    ↪ __attribute__((__used__, __section__(".metadata"), __aligned__(8))) =
    ↪ {{24,48,"5061796C6F61645F412F322F342F632F696E745F73697A65"},{26,52,"5061796
    ↪ C6F61645F422F31362F7A2F772F636861725F73697A65"}};static Payload_Hdr
    ↪ payload_headers[] __attribute__((__used__, __section__(".metadata"))) =
    ↪ {{3, sizeof(payloads[0])},{3, sizeof(payloads[1])}};

int main(int argc, char *argv[]) {

        printf("\n******** codeinsertion finished ********\n");

        return 0;
}
```

Código 5.37: Programa de usuario con líneas *metadata* colocada por el plugin de CLANG

```
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [0] //// p_function_number: 3 - ph_size: 144
4) // decodeMetadataSection() // payload_Binary_func() // Payload_Binary =
    ↪ payloadBinary_decod //// bin_data_size: 24 - charArray_bin_size: 48
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - Canonical
    ↪ hex: 50 61 79 6C 6F 61 64 5F 41 2F 32 2F 34 2F 63 2F 69 6E 74 5F 73 69 7A
    ↪ 65
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - ASCII read:
    ↪ Payload_A/2/4/c/int_size
4) // decodeMetadataSection() // LectorELF // Payload_Hdr = payload_header_decod
    ↪ [1] //// p_function_number: 3 - ph_size: 144
4) // decodeMetadataSection() // payload_Binary_func() // Payload_Binary =
    ↪ payloadBinary_decod //// bin_data_size: 26 - charArray_bin_size: 52
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - Canonical
    ↪ hex: 50 61 79 6C 6F 61 64 5F 42 2F 31 36 2F 7A 2F 77 2F 63 68 61 72 5F 73
    ↪ 69 7A 65
4) // decodeMetadataSection() // payload_Binary_func() // bin_data - ASCII read:
    ↪ Payload_B/16/z/w/char_size
```

Código 5.38: Logs del kernel con la decodificación de las estructuras *Payload* correctamente insertadas por plugin de CLANG [115]

de datos utilizando el *plugin* para `CLANG` desarrollado en esta iteración.

El desarrollo obtenido en esta iteración está reflejado en las etiquetas s71, s72, s74, s75 y s76 de la lista de etiquetas [36] del repositorio principal [52].

5.14.4. Análisis de resultados

Ventajas:

- Obtención de herramienta de procesamiento de datos, reemplazo e inserción de líneas *metadata* para la *suite* de compilación `CLANG/LLVM`.
- Estandarización en uso de identificador a reemplazar: `__PAYLOAD__`.

5.14.5. Próximos pasos

El paso siguiente corresponde a la versión final de la implementación del protocolo de comunicación desde espacio de usuario a espacio de *kernel*.

5.15. Décimoquinta iteración

5.15.1. Objetivos a alcanzar

El objetivo principal de la iteración número 15 ha sido el de obtener la versión final del desarrollo presente.

```
//////////////////// METADATA ////////////////////////////
void  *p_metadata_addr;     /* Address of the allocated data from metadata
    ↪ section */
u_long p_metadata_size;       /* Size of the whole metadata section */
int p_metadata_section_flag;    /* Flag of metadata section presence in
    ↪ executable */
```

Código 5.39: Campos metadata de la estructura `proc` del *header* `proc.h` [116]

5.15.2. Propuesta

La propuesta ha consistido en realizar la evaluación de las distintas partes del desarrollo para poder realizar la limpieza de aquellas funcionalidades que no aportan valor al objetivo final y obtener la mínima versión del protocolo que permita cumplir con el objetivo principal del proyecto.

Sumado a ello, lograr la máxima automatización del uso de las herramientas de inserción por parte del usuario.

Finalmente, se ha propuesto ajustar el uso de recursos de sistema en su mínima expresión. Asimismo, ajustar el tamaño de datos posibles a insertar para reflejar escenarios reales de uso.

5.15.3. Implementación

La implementación ha comenzado con cambios dentro del código de *kernel*, específicamente con la eliminación del campo `p_metadata` de la estructura `proc` del archivo `proc.h` [116].

De tal forma, la estructura `proc` contiene los campos observados en el Código 5.39

Posteriormente, se ha decidido que adoptar el *payload* de tipo `Payload_Binary`; permitiendo así lograr la máxima generalización de tipo de datos a poder insertar en el ejecutable. Para ello, entonces, se ha tenido que borrar toda implementación del resto de tipos de *payload* soportados hasta el momento: `Payload_A` y `Payload_B`.

Esto ha implicado remover las definiciones de estructuras `Payload_A` y `Payload_B` en el *header* `metadata_payloads.h` [117]. El Código 5.40

Dentro del header `metadata_elf_reader.h` [118],
se han removido las declaraciones de las funciones `payload_A_func` y `payload_B_func`. Consecuentemente, dentro del código fuente `metadata_elf_reader.c` [119], se han borrado las definiciones de las funciones anteriormente nombradas así como modificación del arreglo de funciones `payload_functions`, conteniendo solamente la llamada a la única función de decodificación de *payload* disponible: `payload_Binary_func`.

Con respecto al desarrollo realizado en espacio de usuario, se han introducido mejoras dentro de los desarrollos realizados.

En primer lugar, se ha dejado de lado el uso del programa convertidor a codificación binaria

```c
#ifndef METADATA_PAYLOADS_H_
#define METADATA_PAYLOADS_H_

#define PAYLOADS_TOTAL 1      // IMPORTANT: update this number each time you add or remove a Payload_X struct
#define BINPAYLOAD_MAXSIZE 2048

#define PAYLOAD_BINARY_ARGS_COUNT 3     // For security

typedef struct {
    long bin_data_size;               // Size in bytes of the raw binary data
    size_t charArray_bin_size;            // Count of characters of the binary_data as string
    char binary_data[BINPAYLOAD_MAXSIZE];         // Binary data as a string
} Payload_Binary;

typedef struct {
    int p_function_number;         // Function number associated to the payload
    size_t p_size;                 // Size of the payload
} Payload_Hdr;

typedef struct {
    int m_number_payloads;          // Total number of different payloads in the section
    size_t ph_size;                 // Size of an entry in the payload header table
} Metadata_Hdr;

#endif /* METADATA_PAYLOADS_H_ */
```

Código 5.40: Archivo cabecera `metadata_payloads.h` [117] con soporte de único tipo de *payload*: `Payload_Binary`

```
$ sh test.sh prueba.c decl_payA.txt decl_payB.txt
```

Código 5.41: Comando para ejecutar el *script* `test.sh` [125] con los archivos `decl_payA.txt` [126] y `decl_payB.txt` [127] para insertarlos utilizando *plugin* de CLANG

(`binaryGenerator`). Ésto ha resultado de analizar la no existencia de diferencia entre proveer el archivo convertido (salida del convertidor) y la apertura del archivo original con el *flag* `b` al momento de procesar los argumentos pasados a ambos *plugins*.

Luego, siguiendo la decisión de solamente dar soporte al tipo de *payload* `Payload_Binary`, se ha limpiado toda referencia a `Payload_A` y `Payload_B` en los *plugins*. De tal forma, el número de función que representa a `Payload_Binary` es el número 1 y ha sido establecido así dentro del código del *plugin* para GCC [120] y para CLANG [121]. También se ha aumentado el límite del tamaño del contenido binario canónico en formato hexadecimal de dos dígitos a 2048 caracteres. Finalmente, la referencia a la estructura interna de la *struct* `Payload_Binary` se hace en los códigos de *plugins* referenciando a su definición dentro del *header* `metadata_payloads.h` [117] presente en *kernel* y no a un archivo local, como se realizaba hasta la iteración anterior.

La automatización ha incrementado su nivel mediante el desarrollo de *scripts* para ambos *plugins*. El *script* `setup.h`, uno del mismo nombre para GCC [122] como para CLANG [123], permite realizar la compilación del *plugin* detectando automáticamente la versión del sistema operativo y *seteando* los *paths* a los archivos necesarios para generarlos sin necesidad de que el usuario esté modificando archivos Makefile o CMake para lograr la obtención de las librerías compartidas que representan a los *plugins*.

Finalmente, la mayor ganancia de automatización se ha dado mediante el mayor desarrollo del *script* `test.sh`. Al igual que el *script* anterior, existen dos versiones del mismo *script*, uno para cada *plugin*: GCC [124] y CLANG [125]. Éste recibe como primer argumento el nombre del código fuente del programa de usuario que se quiere compilar, y a su correspondiente ejecutable, se desea insertar metadatos. Posteriormente, recibe cero o más argumentos que serán los archivos a insertar mediante el uso de los *plugins*. Ésto permite que el usuario no tenga que ejecutar el comando de compilación de programa de usuario y uso del *plugin* de forma directa, evitando seguir un formato estricto en orden de palabras y banderas.

El Código 5.41 muestra la forma de ejecutar el *script* nombrado para CLANG.

5.15.4. Análisis de resultados

Ventajas:

- Limpieza de funcionalidades no relevantes.
- El desarrollo se sujeta a un modelo listo para ser considerado como *release*.

- Alto grado de transparencia para usuario en cuanto a uso de la herramienta.
- Alto grado de generalización de tipo de dato.

5.15.5. Próximos pasos

El próximo paso consiste en desarrollar la funcionalidad dentro del *kernel* que aproveche el *payload* de tipo `Payload_Binary` enviado dentro de los archivos ELF ejecutables. Este paso y los siguientes quedan en manos de posteriores proyectos integradores y/o contribuidores al parche de esta herramienta.

Capítulo 6

Descripción del modelo experimental

El modelo obtenido como resultado del presente trabajo se expone como el desarrollo de código que permite brindar una herramienta empleada para implementar un protocolo de comunicación entre espacio de usuario y espacio de *kernel* mediante la inserción de datos dentro de archivos ejecutables de formato ELF.

Los datos insertados forman parte de la estructura interna del archivo ejecutable, específicamente dentro de una sección especial creada para ser contenedora de metadatos. Aprovechando el proceso de carga de ejecutables en el sistema operativo cuando estos son ejecutados, se realiza la búsqueda, extracción y decodificación de los datos insertados; teniendo así disponibles dichos datos en espacio de *kernel*.

El modelo está formado por dos componentes fundamentales:

- *Plugin* para inserción de metadatados.
- Parche de *kernel* de FreeBSD para decodificación de metadatos insertados.

6.1. *Plugin* para inserción de metadatos

El modelo provee dos *plugins*, uno para la *suite* de compilación `GCC` y otro para `CLANG/LLVM`, a emplear en espacio de usuario.

El *plugin* se utiliza en el momento en que se realiza la compilación del programa de usuario, que contiene la variable `__PAYLOAD__` que será reemplazada por el *plugin* para insertar código que implemente la inserción de datos.

Los datos a insertar son suministrados como argumentos al *plugin*.

El resultado del uso del *plugin* consiste en el archivo ELF ejecutable que contiene el programa de usuario. Dicho ejecutable ELF contiene en su estructura los datos indicados para la inserción.

El nombre del *plugin* de `GCC` es `InsertPayload_GCCPlugin`. Mientras que el *plugin* para `CLANG/LLVM` es llamado `InsertPayload_Plugin`.

Para llevar a cabo la compilación del *plugin*, el usuario debe solamente ejecutar el *script* `setup.sh`, tanto en su versión para GCC como para CLANG.

Para poder compilar el código de usuario e insertar los datos deseados dentro del programa correspondiente, se debe ejecutar el *script* `test.sh`. Éste permite utilizar el *plugin* compilado anteriormente para realizar la compilación del ejecutable final basándose en el código fuente del programa de usuario e insertando en él los datos pasados como argumentos a dicho *script*.

Por motivos prácticos, cada archivo que se desea insertar en el ejecutable final, debe tener un tamaño menor o igual a 2048 *bytes*. Por los mismos motivos, se ha limitado a 4 (cuatro), los archivos máximos posibles a insertar dentro de los ejecutables, siendo el tamaño máximo de la totalidad de los *payloads* de 8192 *bytes*. Si se violan cualquiera de estos límites, un error por terminal será presentado y el proceso de inserción y compilación del programa de usuario será detenido automáticamente.

Otra limitación se encuentra en el *script* `test.sh` mencionado anteriormente. La implementación actual solamente soporta un único archivo de código fuente (`.c`) como primer argumento del *script* `test.sh`, que representa el código del programa de usuario. Se deja para futuras implementaciones el manejo de más archivos fuente que conformen el programa final.

6.2. Parche de *kernel*

Se han desarrollado dos parches para el *kernel* del sistema operativo FreeBSD. Las versiones a las cuales el modelo da soporte son:

- FreeBSD 12.3.
- FreeBSD 13.

Los parches agregan la funcionalidad necesaria para detectar la existencia de datos insertados dentro de todo archivo ejecutable ELF que se ejecute en el sistema, su extracción, decodificación e impresión en registros (*logs*) del *kernel*.

NOTA: Debido a que las versiones de las *suites* de compilación, utilizadas para el desarrollo de los *plugins* del apartado anterior (Sección 6.1); son incompatibles con la versión 11 de FreeBSD, no se incluye una *release* del parche para el *kernel* de dicha versión del sistema operativo.

6.2.1. *Changelog*

Versión 1.0.0

Adiciones

- Archivo de cabecera `metadata_elf_reader.h` (`sys\sys\metadata_elf_reader.h`) [128] [129] Contiene las declaraciones de funciones de detección, lectura y decodificación de los datos insertados en el ejecutable ELF. También especifica el nombre de la sección a buscar.
- Archivo de cabecera `metadata_payloads.h` (`sys\sys\metadata_payloads.h`) [130] [131] Almacena las definiciones de las estructuras utilizadas para organizar la inserción de datos en los archivos ejecutables ELF y hacer más fácil la decodificación de los datos. Sumado a ello, detalla también macros de seguridad y de límites de tamaño de distintas variables estructuradas.
- Archivo fuente `metadata_elf_reader.c` (`sys\kern\metadata_elf_reader.c`) [132] [133] Contiene la definición de las funciones de detección, lectura y decodificación de los datos insertados en espacio de usuario.

Cambios

- Campo `p_metadata_addr` en estructura `proc` (`sys\sys\proc.h`) [134] [135] Almacena la dirección de inicio del espacio de memoria reservado para guardar el contenido de la sección `.metadata`, extraído de la estructura del ejecutable ELF.
- Campo `p_metadata_size` en estructura `proc` (`sys\sys\proc.h`) [134] [135] Tamaño de la sección `.metadata` del ejecutable ELF.
- Campo `p_metadata_section_flag` en estructura `proc` (`sys\sys\proc.h`) [134] [135] *Flag* que indica la presencia (1), ausencia (-1) o error (-2) en la detección de la sección `.metadata` del ejecutable ELF que se carga en el sistema operativo.
- En `imgact_elf.c` (`sys\kern\imgact_elf.c`) [136] [137], archivo de código fuente del activador de imagenes de archivos ejecutables de formato ELF, se agregan líneas de código que invocan a funciones definidas en `metadata_elf_reader.c` para llevar a cabo la búsqueda, detección, lectura y decodificación de los metadatos insertados.
- En el archivo de configuración `files` (`sys\conf\files`) [138] [139], se agrega línea para indicar la compilación del código definido en `sys\kern\metadata_elf_reader.c` al generar la imagen del *kernel*.

6.2.2. Funcionalidad agregada en parches de *kernel*

La funcionalidad agregada en el parche de *kernel* generado en este proyecto se encuentra contenida en las funciones definidas en el archivo de código fuente de *kernel* `metadata_elf_reader.c` (`sys\kern\metadata_elf_reader.c`) [132] [133] Dichas funciones llevan a cabo la búsqueda, detección, extracción, decodificación e impresión de los metadatos insertados en el archivo ejecutable ELF, siendo este último el generado con la ayuda de los *plugins* de inserción de datos en espacio de usuario. Estas funciones, y la descripción correspondiente a cada una, se encuentran especificadas en la Tabla 6.1

Implicancias de las funciones agregadas frente al resto del *kernel*

El parche de kernel que permite incorporar la búsqueda, detección y decodificación de datos insertados en la sección `.metadata`, incorpora código que está activo solamente en el momento del

Tabla 6.1: Nombre y descripción de las funciones definidas en `metadata_elf_reader.c`.

Función	Descripción
`getMetadataSectionPayload`	Parsea la tabla de secciones ELF del ejecutable en carga en *kernel*. Itera sobre las secciones buscando la sección `.metadata`. Si esta se encuentra, se reserva memoria y se guarda allí el contenido de la sección buscada. Devuelve la dirección de inicio del espacio en memoria en donde se encuentra la copia de la sección `.metadata`.
`copyMetadataToProc`	Setea los campos `p_metadata_addr`, `p_metadata_size` y `p_metadata_section_flag` de la estructura `proc`, asociada al proceso del ejecutable en carga, con el resultado de la búsqueda y detección de la sección `.metadata`.
`decodeMetadataSection`	Parsea la sección `.metadata` extraída a partir de la copia guardada en el espacio de memoria previamente reservado en la búsqueda de la sección. Luego, utilizando información del tamaño y posición de las estructuras internas de organización de la sección `.metadata` (estructuras definidas en `metadata_payloads.h` [130] [131] y empleadas como *wrapper* para los datos insertados) se obtiene la dirección de cada unidad de *payload* y su respectivo tipo para poder invocar a la función de decodificación específica de cada clase de *payload*.
`payload_Binary_func`	Lleva a cabo la decodificación de *payloads* de tipo `Binary`. Extrae los datos *raw* en formato hexadecimal de dos dígitos del arreglo de *char* del *payload*, `binary_data`, y los decodifica en formato ASCII. Finalmente, los imprime por consola.

armado de la imagen del proceso correspondiente al ejecutable que se carga en el sistema operativo, al iniciar su ejecución. Específicamente, el llamado a dichas funciones se agrega dentro del código del activador de imágenes de archivos ELF: `imgact_elf.c`, justo antes de que se realice la iteración sobre las cabeceras de programas del archivo ELF y se inicie la búsqueda de los segmentos `PT_LOAD`, `PT_INTERP` y `PT_GNU_STACK` (Ver Sección 4.1) Al terminar de decodificar los datos insertados, el proceso de carga del programa y llenado de la imagen del proceso asociado continúa de forma normal, sin cambio alguno en los pasos siguientes ya existentes.

Todo proceso que corra bajo la nueva versión del *kernel*, tendrá adicionado tres campos en la estructura `proc` asociada a dicho proceso: `p_metadata_addr`, `p_metadata_size` y `p_metadata_section_flag`. Estos campos reflejan el resultado de la búsqueda de la sección `.metadata`.

Si la sección `.metadata` fue encontrada al iterar sobre la tabla de secciones del ejecutable ELF, se guarda su contenido en un espacio de memoria reservado para ello. Luego, se setean los *flags* de presencia en la estructura `proc` ya mencionada. Posteriormente, se extrae y decodifica cada unidad de datos insertada. A continuación, se continúa con la normal carga del ejecutable y el normal llenado

de la imagen del proceso. La carga del ejecutable finaliza normalmente al realizarse el enlace dinámico de librerías (si fuese necesario), obteniendo así la imagen del proceso completa. Por último, se retorna a espacio de usuario para iniciar la ejecución en el punto de entrada al segmento de código del programa.

Si no fue realizada inserción de datos en espacio de usuario, la carga del ejecutable ELF sigue su curso original. En este caso, la sección `.metadata` no fue insertada en el ejecutable ELF. Entonces, el resultado de la búsqueda será negativo y esto será reflejado en la estructura `proc` del proceso en cuestión, indicando mediante los campos adicionados que el ejecutable no contiene datos insertados. Por lo tanto, no se continuará con la fase extracción y decodificación ya que no hay datos sobre los cuales realizar dicha acción; y se continúa con la carga del ejecutable hasta finalizar.

Los *plugins* que permiten la inserción están limitados a ejecutarse en espacio de usuario, durante la compilación del código que el programador genera.

El parche de *kernel* permite, en espacio de *kernel*, la detección de la sección insertada y realiza las validaciones correspondientes de los datos, sumados utilizando los *plugins*, en cuanto a estructura interna, tamaños y tipos de *payloads* que la sección puede contener en esta versión de prototipo del desarrollo.

Limitaciones del modelo

El desarrollo realizado y el modelo obtenido es una prueba de concepto y una primera versión de prototipo del mecanismo de comunicación entre espacio de usuario a espacio de *kernel* planteado.

En esta versión, la verificación de la correcta decodificación en *kernel* de los datos insertados en espacio de usuario se ha realizado mediante la herramienta `dmesg`. De esta forma, se escriben los datos decodificados en *logs* de *kernel* y pueden estos ser leídos por consola.

Excede al trabajo integrador realizado la realización de mayor tipo de pruebas dentro del *kernel*, ya sea armado de pruebas de sistema que comprendan así tanto espacio de usuario como de *kernel* como un todo. Supera también a nuestro trabajo el análisis de impacto en tamaño de la memoria, que sucede al agregar los tres nuevos campos en la estructura `proc` de todos los procesos del sistema. Sumado a ello, excede a nuestro trabajo el análisis de impacto en el rendimiento del sistema y el retardo en la ejecución de los programas que se produce al haber incorporado los pasos de búsqueda, detección y decodificación de datos insertados en la carga de los ejecutables ELF en el sistema. Se recomienda abordar lo detallado en estas ultimas líneas en trabajos o proyectos integradores futuros, que utilicen como base el mecanismo desarrollado en el presente proyecto (Ver ideas para trabajos futuros en Sección III)

Capítulo 7

Resultados

Los resultados de la herramienta desarrollada se muestran en este apartado mediante el uso de un ejemplo de uso, recorriendo las distintas partes que componen al trabajo realizado. Se muestran también las entradas y salidas de cada elemento así como los comandos de compilación y/o ejecución para el sistema operativo FreeBSD.

7.1. Archivos a insertar

En primer lugar, se deben seleccionar los archivos a insertar dentro del ejecutable ELF que se cargará en el sistema.

En nuestro caso, se han seleccionado dos archivos de texto. Ellos son: `decl_payA.txt` [140] [141] y `decl_payB.txt` [142] [143].

Los Códigos 7.1 y 7.2 muestran el contenido de ambos archivos.

7.2. Código fuente del programa de usuario

El siguiente paso es, ya sea, escribir el código fuente y archivos relacionados necesarios para generar el programa mediante la compilación de los mismos.

Para poder utilizar el protocolo de comunicación desarrollado en este trabajo, el programador solamente debe agregar las siguientes líneas a su código.

1. `#include <sys/metadata_payloads.h>`
2. `__PAYLOAD__;`

```
1 Payload_A/2/4/c/int_size
```

Código 7.1: Contenido del archivo `decl_payA.txt` a insertar con los *plugins* para CLANG [140] y GCC [141]

```
Payload_B/16/z/w/char_size
```

Código 7.2: Contenido del archivo `decl_payB.txt` a insertar con los *plugins* para CLANG [142] y GCC [143]

```c
#include <stdio.h>
#include <string.h>
#include <stddef.h>
#include <inttypes.h>
#include <sys/metadata_payloads.h>  // Archivo cabecera

__PAYLOAD__;    // Variable global

int main(int argc, char *argv[]) {
    printf("\n******** codeinsertion finished ********\n");
    return 0;
}
```

Código 7.3: Código fuente (`prueba.c` [144] para GCC y `test.c` [145] para CLANG) con líneas que habilitan la inserción de metadatos

La primera línea permite invocar al archivo cabecera, también llamado *header*, `metadata_payloads.h`, situado en el directorio `sys/sys` dentro del código de *kernel*. La segunda línea declara la variable global `__PAYLOAD__`, variable que será detectada por los *plugins* de compilación desarrollados y que será reemplazada por código que permita indicar al compilador que inserte los datos leídos de los archivos deseados en el ejecutable final. Se debe respetar el formato de dicha variable doble guión bajo antes y después nombre de la variable, así como el punto y coma final.

El Código 7.3 muestra el código fuente de un programa de usuario, que contiene las líneas mencionadas. Dicho archivo de código fuente de C se denomina `prueba.c` [144] para GCC, y `test.c` [145] para CLANG.

7.3. Compilación de *plugin* de compilación

Para poder llevar a cabo la inserción de metadatos dentro del código de usuario, es necesario utilizar herramientas extra durante la fase de compilación del mismo.

Se entrega un *plugin* que agrega la funcionalidad de la inserción de metadatos a las *suites* de compilación: GCC y CLANG/LLVM. Existe un *plugin* distinto para cada una de las *suites* de compilación empleadas.

En un principio, se debe compilar el *plugin*, generando así una librería compartida que se utilizará luego para la compilación del programa del usuario.

```
$ sudo pkg install llvm11
$ clang11 -v

clang version 11.0.1
Target: x86_64-portbld-freebsd13.0
Thread model: posix
InstalledDir: /usr/local/llvm11/bin
```

Código 7.4: Comandos y salida de la instalación del paquete de LLVM 11

```
$ sh setup.sh
-- The C compiler identification is Clang 11.0.1
-- The CXX compiler identification is Clang 11.0.1
...
...
[100%] Linking CXX shared library libInsertPayload_Plugin.so
/usr/local/bin/cmake -E cmake_link_script CMakeFiles/InsertPayload_Plugin.dir/
    ↪ link.txt --verbose=1
/usr/bin/c++ -fPIC -shared -Wl,-soname,libInsertPayload_Plugin.so -o
    ↪ libInsertPayload_Plugin.so CMakeFiles/InsertPayload_Plugin.dir/
    ↪ insertPayload_Plugin.cpp.o
```

Código 7.5: Comando y salida reducida de la ejecución del *script* `setup.sh` para la compilación del *plugin* para CLANG: `InsertPayload_Plugin` [146]

7.3.1. *Plugin* para CLANG

En primer lugar, se debe instalar el paquete del compilador LLVM en su versión 11. Se recomienda luego, verificar la correcta instalación de dicha versión. También se recomienda verificar que el *path* de instalación del mismo está en la dirección `/usr/local/llvm11/bin`.

El Código 7.4 muestra los comandos necesarios para llevar a cabo la instalacón del paquete LLVM 11.

Dentro del directorio `CLANGPlugin`, se provee el *script* `setup.sh` [146] que se encarga de compilar el *plugin* para CLANG. La ejecución del mismo genera como salida la librería compartida `InsertPayload_Plugin`.

El Código 7.5 muestra cómo realizar esta acción.

7.3.2. *Plugin* para GCC

La compilación del *plugin* para la *suite* GCC implica el uso de la versión 11 del compilador. Por lo tanto, se debe instalar dicha versión.

El Código 7.6 expone el comando a ejecutar para instalar el paquete que contiene dicha versión.

Se facilita el *script* `setup.sh` para realizar la compilación del *plugin* de GCC de manera automática y transparente para el usuario del protocolo.

En el Código 7.7 se observa cómo ejecutar dicho *script*. El resultado del mismo consiste en la obtención de la librería compartida `InsertPayload_GCCPlugin` que representa al *plugin* de GCC.

```
$ sudo pkg install gcc11
$ gcc11 -v
Using built-in specs.
COLLECT_GCC=gcc11
COLLECT_LTO_WRAPPER=/usr/local/libexec/gcc11/gcc/x86_64-portbld-freebsd13
    ↪ .0/11.2.0/lto-wrapper
Target: x86_64-portbld-freebsd13.0
...
...
Thread model: posix
Supported LTO compression algorithms: zlib
gcc version 11.2.0 (FreeBSD Ports Collection)
```

Código 7.6: Comandos y salida de la instalación del paquete de GCC 11

```
$ sh setup.sh
make DIR_GCC_PLUGIN_HEADER='/usr/local/lib/gcc11/gcc/x86_64-portbld-freebsd13
    ↪ .0/11.2.0/plugin/include'
g++11 -shared -O2 -pipe -fPIC -fno-rtti -O2 -I/usr/src/sys -I/usr/local/lib/gcc11
    ↪ /gcc/x86_64-portbld-freebsd13.0/11.2.0/plugin/include
    ↪ insertPayload_GCCPlugin.c -o InsertPayload_GCCPlugin.so
```

Código 7.7: Comando y salida reducida de la ejecución del *script* `setup.sh` [147] para la compilación del *plugin* para GCC: `InsertPayload_GCCPlugin`

7.4. Obtención del ejecutable con metadatos insertados

Una vez compilados y obtenidos ambos *plugins*, el paso siguiente es utilizarlos para obtener el ejecutable ELF final.

El programa de usuario será compilado utilizando la *suite* de compilación elegida por el usuario, junto con el *plugin* para dicha *suite*.

Se toma entonces el código fuente principal y resto de archivos fuente que conformarán el programa que el usuario desea compilar, se invoca el uso del *plugin* como librería compartida y se pasan como argumentos a este último, los archivos que se desean insertar.

El orden de los archivos como argumentos es el mismo orden que tendrán dentro del ejecutable.

En este caso, los archivos involucrados son:

- `prueba.c`: código fuente del programa de usuario, encontrado dentro del directorio `test_src`.
- `decl_payA.txt` y `decl_payB.txt`: archivos con datos a insertar dentro del ejecutable final, ubicados en `test_dataToInsert`.
- `InsertPayload_Plugin` y `InsertPayload_GCCPlugin`: librerías dinámicas que representan a los *plugins* para CLANG y GCC, correspondientemente.

7.4.1. Compilación de código de usuario y uso de *plugin* para CLANG

El *script* `test.sh` [148] es provisto al usuario para facilitar la compilación de su código y pasaje de archivos a ser insertos mediante el *plugin*.

```
$ sh test.sh test/src/test.c test/dataToInsert/decl_payA.txt test/dataToInsert/
    ↪ decl_payB.txt

User program source code: test/src/test.c

Plugin argument - 2: test/dataToInsert/decl_payA.txt
Plugin argument - 3: test/dataToInsert/decl_payB.txt

Count of arguments: 3

clang11 -Xclang -load -Xclang ./build/libInsertPayload_Plugin.so  -Xclang -plugin
    ↪  -Xclang InsertPayload_Plugin   -Xclang -plugin-arg-InsertPayload_Plugin -
    ↪ Xclang test/dataToInsert/decl_payA.txt -Xclang -plugin-arg-
    ↪ InsertPayload_Plugin -Xclang test/dataToInsert/decl_payB.txt  -c test/src/
    ↪ test.c -I/usr/src/sys -Isrc  -Wno-implicit-int

Args[0] = test/dataToInsert/decl_payA.txt
Args[1] = test/dataToInsert/decl_payB.txt

Output file created - /tmp/metadata_test.c

clang11 -o metadataTestProgram /tmp/metadata_test.c -Isrc  -I/usr/src/sys

rm /tmp/metadata_test.c
```

Código 7.8: Comando de ejecución y salida del *script* `test.sh` [148] pasando a `decl_payA.txt` [140] y `decl_payB.txt` [142] como argumentos al *plugin* de CLANG. breaklines

Dicho *script* debe recibir los siguientes argumentos al ser ejecutado:

- Primer argumento: Nombre del programa de usuario a ejecutar. NOTA: Recordar que el primer argumento de este *script* debe ser el código fuente a compilar.
- Resto de los argumentos: Nombre del o los archivos a insertar en el ejecutable del programa de usuario. NOTA: La cantidad de archivos a insertar debe ser como máximo 4 archivos de tamaño menor o igual a 2048 *bytes*.

Para poder simular un caso similar a los encontrados en la realidad, en este ejemplo, el archivo fuente `.c` se encuentra en el subdirectorio `src` del directorio actual. El *script* puede ser modificado fácilmente para modificar este *path*.

La ejecución del *script* `test.sh` con los dos archivos de texto, `decl_payA.txt` y `decl_payB.txt`, mencionados anteriormente, se observa en el Código 7.8.

El resultado de la ejecución del *script*, expuesto en el Código 7.8, es el ejecutable ELF `metadataTestProgram`, presente en el directorio actual.

Al ejecutar la herramienta `readelf` [149] con el *flag* `-S` con el objetivo de listar las secciones del ejecutable ELF, se comprueba que se ha creado una sección `.metadata` en la estructura del ejecutable. La salida de dicho comando se observa en el Código 7.9

```
$ readelf -S metadataTestProgram
There are 38 section headers, starting at offset 0x42a0:
Section Headers:
[Nr] Name      Type     Address   Offset  Size  EntSize  Flags Link Info Align
[ 0]           NULL     00000000  0000    0000  0        0     0    0
..
..
[23] .metadata PROGBITS 00203b20  0b20    1050  0        WA    0    0    16
```

Código 7.9: Salida del comando `readelf` con secciones del ejecutable `metadataTestProgram`

7.4.2. Compilación de código de usuario y uso de *plugin* para GCC

El *script* `test.sh` [150] permite un manejo transparente y fácil para la compilación del código de usuario y pasaje de archivos para ser insertados en el ejecutable final utilizando el *plugin* de GCC: `InsertPayload_GCCPlugin`.

Los argumentos que el *script* debe recibir son:

- Primer argumento: Nombre del programa de usuario a ejecutar. NOTA: Recordar que el primer argumento del *script* debe ser el código fuente a compilar.
- Resto de los argumentos: Nombre del o los archivos a insertar en el ejecutable del programa de usuario. NOTA: La cantidad de archivos a insertar debe ser como máximo 4 archivos de tamaño menor o igual a 2048 *bytes*.

El *script* deja implícito que el código del programa de usuario está en el mismo directorio en que se encuentra el *script*. Si es que el usuario desea colocar su código fuente en otro *path*, el *script* `test.sh` puede ser fácilmente modificado para indicar el *path* en donde buscar dicho archivo fuente.

El *script* `test.sh` [150] se ejecuta pasando como argumentos los siguientes archivos del directorio `test` en el orden listado:

1. `prueba.c` [144]: código fuente del programa de usuario, dentro del directorio `test_src`.
2. `decl_payA.txt` [141]: primer archivo a insertar, ubicado en `test_dataToInsert`.
3. `decl_payB.txt` [143]: segundo archivo a insertar. Archivo con datos a insertar dentro del ejecutable final, localizado en `test_dataToInsert`.

El Código 7.10 muestra el comando de compilación y la salida del *script* al pasarle como argumentos los archivos mencionados.

El resultado de la ejecución del *script*, mostrado en el Código 7.10, es el ejecutable ELF `metadataTestProgramGCC`.

El Código 7.11 presenta la salida de la ejecución de la herramienta `readelf` [149] con el *flag* `-S`, corroborando la existencia de la sección `.metadata` en el ejecutable obtenido.

```
$ sh test.sh test/src/test.c test/dataToInsert/decl_payA.txt test/dataToInsert/
    ↪ decl_payB.txt

User program source code: test/src/test.c

Plugin argument - 2: test/dataToInsert/decl_payA.txt
Plugin argument - 3: test/dataToInsert/decl_payB.txt

Count of arguments: 3

make test PLUGIN_ARGS='-fplugin-arg-InsertPayload_GCCPlugin-1=test/dataToInsert/
    ↪ decl_payA.txt -fplugin-arg-InsertPayload_GCCPlugin-1=test/dataToInsert/
    ↪ decl_payB.txt '
gcc11 -o metadataTestProgramGCC test/src/prueba.c -fplugin=./
    ↪ InsertPayload_GCCPlugin.so -fplugin-arg-InsertPayload_GCCPlugin-1=test/
    ↪ dataToInsert/decl_payA.txt -fplugin-arg-InsertPayload_GCCPlugin-1=test/
    ↪ dataToInsert/decl_payB.txt  -I/usr/src/sys

Number of arguments of this plugin: 2
Arg [0]: - Key: [1] - Value: [test/dataToInsert/decl_payA.txt]
Arg [1]: - Key: [1] - Value: [test/dataToInsert/decl_payB.txt]
```

Código 7.10: Comando de ejecución y salida del *script* `test.sh` [150] pasando a `decl_payA.txt` [141] y `decl_payB.txt` [143] como argumentos al *plugin* de GCC

```
$ readelf -S metadataTestProgram
There are 38 section headers, starting at offset 0x42a0:
Section Headers:
[Nr] Name      Type     Address   Offset   Size   EntSize  Flags Link Info Align
[ 0]           NULL     00000000  0000     0000   0        0     0    0
..
..
[23] .metadata PROGBITS 00600be0  0be0     1060   0        WA    0    0    32
```

Código 7.11: Salida del comando `readelf` con secciones del ejecutable `metadataTestProgram`

```
$ ./metadataTestProgram

******** codeinsertion finished ********
```

Código 7.12: Ejecución y salida en terminal del programa `metadataTestProgram` compilado con CLANG/LLVM

```
$ ./metadataTestProgramGCC

******** codeinsertion finished ********
```

Código 7.13: Ejecución y salida en terminal del programa `metadataTestProgramGCC` compilado con GCC

7.5. Ejecución del programa

Una vez obtenidos el ejecutable del programa de usuario con los datos insertados, el siguiente paso es la ejecución propiamente dicha de los mismos.

El Código 7.12 muestra la ejecución y salida por terminal del programa `metadataTestProgram`, compilado con CLANG/LLVM. Mientras que el Código 7.13 presenta la ejecución y mensajes generados en la terminal del programa `metadataTestProgramGCC`, compilado con GCC.

7.6. Lectura de *log* del *kernel*

Al ejecutarse el programa del usuario con los datos insertados, este es cargado en el sistema operativo. Durante su procesamiento en espacio de *kernel*, el ejecutable es *parseado*. Al detectarse la existencia de la sección `.metadata` en el ELF, se inicia la extracción y decodificación de los datos insertados en dicha sección. Finalmente, los datos recuperados y decodificados se imprimen en los *logs* o registros del sistema.

Utilizando la herramienta `dmesg` para observar el buffer de mensajes del sistema, se verifica la correcta inserción y decodificación de los datos insertados en el ejecutable.

El Código 7.14 muestra los datos recuperados del ejecutable del programa `metadataTestProgram`, compilado con CLANG/LLVM. Por otra parte, el Código 7.15 presenta los datos extraídos y decodificados del ejecutable del programa `metadataTestProgramGCC`, compilado con GCC.

```
Process: /usr/home/freebsd/CLANGPlugin/metadataTestProgram -- Section found --
    -- Section .metadata found. ELF offset: 0xb20 - Size: 4176 bytes--
Process: /usr/home/freebsd/CLANGPlugin/metadataTestProgram contains metadata. --
    ↪ *** METADATA: 0xffffe0096ca8000 -- PAYLOAD SIZE: 4176 -- RETURNED FLAG: 1
    ↪ ***
Metadata_Hdr = metadata_header_decod //// m_number_payloads: 2 - ph_size: 16
Payload_Hdr = payload_header_decod[0] //// address: 0xffffe0096ca9030 -
    ↪ p_function_number: 1 - ph_size: 2064
    Payload_Binary = payloadBinary_decod //// bin_data_size: 24 -
    ↪ charArray_bin_size: 48
    bin_data - Canonical hex:
                        50 61 79 6C 6F 61 64 5F 41 2F 32 2F 34 2F 63 2F 69 6E 74
    ↪ 5F 73 69 7A 65
    bin_data - ASCII read:
                        Payload_A/2/4/c/int_size
Payload_Hdr = payload_header_decod[1] //// address: 0xffffe0096ca9040 -
    ↪ p_function_number: 1 - ph_size: 2064
    Payload_Binary = payloadBinary_decod //// bin_data_size: 26 -
    ↪ charArray_bin_size: 52
    bin_data - Canonical hex:
                        50 61 79 6C 6F 61 64 5F 42 2F 31 36 2F 7A 2F 77 2F 63 68
    ↪ 61 72 5F 73 69 7A 65
    bin_data - ASCII read:
                        Payload_B/16/z/w/char_size
```

Código 7.14: *Logs* de sistema mostrando datos insertados en ejecutable `metadataTestProgram` compilado con CLANG/LLVM.

```
Process: /usr/home/freebsd/GCCPlugin/metadataTestProgramGCC -- Section found --
    -- Section .metadata found. ELF offset: 0xbe0 - Size: 4192 bytes--
Process: /usr/home/freebsd/GCCPlugin/metadataTestProgramGCC contains metadata. --
    ↪  *** METADATA: 0xffffe0096fac000 -- PAYLOAD SIZE: 4192 -- RETURNED FLAG: 1
    ↪  ***
Metadata_Hdr = metadata_header_decod //// m_number_payloads: 2 - ph_size: 16
Payload_Hdr = payload_header_decod[0] //// address: 0xffffe0096fad040 -
    ↪ p_function_number: 1 - ph_size: 2064
    Payload_Binary = payloadBinary_decod //// bin_data_size: 24 -
    ↪ charArray_bin_size: 48
    bin_data - Canonical hex:
                        50 61 79 6C 6F 61 64 5F 41 2F 32 2F 34 2F 63 2F 69 6E 74
    ↪ 5F 73 69 7A 65
    bin_data - ASCII read:
                        Payload_A/2/4/c/int_size
Payload_Hdr = payload_header_decod[1] //// address: 0xffffe0096fad050 -
    ↪ p_function_number: 1 - ph_size: 2064
    Payload_Binary = payloadBinary_decod //// bin_data_size: 26 -
    ↪ charArray_bin_size: 52
    bin_data - Canonical hex:
                        50 61 79 6C 6F 61 64 5F 42 2F 31 36 2F 7A 2F 77 2F 63 68
    ↪ 61 72 5F 73 69 7A 65
    bin_data - ASCII read:
                        Payload_B/16/z/w/char_size
```

Código 7.15: *Logs* de sistema con datos insertados en ejecutable `metadataTestProgramGCC` compilado con GCC.

Parte III

Conclusiones

Conclusiones

El trabajo realizado con motivos del presente proyecto integrador ha resultado en la obtención del modelo prototipo del protocolo de comunicación entre espacio de usuario y espacio del *kernel* expuesto en el objetivo del proyecto.

Se presenta al programador, entonces, un protocolo que permita el envío de datos, en forma de metadatos dentro del ejecutable del programa de usuario, para luego utilizarlo para *setear* o modificar algún parámetro del *kernel*.

Después de sucesivas iteraciones, se ha podido lograr un nivel considerable de abstracción en cuanto a los distintos tipos de datos que se pueden insertar dentro de los ejecutables, así como se ha logrado desarrollar herramientas de fácil uso para que el usuario pueda realizar la inserción solicitada.

Se ha partido desde la investigación de las etapas del proceso de compilación y cómo estas se reflejan en las *suites* de compilación: GCC y CLANG/LLVM; así como conocer el detalle de cómo se conforma la estructura de los archivos ejecutables ELF. El segundo paso importante ha sido utilizar directivas de compilación que provee el lenguaje C para insertar datos y verlos reflejados en el ejecutable. El tercer hito ha sido la formulación de los parches de *kernel* para las distintas versiones del sistema operativo FreeBSD, que permiten detectar, extraer, decodificar e imprimir los datos insertados por el usuario. Finalmente, el cuarto paso relevante ha sido encontrar la manera de insertar datos de forma abstracta y sin complicaciones extra en su decodificación. El último y quinto hito ha sido el desarrollo de *plugins* y *scripts*, que permitan el uso de las herramientas y realización de la inserción de datos de forma transparente para el usuario.

Las principales limitaciones encontradas han sido: la cantidad de archivos a insertar y el tamaño de los datos a insertar. Dado que se ha buscado obtener una versión prototipo, se ha definido una cantidad y tamaño reducidos con respecto a los datos posibles a insertar.

El trabajo actual nos ha provisto de gran conocimiento en las áreas de compiladores, desarrollo de sistemas operativos y prácticas de ingeniería de software; descubriendo un área de investigación y desarrollo interesante y con muchas aristas para explorar y mejorar.

Trabajos futuros

Futuros desarrollos del mecanismo diseñado deberían atacar las limitaciones de la versión prototipo. Los desarrolladores deberían revertir los límites impuestos en cantidad y tamaños de datos a insertar, los cuales son reflejados como *payloads*. Nuevos trabajos deberían sumar, a los parches de *kernel* de FreeBSD, funciones que permitan modificar o *setear* variables de *kernel* para darle funcionalidad real. Por último, el desarrollo podría extenderse a otros sistemas operativos, teniendo como base lo realizado con el parche generado para el *kernel* de FreeBSD.

Bibliografía

[1] I. Sommerville, *Software Engineering*, English. Pearson, 2016.

[2] R. M. S. Brian J. Gough, *An Introduction to GCC*. Network Theory Ltd., 2004, ISBN: 978-0-95-416179-8; 0-9541-6179-3.

[3] S. S. Muchnick, *Advanced Compiler Design and Implementation 1*, illustrated. Morgan Kaufmann, 1997, ISBN: 1-5586-0320-4; 978-1-55-860320-2.

[4] D. Andriesse, *Practical Binary Analysis. Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly*. No Starch Press, 2018, ISBN: 978-1-59-327912-7; 1-5932-7912-4.

[5] L. T. Keith Cooper, *Engineering a Compiler*, 2.ª ed. Morgan Kaufmann, 2011, ISBN: 0-1208-8478-X; 978-0-12-088478-0.

[6] R. S. Alfred Aho Monica Lam y J. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson/Addison Wesley, 2007, ISBN: 978-0-32-148681-3.

[7] P. H. Dave y H. B. Dave, *Compilers: Principles and Practice*, English. Pearson India, 2012, ISBN: 978-8-13-177611-7.

[8] B. C. Lopes y R. Auler, *Getting Started with LLVM Core Libraries: Get to Grips with LLVM Essentials and Use the Core Libraries to Build Advanced Tools.* English. Packt Publishing, 2014.

[9] T. I. S. (Committee, *Executable and Linking Format Specification, Version 1.2*. 1995.

[10] W. Von Hagen, *The definitive guide to GCC*, 2nd ed. Berkeley, CA: Apress, 2006, ISBN: 978-1-59-059585-5.

[11] M.-Y. Hsu, *LLVM Techniques, Tips, and Best Practices Clang and Middle-End Libraries*. Packt Publishing Pvt Ltd, 2021, ISBN: 978-1-83-882495-2; 1-8388-2495-2.

[12] K. Sunitha, *Compiler Construction*. Pearson India, 2013, ISBN: 9-3325-0029-0; 978-9-33-250029-7.

Referencias

[13] R. Shapiro, S. Bratus y S. Smith, *Weird machines in ELF: a spotlight on the underappreciated metadata*, ago. de 2013.

[14] D. Duvarney, S. Bhatkar y V. Venkatakrishnan, «SELF: A transparent security extension for ELF binaries,» *Proceedings New Security Paradigms Workshop*, nov. de 2003. DOI: 10.1145/986655.986661.

[15] J. Cancinos y J. González, *Compiladores y Ejecutables - Informe de trabajo para Práctica Supervisada.* [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/Informes/InformeEntregaPPS/maestro-informeEntregaPPS.pdf .

[16] F. y. N. .-. U. N. d. C. Facultad de Ciencias Exactas, *Laboratorio de Arquitectura de Computadoras*. [En línea]. Disponible en: https://fcefyn.unc.edu.ar/facultad/secretarias/academica/escuelas/ingenieria-computacion/lac/.

[17] N. Papp y T. Turina, *Modelado del planificador a corto plazo con redes de Petri*, 2019.

[18] J. Cancinos y J. Gonzalez, *Programa de inserción de metadatos en ejecutable ELF*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/LecturaELF/ELF/code_macros.c.

[19] Cancinos, Jose and Gonzalez, Julian, *Programa de lectura de metadata insertada en ejecutable ELF*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/LecturaELF/lecturaELF.c.

[20] J. Cancinos y J. Gonzalez, *Repositorio de trabajo - Práctica Supervisada;* [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores.

[21] Cancinos, Jose and Gonzalez, Julian, *Insercion de codigo usando Inline ASM*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/ELF/GCC/InlineASM/simpleEjInline.c.

[22] Cancinos, Jose and Gonzalez, Julian, *Insercion de codigo usando comando objcopy*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/ELF/GCC/AddComment_OBJCOPY/Makefile.

[23] Cancinos, Jose and Gonzalez, Julian, *Insercion de codigo usando atributos de C*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/ELF/GCC/AddComment_AtributeC/simpleEjATR.c.

[24] C. Hock-Chuan, *GCC and Make - Compiling, Linking and Building C/C++ Applications*. [En línea]. Disponible en: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/gcc_make.html.

[25] R. Eklind, *LLVM IR and Go*. [En línea]. Disponible en: https://blog.gopheracademy.com/advent-2018/llvm-ir-and-go/.

[26] LLVM Project, *Código Preprocessor.cpp en repositorio del LLVM Project*. [En línea]. Disponible en: https://github.com/llvm/llvm-project/blob/main/clang/lib/Lex/Preprocessor.cpp.

[27] LLVM Project, *Código Lexer.cpp en repositorio del LLVM Project*. [En línea]. Disponible en: https://github.com/llvm/llvm-project/blob/main/clang/lib/Lex/Lexer.cpp.

[28] LLVM Project, *Código Parser.cpp en repositorio del LLVM Project*. [En línea]. Disponible en: https://github.com/llvm/llvm-project/blob/main/clang/lib/Parse/Parser.cpp.

[29] LLVM Project, *Código Sema.cpp en repositorio del LLVM Project*. [En línea]. Disponible en: https://github.com/llvm/llvm-project/blob/main/clang/lib/Sema/Sema.cpp.

[30] *Drysdale, David. "How programs get run: ELF binaries." 2015*, en-us. [En línea]. Disponible en: https://lwn.net/Articles/631631/.

[31] T. F. Project, *execve(2) - BSD*. [En línea]. Disponible en: https://www.freebsd.org/cgi/man.cgi?query=execve&sektion=2.

[32] OSDev.org, *ELF*. [En línea]. Disponible en: https://wiki.osdev.org/ELF.

[33] T. F. Project, *ld-elf.so*. [En línea]. Disponible en: https://www.freebsd.org/cgi/man.cgi?query=ld-elf.so.

[34] J. Cancinos y J. Gonzalez, *Versión inicial de inserción de datos utilizando macro seteada desde terminal*. [En línea]. Disponible en: https://bit.ly/3wntEG0.

[35] Cancinos, Jose and Gonzalez, Julian, *Módulo de kernel para leer metadatos insertados en ELF para FreeBSD 11*. [En línea]. Disponible en: https://bit.ly/3FUaDxP.

[36] Cancinos, Jose and Gonzalez, Julian, *Tags del repositorio principal de trabajo*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/tags.

[37] Cancinos, Jose and Gonzalez, Julian, *Versión inicial de código de detección y extracción de metadata en kernel FreeBSD 11*. [En línea]. Disponible en: https://bit.ly/3liU3hF.

[38] Cancinos, Jose and Gonzalez, Julian, *Versión inicial del archivo cabecera para código metadata ELF reader en kernel FreeBSD 11*. [En línea]. Disponible en: https://bit.ly/3wTdT8T.

[39] Cancinos, Jose and Gonzalez, Julian, *Código activador de ELF, modificado, en kernel FreeBSD 11*. [En línea]. Disponible en: https://bit.ly/3PrUPqB.

[40] Cancinos, Jose and Gonzalez, Julian, *Header proc.h con campos metadata en kernel FreeBSD 11*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/df69fc3582f43e684f2b513634729b9a7b183707/AnalisisCodigo/proc.h.

[41] Cancinos, Jose and Gonzalez, Julian, *Código fuente de section_insertion.c*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/67c190fcee74be73c5b692a941e537d3ba3c548b/MacrosLibraryELFCLANG/Defined/src/section_insertion.c.

[42] Cancinos, Jose and Gonzalez, Julian, *Código fuente de code_macros.c*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/67c190fcee74be73c5b692a941e537d3ba3c548b/MacrosLibraryELFCLANG/Defined/src/code_macros.c.

[43] Cancinos, Jose and Gonzalez, Julian, *Archivo cabecera metadata_elf.h*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/0c0b5489cc631646d7762b2e1f5f7e86f1c28f49/PI/ELFEncoder/include/metadata_elf.h.

[44] Cancinos, Jose and Gonzalez, Julian, *Código de programa principal main.c para probar inserción múltiple codificada*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/0c0b5489cc631646d7762b2e1f5f7e86f1c28f49/PI/ELFEncoder/src/main.c.

[45] Cancinos, Jose and Gonzalez, Julian, *Header section_insertion.h que contiene definición de estructura Payload y macro INSERT_METADATA*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/afa734aece2591319a275ccf4f8181aec4a9e8ca/PI/ELFEncoderDecoderTest/include/section_insertion.h.

[46] Cancinos, Jose and Gonzalez, Julian, *Código main.c para inserción de estructuras Payload y lectura de las mismas en espacio de usuario*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/afa734aece2591319a275ccf4f8181aec4a9e8ca/PI/ELFEncoderDecoderTest/src/main.c.

[47] Cancinos, Jose and Gonzalez, Julian, *Repositorio de trabajo para FreeBSD 11*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD.

[48] Cancinos, Jose and Gonzalez, Julian, *Archivo cabecera metadata_elf_reader.h con Payload struct definida*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/f2b6360ce791f370e995af07fa8cfe878996eda9/sys/sys/metadata_elf_reader.h.

[49] Cancinos, Jose and Gonzalez, Julian, *Código metadata_elf_reader.c con decodificación de structs Payload*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/f2b6360ce791f370e995af07fa8cfe878996eda9/sys/kern/metadata_elf_reader.c.

[50] Cancinos, Jose and Gonzalez, Julian, *Código imgact_elf en iteración 6*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/f2b6360ce791f370e995af07fa8cfe878996eda9/sys/kern/imgact_elf.c.

[51] Cancinos, Jose and Gonzalez, Julian, *Salida de logs de kernel mostrando decodificación de struct Payload insertada*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/93145d658f1a865ed3e257ff1967eacd38e9e21b/AnalisisCodigo/dmesglog.txt.

[52] Cancinos, Jose and Gonzalez, Julian, *Repositorio principal de trabajo*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC.

[53] Cancinos, Jose and Gonzalez, Julian, *Tags del repositorio de FreeBSD 11*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/tags.

[54] Cancinos, Jose and Gonzalez, Julian, *Header metadata_payloads.h*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/7ad26a9c9ac6798ef48d88f6ebea0aaf75f04024/sys/sys/metadata_payloads.h.

[55] Cancinos, Jose and Gonzalez, Julian, *Inserción de datos en forma de estructuras metadata. Código main.c*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/d065d696f6857cbf5506493827b98080141e1fb7/ModKernelLectorELFBSD/LecturaELFBSD/FreeBSDELF/src/main.c.

[56] Cancinos, Jose and Gonzalez, Julian, *Decodificación de datos insertados en forma de estructuras metadata. Código metadata_elf_reader.c*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/7ad26a9c9ac6798ef48d88f6ebea0aaf75f04024/sys/kern/metadata_elf_reader.c.

[57] Cancinos, Jose and Gonzalez, Julian, *Salida de logs de kernel mostrando decodificación de struct Payload_A y Payload_B*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/d065d696f6857cbf5506493827b98080141e1fb7/AnalisisCodigo/dmesglog.txt.

[58] Cancinos, Jose and Gonzalez, Julian, *Código metadata_elf_reader.c con arreglo de funciones de decodificación*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/48af0362d50a0b0447904fde30bb950db3cdcb13/sys/kern/metadata_elf_reader.c.

[59] Cancinos, Jose and Gonzalez, Julian, *Header metadata_elf_reader.h con declaración de funciones de decodificación de los distintos payloads*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/48af0362d50a0b0447904fde30bb950db3cdcb13/sys/sys/metadata_elf_reader.h.

[60] Cancinos, Jose and Gonzalez, Julian, *Salida de logs de kernel mostrando decodificación de metadatos de tipo Payload_A y Payload_B*. [En línea]. Disponible en: https://github.co

m/josemacan/PPSCompiladoresUNC/blob/2e86974086d3267c63dd526db35d6ff511ac889d/AnalisisCodigo/dmesglog.txt.

[61] Cancinos, Jose and Gonzalez, Julian, *Archivo con los datos a insertar usando plugin de GCC*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/9a930f2646a057b81fd22b879a565376011739b8/PI/MetadataGCCPlugin/PluginGCCFreeBSD/declarations.txt.

[62] Cancinos, Jose and Gonzalez, Julian, *Makefile para compilación y uso del plugin de GCC*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/9a930f2646a057b81fd22b879a565376011739b8/PI/MetadataGCCPlugin/PluginGCCFreeBSD/Makefile.

[63] Cancinos, Jose and Gonzalez, Julian, *Archivo cabecera section_insertion.h con definiciones de estructuras metadata*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/9a930f2646a057b81fd22b879a565376011739b8/PI/MetadataGCCPlugin/PluginGCCFreeBSD/section_insertion.h.

[64] Cancinos, Jose and Gonzalez, Julian, *Código de usuario con variable global a ser reemplazada por líneas de inserción mediante plugin de GCC*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/9a930f2646a057b81fd22b879a565376011739b8/PI/MetadataGCCPlugin/PluginGCCFreeBSD/prueba.c.

[65] The GCC Project, *Plugin API. GCC Internals*. [En línea]. Disponible en: https://gcc.gnu.org/onlinedocs/gccint/Plugin-API.html.

[66] ISO/IEC, *C99. Draft version*. [En línea]. Disponible en: https://www.open-std.org/jtc1/sc22/WG14/www/docs/n1256.pdf.

[67] The FreeBSD Project, *FreeBSD General Commands Manual: Objcopy*. [En línea]. Disponible en: https://www.freebsd.org/cgi/man.cgi?query=objcopy&apropos=0.

[68] Cancinos, Jose and Gonzalez, Julian, *Código main.c con payloads y estructuras metadata*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/27d822cb34ab7406df33048ef9da65f029fd5b68/PI/ExtractSectionCombineELF/OriginalMetadataInsert/src/main.c.

[69] Cancinos, Jose and Gonzalez, Julian, *Makefile con uso de objcopy para insertar sección*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/27d822cb34ab7406df33048ef9da65f029fd5b68/PI/ExtractSectionCombineELF/Makefile.

[70] Cancinos, Jose and Gonzalez, Julian, *Log de kernel mostrando inserción exitosa al usar objcopy*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/27d822cb34ab7406df33048ef9da65f029fd5b68/AnalisisCodigo/dmesglog.txt.

[71] The Clang Team, *Clang Plugins. Clang 15.0.0git documentation*. [En línea]. Disponible en: https://clang.llvm.org/docs/ClangPlugins.html.

[72] Cancinos, Jose and Gonzalez, Julian, *Código fuente RenameMacro.cpp de la primera versión del plugin de CLANG para inserción de datos en ELF.* [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/1607fb354f337e1197fe1879395f3f7054281e43/PI/pluginClang/RenameMacro.cpp.

[73] The Clang Team, *clang RecursiveASTVisitor Derived Class Template Reference*. [En línea]. Disponible en: https://clang.llvm.org/doxygen/classclang_1_1RecursiveASTVisitor.html.

[74] Cancinos, Jose and Gonzalez, Julian, *Archivo CMakeLists.txt para compilar plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/1607fb354f337e1197fe1879395f3f7054281e43/PI/pluginClang/CMakeLists.txt.

[75] Cancinos, Jose and Gonzalez, Julian, *Código test.c de programa de prueba con función a ser modificada por plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/1607fb354f337e1197fe1879395f3f7054281e43/PI/pluginClang/src/test.c.

[76] Cancinos, Jose and Gonzalez, Julian, *Header metadata_elf_reader.h en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/5b0cf0ee3713027fce92d51d738baa75053b33e2/sys/sys/metadata_elf_reader.h.

[77] Cancinos, Jose and Gonzalez, Julian, *Header metadata_elf_reader.h en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/4ef0a47b8bcc83c5ab618d433693fbe7b05cb156/sys/sys/metadata_elf_reader.h.

[78] Cancinos, Jose and Gonzalez, Julian, *Código fuente metadata_elf_reader.c en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/5b0cf0ee3713027fce92d51d738baa75053b33e2/sys/kern/metadata_elf_reader.c.

[79] Cancinos, Jose and Gonzalez, Julian, *Código fuente metadata_elf_reader.c en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/4ef0a47b8bcc83c5ab618d433693fbe7b05cb156/sys/kern/metadata_elf_reader.c.

[80] Cancinos, Jose and Gonzalez, Julian, *Header metadata_payloads.h en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/5b0cf0ee3713027fce92d51d738baa75053b33e2/sys/sys/metadata_payloads.h.

[81] Cancinos, Jose and Gonzalez, Julian, *Header metadata_payloads.h en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/4ef0a47b8bcc83c5ab618d433693fbe7b05cb156/sys/sys/metadata_payloads.h.

[82] Cancinos, Jose and Gonzalez, Julian, *Header proc.h en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/5b0cf0ee3713027fce92d51d738baa75053b33e2/sys/sys/proc.h.

[83] Cancinos, Jose and Gonzalez, Julian, *Header proc.h en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/4ef0a47b8bcc83c5ab618d433693fbe7b05cb156/sys/sys/proc.h.

[84] Cancinos, Jose and Gonzalez, Julian, *Código fuente imgact_elf.c en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/5b0cf0ee3713027fce92d51d738baa75053b33e2/sys/kern/imgact_elf.c.

[85] Cancinos, Jose and Gonzalez, Julian, *Código fuente imgact_elf.c en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/4ef0a47b8bcc83c5ab618d433693fbe7b05cb156/sys/kern/imgact_elf.c.

[86] The FreeBSD Project, *vn_lock(9)*. [En línea]. Disponible en: https://www.freebsd.org/cgi/man.cgi?query=vn_lock&sektion=9&format=html.

[87] Cancinos, Jose and Gonzalez, Julian, *Código fuente metadata_elf_reader.c en FreeBSD 11*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/2fa96698cdc38a4e56b2a8054cee9c6def0ec1fc/sys/kern/metadata_elf_reader.c.

[88] Cancinos, Jose and Gonzalez, Julian, *Tags del repositorio de FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/tags.

[89] Cancinos, Jose and Gonzalez, Julian, *Repositorio de trabajo para FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3.

[90] Cancinos, Jose and Gonzalez, Julian, *Tags del repositorio de FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/tags.

[91] Cancinos, Jose and Gonzalez, Julian, *Repositorio de trabajo para FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13.

[92] Cancinos, Jose and Gonzalez, Julian, *Código chararray_binRuntimeEncDec.c para convertir archivos en binario para uso en inserción*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/cddd29c833ddb4d92a536162f20f1ccf80cd59f5/PI/BinaryRawMetadata/BinaryRunTimeEncoderDecoder/src/chararray_binRuntimeEncDec.c.

[93] Cancinos, Jose and Gonzalez, Julian, *Archivo decl_payA.txt a ser convertido en binario para posterior inserción en ejecutable ELF*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/cddd29c833ddb4d92a536162f20f1ccf80cd59f5/PI/BinaryRawMetadata/BinaryRunTimeEncoderDecoder/decl_payA.txt.

[94] The FreeBSD Project, *Hexdump(1)*. [En línea]. Disponible en: https://www.freebsd.org/cgi/man.cgi?query=hexdump.

[95] Cancinos, Jose and Gonzalez, Julian, *Header section_insertion_rawbin.h con definición de estructura Payload_Binary*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/0de9949d8d66a30c18278ae1bc503989c9b5f6a4/PI/BinaryRawMetadata/MultipleRawBinaryPluginGCCFreeBSD/section_insertion_rawbin.h.

[96] Cancinos, Jose and Gonzalez, Julian, *Makefile de plugin GCC con múltiples argumentos binarios*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/0de9949d8d66a30c18278ae1bc503989c9b5f6a4/PI/BinaryRawMetadata/MultipleRawBinaryPluginGCCFreeBSD/Makefile.

[97] Cancinos, Jose and Gonzalez, Julian, *Código pluginsegundo.c del plugin para GCC con inserción de múltiples estructuras Payload_Binary*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/0de9949d8d66a30c18278ae1bc503989c9b5f6a4/PI/BinaryRawMetadata/MultipleRawBinaryPluginGCCFreeBSD/pluginsegundo.c.

[98] Cancinos, Jose and Gonzalez, Julian, *Código prueba.c de programa de usuario*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/0de9949d8d66a30c18278ae1bc503989c9b5f6a4/PI/BinaryRawMetadata/MultipleRawBinaryPluginGCCFreeBSD/prueba.c.

[99] Cancinos, Jose and Gonzalez, Julian, *Código fuente metadata_elf_reader.c con soporte de struct Payload_Binary en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/7018849d8686edbe09aac36fac1226610eac5d03/sys/kern/metadata_elf_reader.c.

[100] Cancinos, Jose and Gonzalez, Julian, *Código fuente metadata_elf_reader.c con soporte de struct Payload_Binary en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/6d44c695cc66a5c67a2e42ee194c5ba572a65f06/sys/kern/metadata_elf_reader.c.

[101] Cancinos, Jose and Gonzalez, Julian, *Código fuente metadata_elf_reader.c con soporte de struct Payload_Binary en FreeBSD 11*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/1e68e91c9f142a290358cce61ae70f1698b75f2d/sys/kern/metadata_elf_reader.c.

[102] Cancinos, Jose and Gonzalez, Julian, *Header metadata_elf_reader.h con soporte de struct Payload_Binary en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/7018849d8686edbe09aac36fac1226610eac5d03/sys/sys/metadata_elf_reader.h.

[103] Cancinos, Jose and Gonzalez, Julian, *Header metadata_elf_reader.h con soporte de struct Payload_Binary en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/6d44c695cc66a5c67a2e42ee194c5ba572a65f06/sys/sys/metadata_elf_reader.h.

[104] Cancinos, Jose and Gonzalez, Julian, *Header metadata_elf_reader.h con soporte de struct Payload_Binary en FreeBSD 11*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/1e68e91c9f142a290358cce61ae70f1698b75f2d/sys/sys/metadata_elf_reader.h.

[105] Cancinos, Jose and Gonzalez, Julian, *Header metadata_payloads.h con definición de struct Payload_Binary en FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/7018849d8686edbe09aac36fac1226610eac5d03/sys/sys/metadata_payloads.h.

[106] Cancinos, Jose and Gonzalez, Julian, *Header metadata_payloads.h con definición de struct Payload_Binary en FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/6d44c695cc66a5c67a2e42ee194c5ba572a65f06/sys/sys/metadata_payloads.h.

[107] Cancinos, Jose and Gonzalez, Julian, *Header metadata_payloads.h con definición de struct Payload_Binary en FreeBSD 11*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/1e68e91c9f142a290358cce61ae70f1698b75f2d/sys/sys/metadata_payloads.h.

[108] Cancinos, Jose and Gonzalez, Julian, *Código prueba.cpp de apertura, procesamiento y formulación de líneas metadata en lenguaje C++*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/11fbfb552a148a5a3ef25199aa433e347e9794bb/PI/BinaryRawMetadata/CPP_PayloadLines/src/prueba.cpp.

[109] Cancinos, José and Gonzalez, Julián, *Salida del convertidor binario para decl_payA.txt*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/140d13db586783b428a05ad168fb4aa7b7e9bc1e/PI/BinaryRawMetadata/CLANGPlugin_PayloadLines/decl_payA.txt_binary.bin.

[110] Cancinos, José and Gonzalez, Julián, *Salida del convertidor binario para decl_payB.txt*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/140d13db586783b428a05ad168fb4aa7b7e9bc1e/PI/BinaryRawMetadata/CLANGPlugin_PayloadLines/decl_payB.txt_binary.bin.

[111] Cancinos, José and Gonzalez, Julián, *Código RenameMacro_Payload.cpp del plugin de CLANG para inserción usando estructuras Payload_Binary*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/140d13db586783b428a05ad168fb4aa7b7e9bc1e/PI/BinaryRawMetadata/CLANGPlugin_PayloadLines/RenameMacro_Payload.cpp.

[112] Cancinos, José and Gonzalez, Julián, *Archivo test.c del programa de usuario con variable __PAYLOAD__ a ser reemplazada.* [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/140d13db586783b428a05ad168fb4aa7b7e9bc1e/PI/BinaryRawMetadata/CLANGPlugin_PayloadLines/src/test.c.

[113] Cancinos, José and Gonzalez, Julián, *Script test.sh para compilar programa de prueba de usuario con plugin de CLANG con dos archivos a insertar*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/140d13db586783b428a05ad168fb4aa7b7e9bc1e/PI/BinaryRawMetadata/CLANGPlugin_PayloadLines/test.sh.

[114] Cancinos, José and Gonzalez, Julián, *Archivo CMakeLists.txt para compilar plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/140d13db586783b428a05ad168fb4aa7b7e9bc1e/PI/BinaryRawMetadata/CLANGPlugin_PayloadLines/CMakeLists.txt.

[115] Cancinos, José and Gonzalez, Julián, *Logs del kernel con resultados de decodificación de programa de usuario modificado por plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/140d13db586783b428a05ad168fb4aa7b7e9bc1e/AnalisisCodigo/dmesglog.txt.

[116] Cancinos, José and Gonzalez, Julián, *Header proc.h sin campo p_metadata en struct proc*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/3ffce1c2e8e58e33864e12aad93b931c14f9f8a4/sys/sys/proc.h.

[117] Cancinos, José and Gonzalez, Julián, *Header metadata_payloads.h con soporte único para payloads tipo Payload_Binary y tamaño máximo de contenido binario incrementado*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/3ffce1c2e8e58e33864e12aad93b931c14f9f8a4/sys/sys/metadata_payloads.h.

[118] Cancinos, José and Gonzalez, Julián, *Header metadata_elf_reader.h con soporte único para payloads tipo Payload_Binary*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/3ffce1c2e8e58e33864e12aad93b931c14f9f8a4/sys/sys/metadata_elf_reader.h.

[119] Cancinos, José and Gonzalez, Julián, *Código metadata_elf_reader.c con soporte único para payloads tipo Payload_Binary*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/3ffce1c2e8e58e33864e12aad93b931c14f9f8a4/sys/kern/metadata_elf_reader.c.

[120] Cancinos, José and Gonzalez, Julián, *Código fuente insertPayload_GCCPlugin.cpp para plugin de GCC*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/GCCPlugin/insertPayload_GCCPlugin.c.

[121] Cancinos, José and Gonzalez, Julián, *Código fuente insertPayload_Plugin.cpp para plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/CLANGPlugin/insertPayload_Plugin.cpp.

[122] Cancinos, José and Gonzalez, Julián, *Script setup.sh para compilación automátizada de plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/CLANGPlugin/setup.sh.

[123] Cancinos, José and Gonzalez, Julián, *Script setup.sh para compilación automátizada de plugin de GCC*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/GCCPlugin/setup.sh.

[124] Cancinos, José and Gonzalez, Julián, *Script test.sh para paso de argumentos y uso de plugin GCC en la compilación de programa de usuario*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/GCCPlugin/test.sh.

[125] Cancinos, José and Gonzalez, Julián, *Script test.sh para paso de argumentos y uso de plugin CLANG en la compilación de programa de usuario*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/CLANGPlugin/test.sh.

[126] Cancinos, José and Gonzalez, Julián, *Archivo decl_payA.txt a insertar mediante uso de plugins*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/CLANGPlugin/decl_payA.txt.

[127] Cancinos, José and Gonzalez, Julián, *Archivo decl_payB.txt a insertar mediante uso de plugins*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladores

UNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/CLANGPlugin/decl_payB.txt.

[128] Cancinos, José and Gonzalez, Julián, *Header metadata_elf_reader.h. Parche para FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/main/sys/sys/metadata_elf_reader.h.

[129] Cancinos, José and Gonzalez, Julián, *Header metadata_elf_reader.h. Parche para FreeBSD*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/main/sys/sys/metadata_elf_reader.h.

[130] Cancinos, José and Gonzalez, Julián, *Header metadata_payloads.h. Parche para FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/main/sys/sys/metadata_payloads.h.

[131] Cancinos, José and Gonzalez, Julián, *Header metadata_payloads.h. Parche para FreeBSD*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/main/sys/sys/metadata_payloads.h.

[132] Cancinos, José and Gonzalez, Julián, *Código metadata_elf_reader.c. Parche para FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/main/sys/kern/metadata_elf_reader.c.

[133] Cancinos, José and Gonzalez, Julián, *Código metadata_elf_reader.c. Parche para FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/main/sys/kern/metadata_elf_reader.c.

[134] Cancinos, José and Gonzalez, Julián, *Header proc.h. Parche para FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/main/sys/sys/proc.h.

[135] Cancinos, José and Gonzalez, Julián, *Header proc.h. Parche para FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/main/sys/sys/proc.h.

[136] Cancinos, José and Gonzalez, Julián, *Código imgact_elf.c. Parche para FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/main/sys/kern/imgact_elf.c.

[137] Cancinos, José and Gonzalez, Julián, *Código imgact_elf.c. Parche para FreeBSD 12.3*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/main/sys/kern/imgact_elf.c.

[138] Cancinos, José and Gonzalez, Julián, *Archivo de configuración de kernel files. Parche para FreeBSD 13*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng13/blob/main/sys/conf/files.

[139] Cancinos, José and Gonzalez, Julián, *Archivo de configuración de kernel files. Parche para FreeBSD*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD_releng12.3/blob/main/sys/conf/files.

[140] Cancinos, José and Gonzalez, Julián, *Archivo decl_payA.txt a insertar usando el plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/CLANGPlugin/test/dataToInsert/decl_payA.txt.

[141] Cancinos, José and Gonzalez, Julián, *Archivo decl_payA.txt a insertar usando el plugin de GCC*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/GCCPlugin/test/dataToInsert/decl_payA.txt.

[142] Cancinos, José and Gonzalez, Julián, *Archivo decl_payB.txt a insertar usando el plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/CLANGPlugin/test/dataToInsert/decl_payB.txt.

[143] Cancinos, José and Gonzalez, Julián, *Archivo decl_payB.txt a insertar usando el plugin de GCC*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/GCCPlugin/test/dataToInsert/decl_payB.txt.

[144] Cancinos, José and Gonzalez, Julián, *Código fuente al cual se insertarán datos usando el plugin de CLANG para su compilación*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/GCCPlugin/prueba.c.

[145] Cancinos, José and Gonzalez, Julián, *Código fuente al cual se insertarán datos usando el plugin de GCC para su compilación*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/CLANGPlugin/src/test.c.

[146] Cancinos, José and Gonzalez, Julián, *Script setup.sh para la instalación de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/CLANGPlugin/setup.sh.

[147] Cancinos, José and Gonzalez, Julián, *Script setup.sh para la instalación de GCC*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/CLANGPlugin/src/test.c.

[148] Cancinos, José and Gonzalez, Julián, *Script test.sh para realizar la compilación e inserción de datos en CLANG*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/CLANGPlugin/test.sh.

[149] The FreeBSD Project, *READELF(1) - FreeBSD General Commands Manual*. [En línea]. Disponible en: https://www.freebsd.org/cgi/man.cgi?query=readelf&sektion=1&manpath=freebsd-release-ports.

[150] Cancinos, José and Gonzalez, Julián, *Script test.sh realizar la compilación e inserción de datos en GCC*. [En línea]. Disponible en: https://github.com/josemacan/ELFBag/blob/main/GCCPlugin/test.sh.

[151] *Three address code in Compiler*, en-us, mayo de 2018. [En línea]. Disponible en: https://www.geeksforgeeks.org/three-address-code-compiler/.

[152] *GCC Frontend HOWTO: Compiler Tools*, publisher: The Linux Documentation Project. [En línea]. Disponible en: https://tldp.org/HOWTO/GCC-Frontend-HOWTO-3.html.

[153] Department of Computer Science and Engineering - Indian Institute of Technology, *Introduction to RTL*, Bombay, 2010. [En línea]. Disponible en: https://www.cse.iitb.ac.in/~uday/courses/cs715-09/gcc-rtl.pdf.

[154] Department of Computer Science and Engineering ,Indian Institute of Technology y U. Khedker, *GCC Translation Sequence and Gimple IR*, Bombay, 2010. [En línea]. Disponible en: https://www.cse.iitb.ac.in/~uday/courses/cs715-09/gcc-gimple.pdf.

[155] LLVM, *Passes in LLVM*. [En línea]. Disponible en: https://llvm.org/devmtg/2014-04/PDFs/Talks/Passes.pdf.

[156] LLVM Project, *LLVM Language Reference Manual — LLVM 12 documentation*. [En línea]. Disponible en: https://llvm.org/docs/LangRef.html.

[157] *Toolchain build procedures*. [En línea]. Disponible en: https://hugh712.gitbooks.io/embeddedsystem/content/toolchain.html.

[158] LinuxBaya, *Cross-compiler example*. [En línea]. Disponible en: https://linuxbaya.blogspot.com/2020/09/cross-platform-development-toolchain.html.

[159] J. Cancinos y J. Gonzalez, *Repositorio de trabajo*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/tree/main/PPS/GCCvsCLANGLLVM.

[160] J. Cancinos y J. Gonzalez, *Codigo fuente - ejemplo.c*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c.

[161] J. Cancinos y J. Gonzalez, *Código resultante del preprocesador en GNU GCC - ejemplo.i*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/GCC/ejemplo.i.

[162] J. Cancinos y J. Gonzalez, *Código resultante del preprocesador en CLANG/LLVM - ejemplo.i*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.i.

[163] J. Cancinos y J. Gonzalez, *CLANG/LLVM - tokens.txt*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/CLANGLLVM/tokens.txt.

[164] J. Cancinos y J. Gonzalez, *GCC - ejemplo.c.004t.original*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c.004t.original.

[165] J. Cancinos y J. Gonzalez, *CLANG/LLVM - ejemplo.ast*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.ast.

[166] J. Cancinos y J. Gonzalez, *GCC - ejemplo.c.005t.gimple*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c.005t.gimple.

[167] J. Cancinos y J. Gonzalez, *CLANG/LLVM - Readable LLVM IR - ejemplo.ll*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.ll.

[168] J. Cancinos y J. Gonzalez, *GCC - ejemplo.c.317r.final*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/GCC/ejemplo.c.317r.final.

[169] J. Cancinos y J. Gonzalez, *CLANG/LLVM - machineinstruct.txt*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/CLANGLLVM/machineinstruct.txt.

[170] J. Cancinos y J. Gonzalez, *GCC- ejemplo.s*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/GCC/ejemplo.s.

[171] J. Cancinos y J. Gonzalez, *CLANG/LLVM - ejemplo.s*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/GCCvsCLANGLLVM/CLANGLLVM/ejemplo.s.

[172] *Understanding the ELF File Format, linuxhint*, en-us. [En línea]. Disponible en: https://linuxhint.com/understanding_elf_file_format/.

[173] *What is the significance of ".comment"section in ELF?, quora*, en-us. [En línea]. Disponible en: https://www.quora.com/What-is-the-significance-of-comment-section-in-ELF.

[174] *Declaring Attributes of Functions, GNU*, en-us. [En línea]. Disponible en: https://gcc.gnu.org/onlinedocs/gcc-3.2/gcc/Function-Attributes.html.

[175] *Assembler warning with gcc warning when placing data in .text, stackoverflow*, en-us. [En línea]. Disponible en: https://stackoverflow.com/questions/58455300/assembler-warning-with-gcc-warning-when-placing-data-in-text.

[176] *Add ASCII comments to a C binary?, stackoverflow*, en-us. [En línea]. Disponible en: https://stackoverflow.com/questions/32740668/add-ascii-comments-to-a-c-binary.

[177] *How to define a string literal in gcc command line?, stackoverflow*, en-us. [En línea]. Disponible en: https://stackoverflow.com/questions/2410976/how-to-define-a-string-literal-in-gcc-command-line.

[178] Cancinos, Jose and Gonzalez, Julian, *Repositorio de trabajo*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/tree/main/PPS/ELF/GCC.

[179] Cancinos, Jose and Gonzalez, Julian, *Insercion de codigo en seccion read only data*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/ELF/GCC/String_rodata/cod_rodata.c.

[180] Cancinos, Jose and Gonzalez, Julian, *Insercion de codigo utilizando macro y atributos de C en una nueva seccion en ejecutable*. [En línea]. Disponible en: https://github.com/josemacan/PSCompiladores/blob/main/PPS/ELF/GCC/MacrosComment/code_macros.c.

[181] Cancinos, Jose and Gonzalez, Julian, *Código metadata_elf.c para probar inserción múltiple codificada*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/0c0b5489cc631646d7762b2e1f5f7e86f1c28f49/PI/ELFEncoder/src/metadata_elf.c.

[182] Cancinos, Jose and Gonzalez, Julian, *Header metadata_payloads.h con punteros y direcciones*. [En línea]. Disponible en: https://github.com/josemacan/metadataFreeBSD/blob/015721ad6e80a105f9123e5f1472f4d7ee2084f4/sys/sys/metadata_payloads.h.

[183] Cancinos, Jose and Gonzalez, Julian, *Código pluginsegundo.c con desarrollo de plugin de GCC para insertar metadatos de manera automática*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/9a930f2646a057b81fd22b879a565376011739b8/PI/MetadataGCCPlugin/PluginGCCFreeBSD/pluginsegundo.c.

[184] Cancinos, Jose and Gonzalez, Julian, *Código helloworld.c con payloads y estructuras metadata*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/27d822cb34ab7406df33048ef9da65f029fd5b68/PI/ExtractSectionCombineELF/OriginalMetadataInsert/src/main.c.

[185] Cancinos, José and Gonzalez, Julián, *Código fuente test.c del programa de usuario a ser compilado con plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/CLANGPlugin/src/test.c.

[186] Cancinos, José and Gonzalez, Julián, *Código fuente prueba.c del programa de usuario a ser compilado con plugin de CLANG*. [En línea]. Disponible en: https://github.com/josemacan/PPSCompiladoresUNC/blob/af7a40e254b630cc8954257e87948bda052d1463/PI/BinaryMetadataInsertion/GCCPlugin/prueba.c.

[187] GeeksforGeeks, *Fork() Bomb*, en-us. [En línea]. Disponible en: https://www.geeksforgeeks.org/fork-bomb/.

Anexo I

Llamada a funciones durante la carga de ejecutables ELF en el sistema

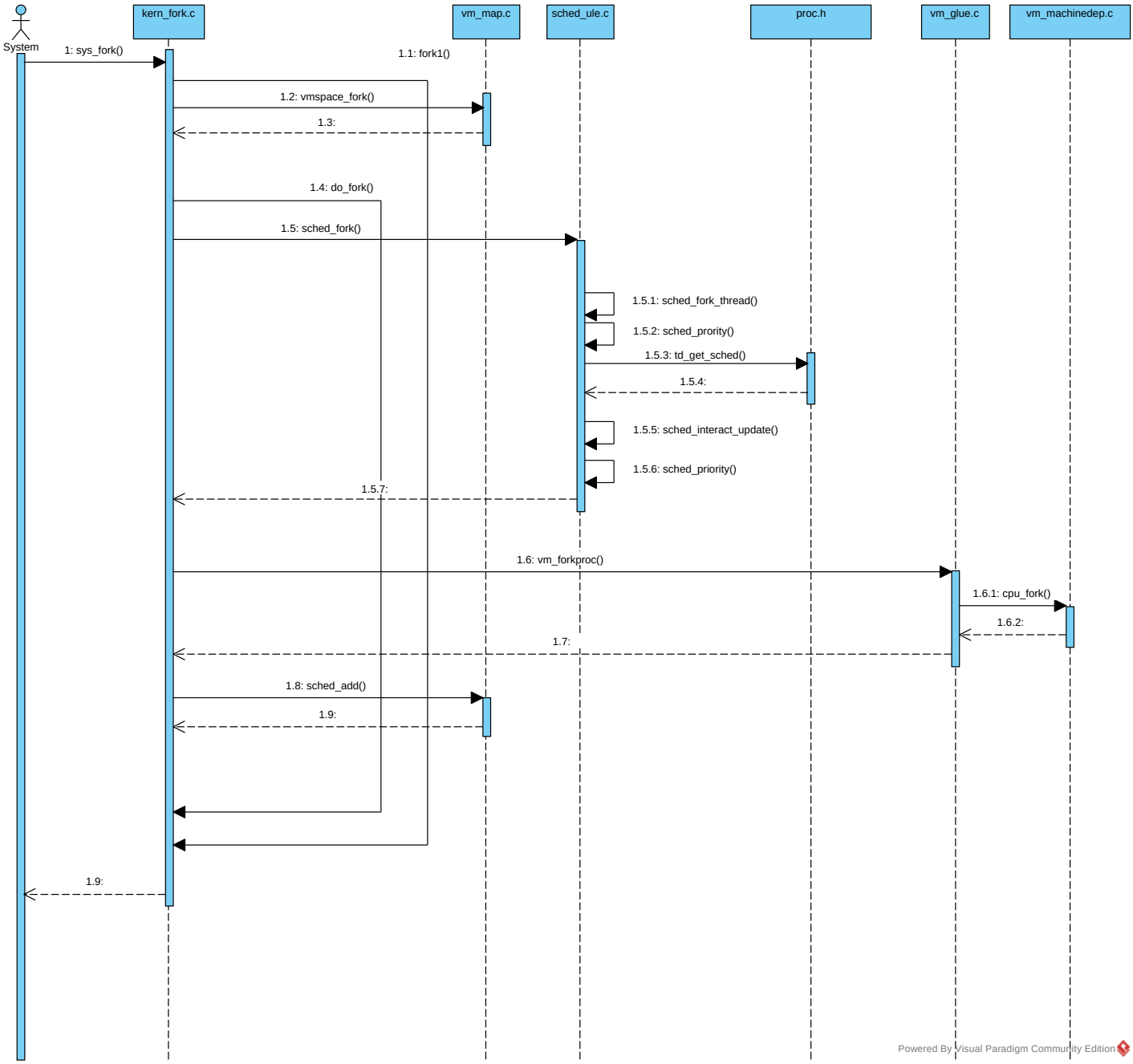
Al momento de ejecutar un programa en el sistema, se desencadena una serie de llamadas a funciones que tiene como objetivo realizar la carga del ejecutable dentro del sistema operativo y el procesamiento del mismo en espacio reservado, o de *kernel*, para poder crear la imagen del proceso del programa.

El trabajo con archivos ejecutables de formato ELF ha permitido llegar a conocer cuáles son las funciones que se llaman durante este proceso.

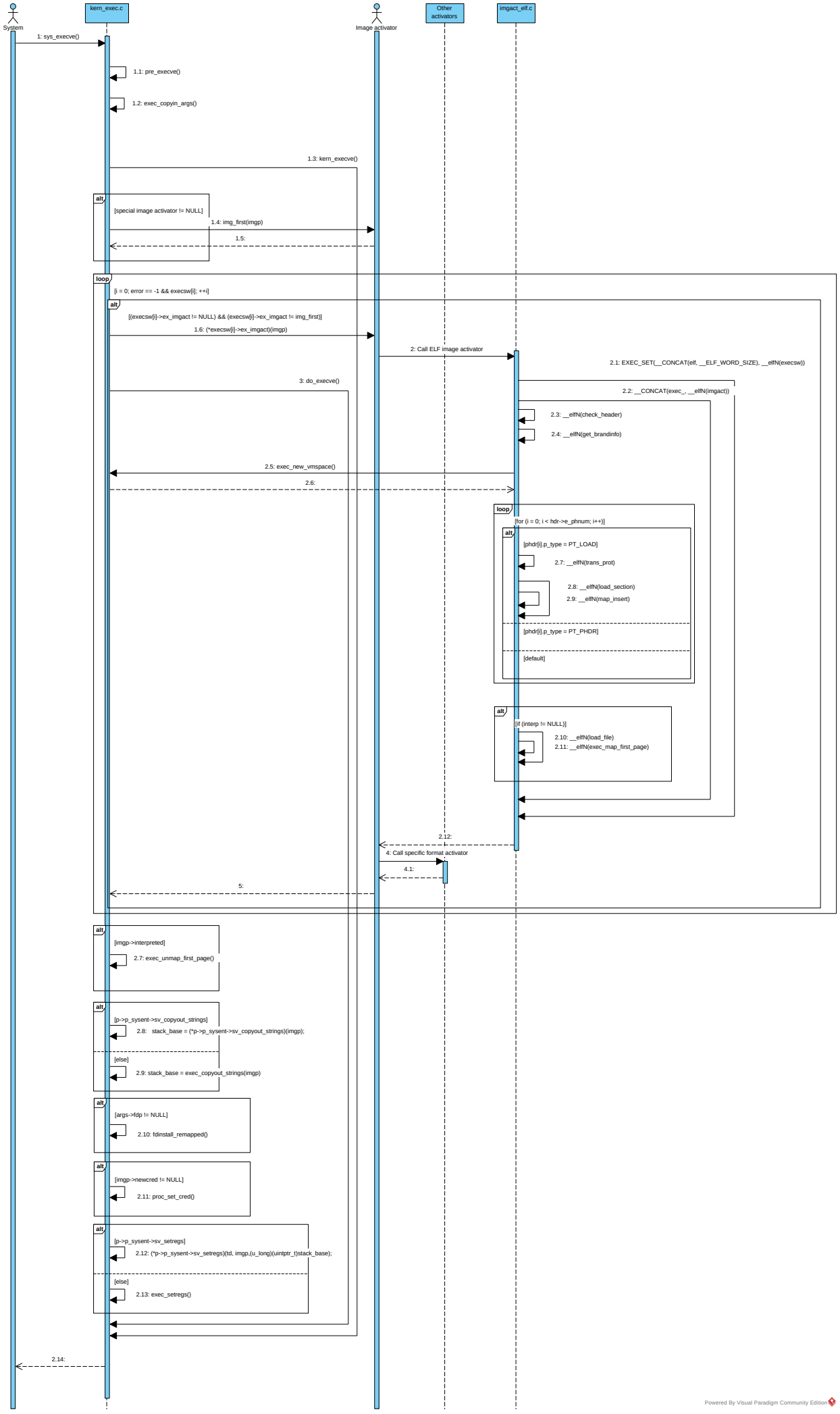
Para dar mayor ilustración de esta serie de funciones, se han creado dos diagramas de secuencias que muestran las funciones que involucran las dos llamadas a funciones de sistema que se dan al momento de ejecutar un programa: `fork()` y `exec()`.

Cabe aclarar que no se presenta la totalidad de funciones que se invocan, si no que, se presentan aquellas que se han evaluado como relevantes para la obtención de la imagen final del proceso en memoria del sistema.

I.1. Llamada a función de sistema `fork()`


System
kern_fork.c
vm_map.c
sched_ule.c
proc.h
vm_glue.c
vm_machinedep.c
1: sys_fork()
1.1: fork1()
1.2: vmspace_fork()
1.3:
1.4: do_fork()
1.5: sched_fork()
1.5.1: sched_fork_thread()
1.5.2: sched_prority()
1.5.3: td_get_sched()
1.5.4:
1.5.5: sched_interact_update()
1.5.6: sched_priority()
1.5.7:
1.6: vm_forkproc()
1.6.1: cpu_fork()
1.6.2:
1.7:
1.8: sched_add()
1.9:
1.9:
Powered By Visual Paradigm Community Edition

I.2. Llamada a función de sistema `exec()`



Powered By Visual Paradigm Community Edition

Anexo II

Preparación del entorno de desarrollo para FreeBSD

II.1. Compilación e instalación del *kernel* de FreeBSD

Para poder compilar el *kernel* de FreeBSD, es necesario seguir los siguientes pasos:

1. Descargar el código fuente del *kernel* de FreeBSD. Al momento de presentación de este informe, el repositorio recomendado por el proyecto FreeBSD se accede en https://github.com/freebsd/freebsd-src. Seleccionar la rama con la versión deseada.

   ```
   git clone -b <rama> --single-branch git@github.com:freebsd/freebsd-src.git
   ```

2. Copiar los archivos descargados al directorio `/usr/src`.

   ```
   cp -r <dir descargado> /usr/src
   ```

3. Dirigirse al directorio de kernel `/usr/src/sys/amd64/conf`.

   ```
   cd /usr/src/sys/amd64/conf
   cp GENERIC KERNELCONF
   ```

4. Dentro del directorio `/usr/src`, compilar el *kernel*.

   ```
   make buildkernel KERNCONF=KERNELCONF
   ```

5. Instalar el *kernel* compilado.

   ```
   make installkernel KERNCONF=KERNELCONF
   ```

II.2. Compilación e instalación del parche de *kernel* de FreeBSD

Para compilar e instalar el parche de *kernel* desarrollado en este proyecto, se deben seguir las siguientes instrucciones.

1. Descargar el código fuente de *kernel* que contiene el desarrollo realizado.

```
# Para descargar parche para FreeBSD releng 13
git clone git@github.com:josemacan/metadataFreeBSD_releng13.git
# Para descargar parche para FreeBSD releng 12.3
git git@github.com:josemacan/metadataFreeBSD_releng12.3.git
```

2. Copiar los archivos descargados al directorio `/usr/src`.

```
# Para FreeBSD releng 13
cp -r metadataFreeBSD_releng13/ /usr/src
# Para FreeBSD releng 12.3
cp -r metadataFreeBSD_releng12.3/ /usr/src
```

3. Dentro del directorio `/usr/src`, compilar el *kernel*.

```
make buildkernel KERNCONF=KERNELCONF
```

4. Instalar el *kernel* compilado.

```
make installkernel KERNCONF=KERNELCONF
```

II.3. Recompilado rápida e instalación de *kernel* de FreeBSD

Para llevar a cabo la recompilación rápida del *kernel* de FreeBSD, impactando solamente en aquellos módulos del sistema operativo que son alcanzados por los cambios, se recomienda ejecutar los siguientes comandos.

Para la recompilación rápida:

```
make NO_KERNELCLEAN=yes NO_KERNELDEPEND=yes MODULES_WITH_WORLD=yes
↪ buildkernel KERNCONF=KERNELCONF
```

Para su posterior instalación:

```
make installkernel KERNCONF=KERNELCONF
```